Luxor university

Faculty of Computer and Information

Computer Science Department

The Management Intelligent System For Managing Individuals' Residence in Cities

Graduation Project - Part 2
2024/2025

Under Supervision
Dr. Bassem Abd-El-Atty

Submitted by

Mohamed Badawy Sayed

Mostafa Ahmed Hasan

Mahmoud Ahmed Khalil

Ahmed Nageh Abbas

Nada Maaman Abdel Karim

Basant Benyamen Barsoum

Abstract

The Management Intelligent System For Managing Individuals' Residence in Cities, known as Semsary, is a web-based system designed with .NET, React, and Python (for machine learning integrations) to address the complex demands of short-term rental management in urban settings. Tailored for tenants, landlords, and families, Semsary streamlines the apartment rental experience by consolidating reliable listings, automating price assessments, and providing a secure booking environment.

Semsary tackles key challenges such as providing precise rental information, ensuring fair pricing, and connecting users to essential amenities. Through machine learning analysis, the platform recommends optimal rental prices and offers feedback to landlords on market standards, promoting transparency and helping renters make informed choices. This data-driven approach minimizes risk of fraud and ensures fair valuation, enabling users to find accommodations that align with their preferences and budget constraints.

Beyond a listing repository, Semsary's intuitive interface allows users to filter apartments by features like proximity to transportation and schools. Verified reviews foster informed decision-making, and location-based insights offer an enriched user experience. Secure, streamlined booking and payment options further simplify the rental process.

Semsary also features a dynamic administrative layer where managers handle advertising approvals, resolve disputes, and review tenant reports. Dedicated ID verification for both local and international users enhances trust and security on the platform. In cases of dissatisfaction, tenants can request refunds or rate properties, supporting accountability and enhancing service standards.

In conclusion, Semsary is positioned to drive positive change in urban rental management. By centralizing resources, integrating machine learning for equitable pricing, and enhancing user security and convenience, Semsary fosters a sustainable rental environment that aligns with the goals of sustainable urban growth and community well-being.

Table of contents

List of Abbreviations

Abbreviation	Meaning
SDGs	Sustainable Development Goals
AI	Artificial Intelligence
ML	Machine Learning
API	Application Programming Interface
SMS	Short Message Service
JWT	JSON Web Token
SQL	Structured Query Language
HTTPS	Hypertext Transfer Protocol Secure
TLS	Transport Layer Security
XSS	Cross-Site Scripting
CI/CD	Continuous Integration/Continuous Deployment
HMR	Hot Module Replacement
SDK	Software Development Kit
OTP	One-Time Password
OCR	Optical character recognition
ERD	Entity Relationship Diagram

List of tables

List of Figures

Chapter 1

System Overview

1.1 Motivation

In today's world, people frequently move to different cities for various reasons, such as pursuing university studies, new work opportunities, vacations, and more .In addition to the increase in the number of expatriates to all governorates of Egypt in the recent period. Despite their varied purposes, one essential need unites them: finding a suitable place to live.

For many of these individuals, especially expatriates and students, the rental process can be challenging. Options for short-term, flexible, and furnished rentals are often limited, and securing a safe, well-priced residence in an unfamiliar city is not always straightforward.

Our project addresses this need with a platform designed to support sustainable cities and communities. By enabling homeowners to list available rentals and allowing tenants to compare and book our solution simplifies the rental process for everyone involved. In doing so, we aim to facilitate a more accessible, trustworthy, and sustainable rental market that empowers individuals and contributes to local community development.

Our motivation stems from the urgent need to transform our countries into sustainable cities and communities. While there are many factors to achieve this goal, developing the process of renting apartments to make it easier and smarter is one of the first steps to achieve this goal.

1.2 Problem Statement

Ineffectiveness of available methods of displaying available apartments, making it difficult for tenants to find the best match for their needs. Without clear information on prices, apartment contents, and nearby facilities, tenants struggle to make informed decisions about where to live.

Exploitation by brokers of both the landlord and the tenant, the get a commission from both of them in exchange for bringing the tenant to the landlord. They also often use misleading information to convince the tenant of the apartment.

There is a lack of effective methods to ensure that the tenant's expectations are met, which exposes him to the risk of receiving an apartment that does not conform to the agreed upon specifications or even that this apartment does not actually exist on the ground. This discrepancy between what was promised and what was delivered can lead to dissatisfaction and trust issues in the rental process.

Many tenants face challenges in obtaining real, transparent feedback about apartments, leaving them vulnerable to misinformation or deceptive practices. Without reliable reviews or insights, tenants cannot confidently evaluate the quality of an apartment before committing to a rental.

Landlords often struggle to reach a large and relevant audience of potential tenants, limiting their ability to fill available apartments efficiently. Without effective tools to market their properties, landlords miss out on valuable opportunities to connect with suitable tenants who may be actively searching for housing options.

1.3 Overview

Our project, **Semsary**, is a unique web platform designed to simplify the management of short-term rentals in cities. Our website will serve as a reliable hub for finding verified apartment listings, learning about fair pricing, and exploring nearby essential services.

We prioritize key aspects such as secure rental options, transparency, and verified reviews to make renting safer and more convenient. The platform will feature tools for recommending optimal rental prices, providing location-based insights to support better decision-making, and enabling seamless communication between landlords and tenants. Users will also benefit from advanced filtering options to easily find properties that meet their needs.

Our goal is to streamline the rental process for tenants, landlords, and families by creating a trustworthy and user-friendly environment. With secure payment methods and mechanisms for handling reviews and disputes, we aim to deliver a smooth, reliable, and satisfying rental experience for everyone.

1.4 Objectives and scope

Our system, Semsary, is strategically designed to address key challenges in the short-term rental market, offering a streamlined, user-friendly, and data-driven system to facilitate apartment management and booking. By focusing on efficiency, security, and transparency, Semsary aims to revolutionize the urban rental experience for students, families, working individuals, and property owners alike.

Our primary objectives center on enhancing user experience, enabling data-backed rental pricing, simplifying booking processes, and ensuring informed decision-making. Through comprehensive apartment listings that detail essential features, amenities, and locations, Semsary empowers users to make well-informed choices, significantly reducing time spent on property searches. With advanced filtering options, users can effortlessly tailor their search to specific needs, including room count, rental type, and proximity to essential services such as transportation, schools, and markets.

Semsary also integrates machine learning capabilities to evaluate and recommend optimal rental prices based on input data, including room specifications, nearby services, and market trends. This approach provides landlords with pricing guidance aligned with market standards, while giving tenants confidence in fair and competitive pricing. The platform's data-driven pricing analysis supports transparency, minimizes the risk of fraud, and promotes financial fairness within the rental ecosystem.

In addition, the system enables seamless booking and secure payment options, which are essential for a smooth user experience. Property owners benefit from efficient request management, allowing them to review and respond to booking requests while ensuring exclusivity for each listing. A transparent request process also allows users to preview and cancel bookings within a specified time-frame, with funds held securely until completion or cancellation.

Semsary's comprehensive scope includes account creation, user interest customization, detailed transaction management, and a dynamic advertisement system, all contributing to an ecosystem that prioritizes sustainable urban living. By offering premium accounts, paid advertisements, and cancellation fees, Semsary not only enhances user experience but also ensures a sustainable revenue model.

1.5 Target Audience and Expected Impact

Our primary target audience includes Egyptian students, expatriate students, workers, Egyptian and expatriate families who are relocating to new cities for work, education, or personal reasons. These individuals often face challenges in finding short-term, flexible, and furnished rental accommodations that suit their needs. Additionally, landlords looking to rent out apartments or rooms to a wide range of tenants will also benefit from the platform. This includes homeowners in busy urban areas who require an easy and effective way to connect with potential tenants.

The expected impact of this platform is twofold: for tenants, it will make the process of finding reliable, affordable, and flexible housing options faster and more transparent, reducing the time and effort spent searching for suitable homes. For landlords, it will offer a more efficient way to connect with a wider pool of tenants, increasing the likelihood of renting out properties quickly. In the long term, the platform will contribute to building more sustainable communities by making housing access more equitable and transparent, promoting urban mobility, and supporting sustainable urban development.

Chapter 2

Related Works

2.1 Introduction

The rapid evolution of the digital economy has profoundly reshaped traditional industries, with the real estate and hospitality sectors standing out as prime examples of this transformation. Over the past decade, innovative online platforms have disrupted conventional methods of property search, booking, and accommodation, offering users greater flexibility, transparency, and accessibility. These platforms have not only addressed longstanding inefficiencies in their respective markets but have also redefined consumer expectations, setting new standards for convenience, personalization, and sustainability. From global giants like Airbnb to regional leaders such as Property Finder, Aqarmap, Bayut, and Aqar Ya Masr, these companies have demonstrated the power of technology to create value for both users and businesses. By leveraging peer-to-peer models, advanced data analytics, and user-centric designs, they have successfully bridged gaps between supply and demand, fostering trust and efficiency in markets that were once dominated by intermediaries and opaque practices.

Airbnb, for instance, revolutionized the hospitality industry by introducing a global marketplace that connects hosts and travelers, offering unique accommodations and localized experiences. Similarly, Property Finder and Bayut have become indispensable tools in the UAE's real estate market, providing comprehensive property listings and innovative solutions for buyers, sellers, and agents. In Egypt, platforms like Aqarmap and Aqar Ya Masr have addressed the challenges of a rapidly growing urban population by simplifying property searches and offering transparent, data-driven insights. These platforms have not only enhanced the user experience but have also contributed to community development and sustainable urban living. Their success stories highlight the importance of adapting to local market dynamics while maintaining a global perspective on innovation and scalability.

Against this backdrop, **Semsary** emerges as an innovative solution tailored to the unique challenges of Egypt's rental market. By focusing on short-term, flexible, and furnished accommodations, **Semsary** aims to streamline the rental process for tenants and landlords alike, addressing issues such as broker exploitation, lack of transparency, and limited access to accurate information. The platform's user-friendly design, advanced search capabilities, and commitment to sustainability position it as a potential game-changer in the region. This chapter delves into the related works of prominent platforms in the real estate and hospitality sectors, examining their business models, technological advancements, and community impacts. By analyzing their strategies and achievements, we aim to provide a comprehensive context for understanding Semsary's mission and its potential to redefine the rental market in Egypt. Through this exploration, we seek to highlight the lessons learned from existing platforms and illustrate how **Semsary** can build upon their successes to create a more equitable, efficient, and sustainable housing ecosystem.

2.2 Airbnb

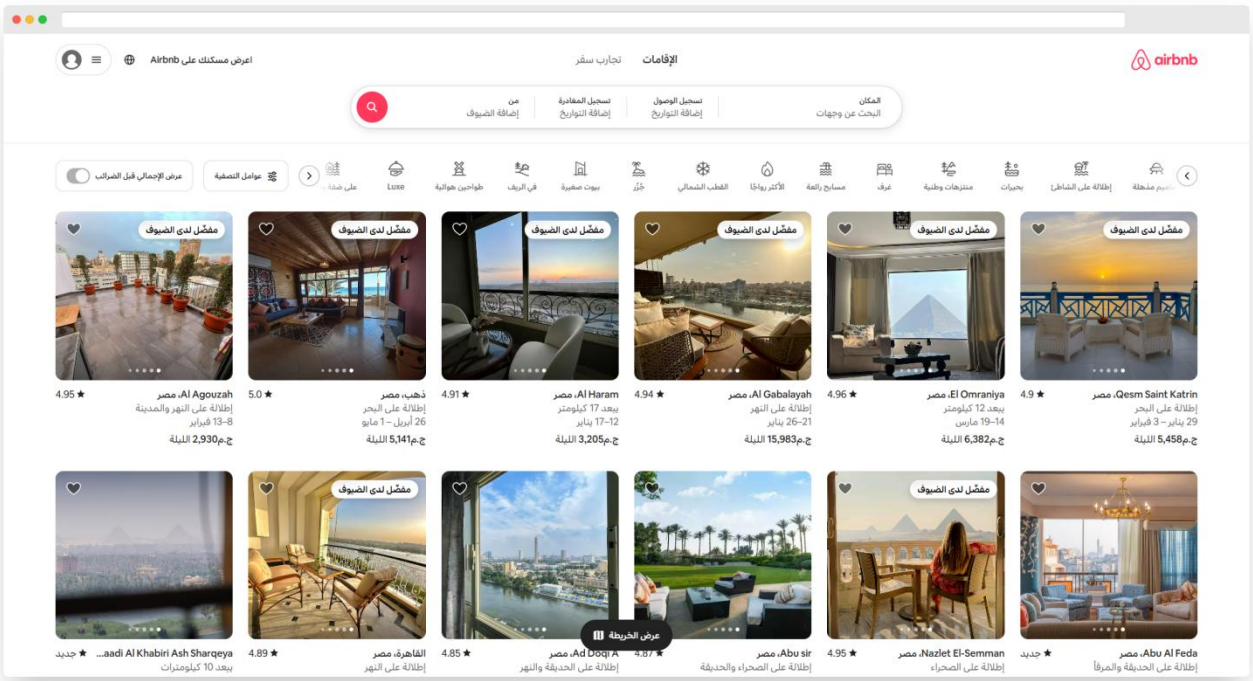


Figure 1 : "Airbnb" Website Home Page Banner Image

Airbnb[1] is a global online marketplace, founded in 2008 in San Francisco, that connects hosts offering short-term rentals with travelers seeking diverse accommodations, from shared spaces to unique stays like castles and yurts. Operating in over 220 countries, Airbnb revolutionized travel by providing flexible and personalized lodging options.

In addition to accommodations, Airbnb offers "Experiences," enabling travelers to participate in local activities hosted by residents, enhancing cultural immersion. The platform's peer-to-peer model charges service fees to both hosts and guests, creating a sustainable revenue stream.

Through innovative technology, transparency, and a focus on community and sustainability, Airbnb has transformed the hospitality industry, fostering global connections and redefining how people travel and explore the world.

Business Model

Airbnb uses a peer-to-peer model. It charges service fees to both hosts and guests for bookings made through the platform. Hosts benefit from a global audience for their properties, while guests enjoy a variety of accommodations often at more competitive prices than hotels.

[1] https://www.airbnb.com

2.3 Propertyfinder

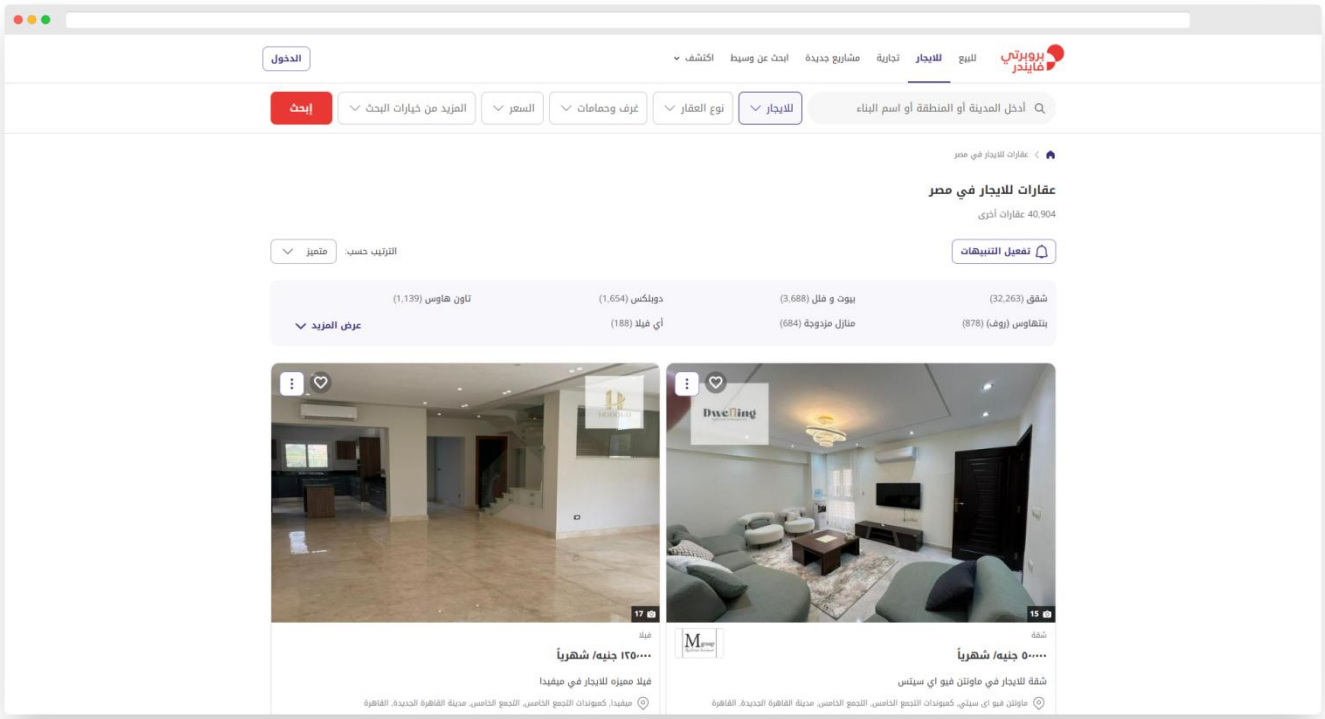


Figure 2 : "PropertyFinder" Website Home Page Banner Image

Property Finder[2] was founded by Michael Lahyani in Dubai, initially launching as a printed real estate magazine called "Al Bab World" in 2007. The platform transitioned to a digital real estate portal, propertyfinder.ae, which quickly became the leading platform in the UAE.

Community Impact

Property Finder emphasizes community growth by offering innovative solutions that enhance the home-buying journey and foster sustainable real estate practices.

Business Model

Property Finder operates under a commission-based model, where it generates revenue by charging real estate agents and developers a fee to list properties on its platform. Additionally, the platform offers premium services for agents, such as "SuperAgent" and marketing tools. The company also monetizes through advertisements and offers data insights for industry professionals. This model enables Property Finder to provide value to both users and real estate professionals while continuously investing in new technology and services.

[2] https://www.propertyfinder.eg

2.4 Aqarmap

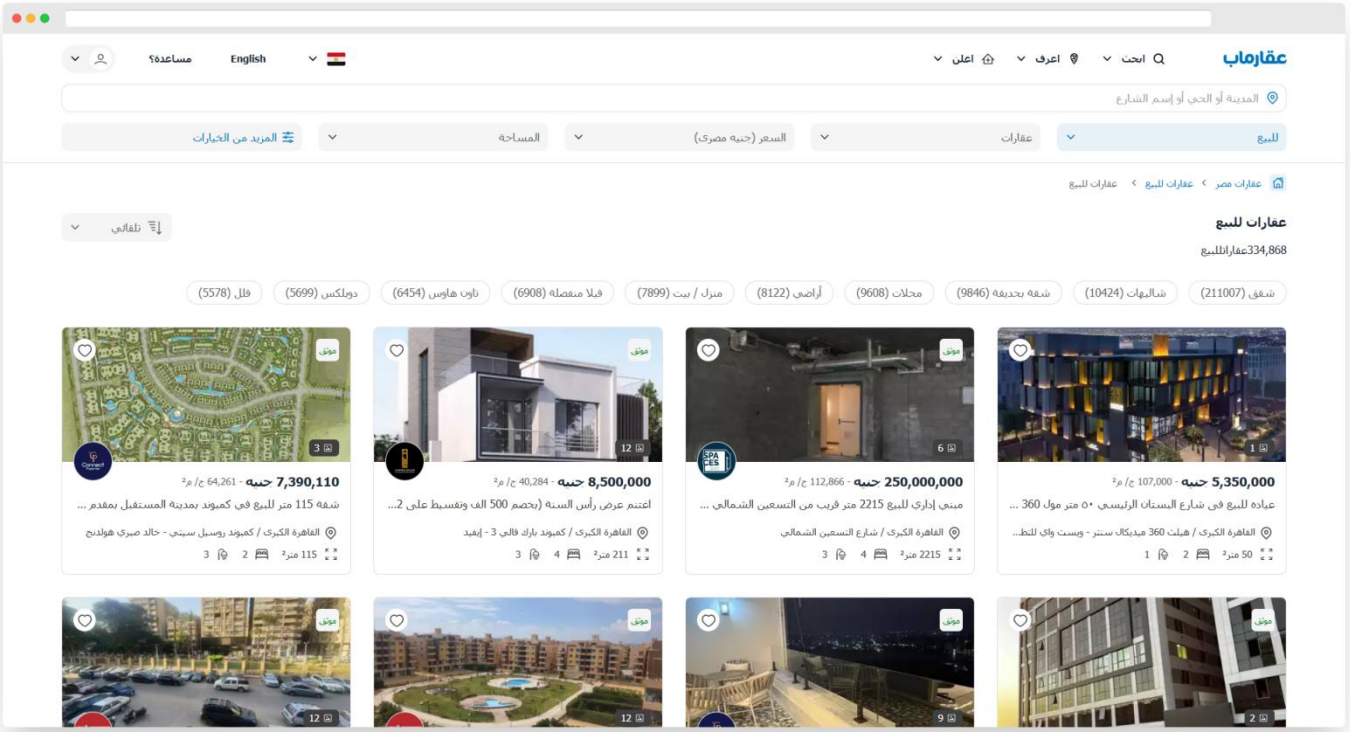


Figure 3 : "Aqarmap" Website Home Page Banner Image

Aqarmap[3] is the first Arabic real estate marketing platform, launched in 2010 by Imad Al-Masoudi. It offers a comprehensive platform for buying, selling, and renting real estate across Egypt, with more than 300,000 properties and over 1,000 compounds listed. The platform serves over 2 million visitors monthly and provides detailed, up-to-date information about properties and developments, helping users make informed decisions.

Business Model

Aqarmap generates revenue through a commission-based model, where real estate developers, property owners, and agents pay to list their properties on the platform. They offer additional paid services such as premium listings, advertising, and market research for real estate companies. The platform also partners with banks and financial institutions to provide users with access to financing options. Furthermore, Aqarmap monetizes through its real estate expos by charging developers and exhibitors to showcase their properties.

[3] https://aqarmap.com.eg

2.5 Bayut

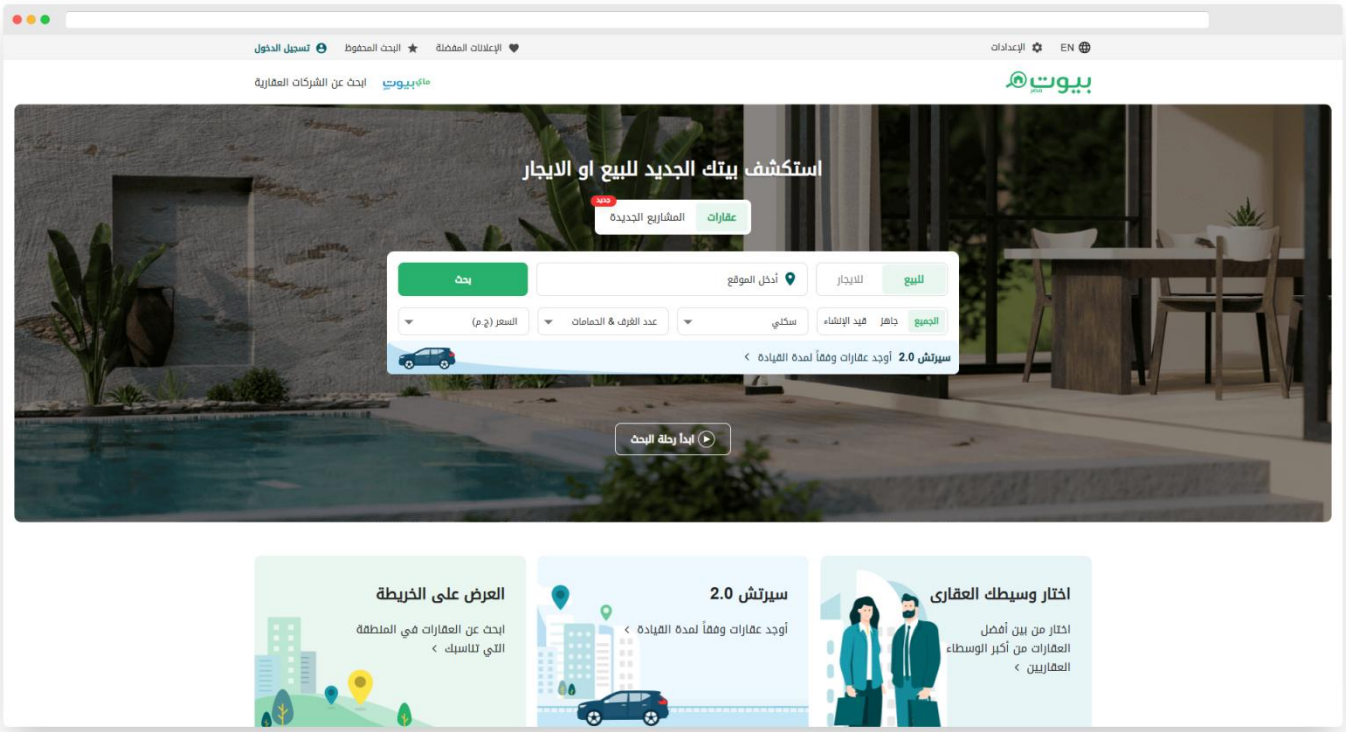


Figure 4 : "Bayut" Website Home Page Banner Image

Bayut[4] is a leading property listing platform in the United Arab Emirates, connecting buyers, investors, property owners, sellers, tenants, and agents in a seamless and user-friendly manner. The company is part of Dubizzle Group Holdings Limited, one of the first billion-dollar companies in the UAE, and operates under the motto "Stronger Together" in partnership with Dubizzle, the leading classifieds website in the UAE.

Business Model

Bayut operates on a freemium business model, where property owners and real estate agents can list properties for free or choose to pay for premium listings for greater visibility. The platform generates revenue through advertisements, subscription plans, and featured listings, offering additional exposure and tools for agents and developers. The company's strategic partnerships with investors, developers, and financial institutions further strengthen its presence in the real estate market.

[4] https://www.bayut.com

2.6 Aqaryamasr

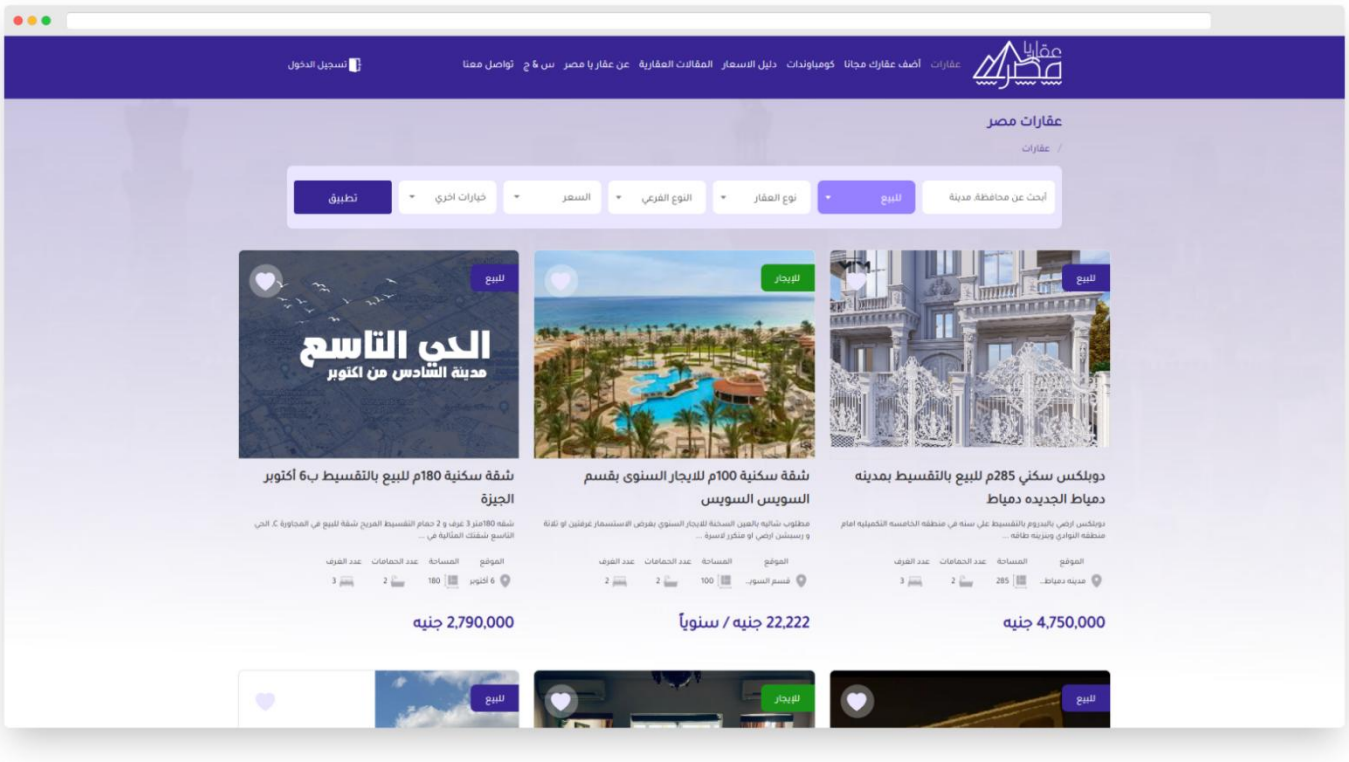


Figure 5 : "Aqaryamasr" Website Home Page Banner Image

The real estate market in Egypt continues to expand rapidly, despite the significant urban development happening across the country. However, many Egyptians still face difficulty finding ideal residential properties at reasonable prices, whether for sale or rent.

To solve this problem, we created AqarYaMasr[5] —a platform designed to bridge the gap between buyers' needs and what's available in the market, without the involvement of brokers. The platform offers a powerful search engine that allows users to set specific criteria for their ideal property. Once the criteria are entered, the platform quickly filters through listings, displaying only those that match the buyer's specifications. This offers flexibility and freedom of choice, ensuring users can find the best available options.

One of the key features of our advanced search is its ability to provide an overview of current property prices and detailed information on residential, commercial, or administrative units in the selected area. This transparency is crucial for anyone looking to move to a new city, as it helps compare different areas for living, setting up a business, or finding a suitable office, all based on accurate market data.

[5] https://aqaryamasr.com

2.7 Aspects of our project

Semsary is an innovative online platform designed to transform the rental market in Egypt by addressing the unique challenges faced by tenants and landlords. The platform simplifies the process of finding and renting apartments, particularly for short-term, flexible, and furnished accommodations. By leveraging technology, Semsary provides a streamlined and user-friendly experience that empowers individuals to make informed decisions based on detailed property listings, transparent reviews, and data-driven insights.

Semsary's broader goal is to support sustainable urban living by promoting equitable access to housing and encouraging community development. The platform uses advanced technologies, including data analytics and machine learning, to create a transparent and fair ecosystem for all users. By prioritizing trust, accessibility, and sustainability, Semsary aims to redefine the rental market and address the inefficiencies of traditional methods, such as broker exploitation and lack of accurate information.

With a scalable design, Semsary targets a diverse audience, including Egyptian students, expatriates, families, and working professionals. It seeks to make the process of finding and renting apartments faster, more reliable, and less stressful, while also helping landlords maximize their property's visibility and profitability. By fostering transparency, reducing friction, and ensuring fair pricing, Semsary aspires to revolutionize the rental market and contribute to the development of smarter, more sustainable cities.

Business Model

Semsary operates on a freemium business model, combining free access to essential services with optional paid features to ensure long-term financial sustainability. Key revenue streams include:

1. **Premium Accounts**: Offering enhanced features such as priority listing placement, detailed analytics, and advanced marketing tools for landlords and property managers.
2. **Paid Advertisements**: Allowing landlords to promote their listings prominently on the platform, ensuring higher visibility to potential tenants.
3. **Service Fees**: Charging a nominal fee for successful bookings to cover platform maintenance and development.
4. **Cancellation Fees**: Implementing a fee for last-minute booking cancellations, ensuring commitment and reducing misuse of the system.
5. **Data Insights**: Providing anonymized market insights and rental trend reports to stakeholders in the real estate market.

By integrating these revenue streams, Semsary ensures a robust financial foundation while delivering value to its users and fostering trust in the rental ecosystem. The business model not only supports operational growth but also aligns with the platform's mission to create equitable, sustainable, and user-friendly housing solutions.

Chapter 3

Domain Analysis and Technique

3.1 Domain Analysis

3.1.1 General knowledge about the domain

1) Efficient Rental Management Module:
This module focuses on addressing the inefficiencies and challenges in the short-term rental market for individuals and families, particularly students and workers. It aims to create a seamless platform for finding, filtering, and rental apartments while reducing time and effort spent searching for suitable accommodations.

2) Advanced Search and Filtering Module:
This module provides comprehensive search functionality, enabling users to filter apartments based on their preferences, such as the number of rooms, type of rent, proximity to schools, workplaces, and access to amenities like markets and transportation hubs.

3) User Experience Enhancement Module:
This module work through an intuitive and user-friendly interface, the platform ensures a smooth experience for all users. Features include account creation, customized interest pages, secure payment options, and location-based recommendations for nearby amenities.

4) Pricing Analysis and Transparency Module:
This module incorporates machine learning techniques to provide fair and transparent pricing for apartments. It evaluates data such as room specifications, services, and market trends to recommend optimal pricing to landlords and tenants.

5) Fraud Prevention and Reliability Module:
This module offering detailed listings, verified user reviews, and secure payment gateways, the platform minimizes the risk of fraud and ensures reliable transactions between landlords and tenants.

6) Support for Landlords Module:
This module provide to Landlords the ability to easy manage their properties, view rental requests, and adjust prices based on system recommendations. The platform also assists in assessing rental demand and promoting properties through advertisements.

7) Administrative Oversight Module:
This module gives system administrators and customer service the responsibility for content moderation, advertisement approvals, and user account management. They also handle reports, disputes, and ensure the platform operates efficiently and reliably.

8) Community Engagement Module:
This module encourages users to share experiences, provide feedback, and contribute to a repository of reviews. This fosters a collaborative environment where tenants can learn from each other's insights and make informed decisions.

3.1.2 Clients and users

1. **Users:**

Customer Service: is responsible for handling accepting or rejecting requests to add a house from tenants, confirming the inspection, adding the house data after completing the inspection, reviewing all complaints from tenants on the site, and taking the necessary action.

Administrator: is responsible for managing customer service agents by adding their accounts and distributing login credentials, overseeing system operations including dispute resolution, advertisement approvals and ensuring overall platform integrity and compliance.

2. Clients:

Tenants: they can use the platform to search for a house suitable for their specifications, interests, and money, rent a house during the appropriate period for them, and cancel the rental request during the period allowed for them to cancel. They must confirm arrival to the house in order to recover their money. They can use the platform to send a complaint in the event of any change in the specifications of the house or they were deceived and can set rate or comments.

Landlords: They are responsible for requesting to add a house, and after completing the inspection, they can review the house, publish the house, update the house's information, or delete the house. They are responsible for accepting or rejecting rental requests from tenants and confirming tenants' arrival to the house.

3.1.3 The Environment

Web-Based Application:

- **Framework:** Utilizes React JS and Tailwind for the frontend, providing a responsive and interactive user interface.
- **Backend Operations:** .Net, Sql server for server-side operations, ensuring efficient data processing and management.

Server Infrastructure:

- **Hosting Services:** Accommodates databases, applications, and essential services crucial for user interaction and data storage
- **ASP.NET Integration:** Optimized for efficient database interactions and LINQ queries for optimized data access.

Cloud Services and Networking:

- **Google Translation API:** used to translate dynamic English content to Arabic. (For static content, we use I18Next Library provided by React.)
- **Firebase Storage:** used to store high-resolution images of rental properties uploaded by landlords.

Development and Testing:

- **GitHub:** version control system that helps us to apply Agile Development in the application, makes it flexible and easy to update by any team member.
- **Google Analytics**: for user behavior tracking and Firebase Performance monitoring for real-time insights.
- **Google Lighthouse:** Web Application Evaluation tool that is used to check the performance, accessibility, and other important properties of the website

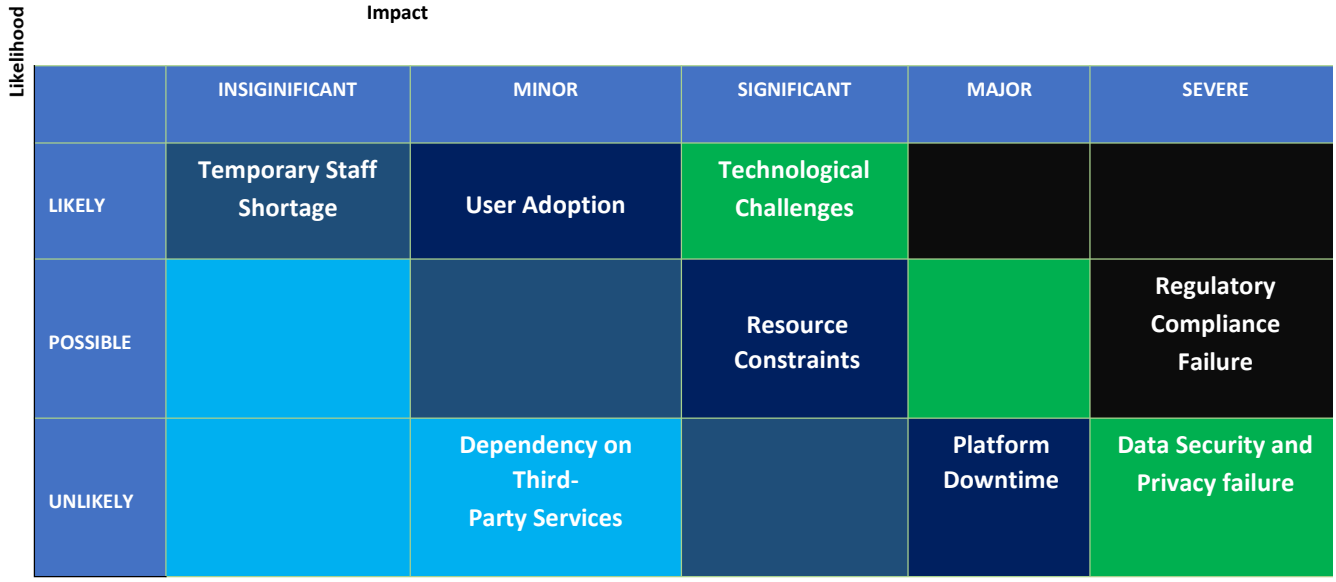
3.2 Risks

Risk Management Plan for Platform Development

Risk	Description	Impact	Mitigation
Regulatory Compliance Failure	Mishandling or improper storage of tenant or landlord data.	Loss of user trust, leading to decreased platform adoption and retention.	Implement a privacy policy outlining data collection, storage, and sharing practices.
Technological Challenges	Compatibility issues between frontend and backend, leading to functionality problems.	System malfunctions, slow performance, and potential downtime affecting user experience	Conduct rigorous testing at each development phase. Allocate additional resources for tech compatibility checks and system integration.
Resource Constraints	Insufficient budget allocation or limited skilled workforce for development and maintenance	Delays in project timelines, compromised quality, and incomplete feature implementation	Prioritize project requirements and allocate resources accordingly. Consider outsourcing for specialized skills if necessary
User Adoption	Low user engagement and participation within the platform's community and forums	Reduced knowledge sharing, lack of collaboration, and limited platform utilization	Implement incentive programs to encourage user contributions. Enhance user experience and interface for better engagement
Data Security and Privacy failure	Breaches in data security or failure to comply with privacy regulations	Compromised user data, legal implications, loss of trust, and damage to reputation	Implement robust encryption methods, regular security audits, and compliance checks. Train staff on data handling protocols
Dependency on Third-Party Services	Reliance on external cloud services or APIs for platform functionalities	Service disruptions, limited control over downtime, and potential data loss	Diversify service providers, ensure contingency plans for service outages, and regular data backups
Platform Downtime	Unplanned interruptions in platform accessibility due to technical issues or maintenance	Disruptions could hinder user experience and engagement temporarily	Regularly schedule maintenance during off-peak hours. Implement robust backup and recovery mechanisms to minimize downtime
Temporary Staff Shortage	Short-term unavailability of specific skilled staff due to unforeseen circumstances (e.g., sickness, temporary leave).	Could cause minor delays or shifts in responsibilities.	Cross-train team members, maintain clear documentation of responsibilities, and have contingency plans for temporary staff shortages

Table 1: Risk Management Plan

Risk Assessment Matrix



3.3 Constrains

Regulatory compliance

consent management: Explicitly obtain user consent for data collection and inform users about how their data will be used.

short-term rental laws: Adhere to city- or country-specific laws governing short-term rentals, such as maximum stay durations or landlord registration requirements.

Technological limitations

high traffic and load management: Handling increased user traffic, especially during peak rental seasons or for popular listings, might strain the system. Scaling a .net and react-based system dynamically to accommodate fluctuating demand can be challenging.

machine learning latency: Real-time pricing recommendations and market analysis powered by python-based machine learning could face delays, especially if large datasets are involved or the server lacks sufficient computational power.

model maintenance: Constantly updating models to reflect changing market conditions is resource-intensive and requires skilled personnel.

Resource constraints

human resources skilled personnel: Developing and maintaining a platform using .net, react, and python (for machine learning) requires expertise in multiple programming languages and frameworks, which may be hard to source or expensive.

operational costs: Hosting, scaling infrastructure, and integrating third-party apis (e.g., id verification, payment gateways) can be expensive.

Time constraints

integration delays: Time-consuming processes for integrating third-party apis and ensuring compatibility across different technologies.

iterative testing: Thorough testing for security, performance, and user experience often takes longer than anticipated, delaying deployment.

Security considerations

breach prevention: Robust cybersecurity measures are essential to prevent unauthorized access, data theft, or ransomware attacks.

review manipulation: Preventing false reviews or ratings that could undermine trust.

open-source dependencies: Ensuring all libraries and frameworks used are up-to-date and free from known vulnerabilities.

User adoption and engagement

intuitive design: A clean, responsive, and user-friendly interface that guides users smoothly through the rental process.

streamlined workflows: Simple search, filter, and rental processes that require minimal effort from users.

Device compatibility: Optimized performance on both desktop and mobile devices.

3.4 Project plan

3.4.1 Project plan Gannt chart

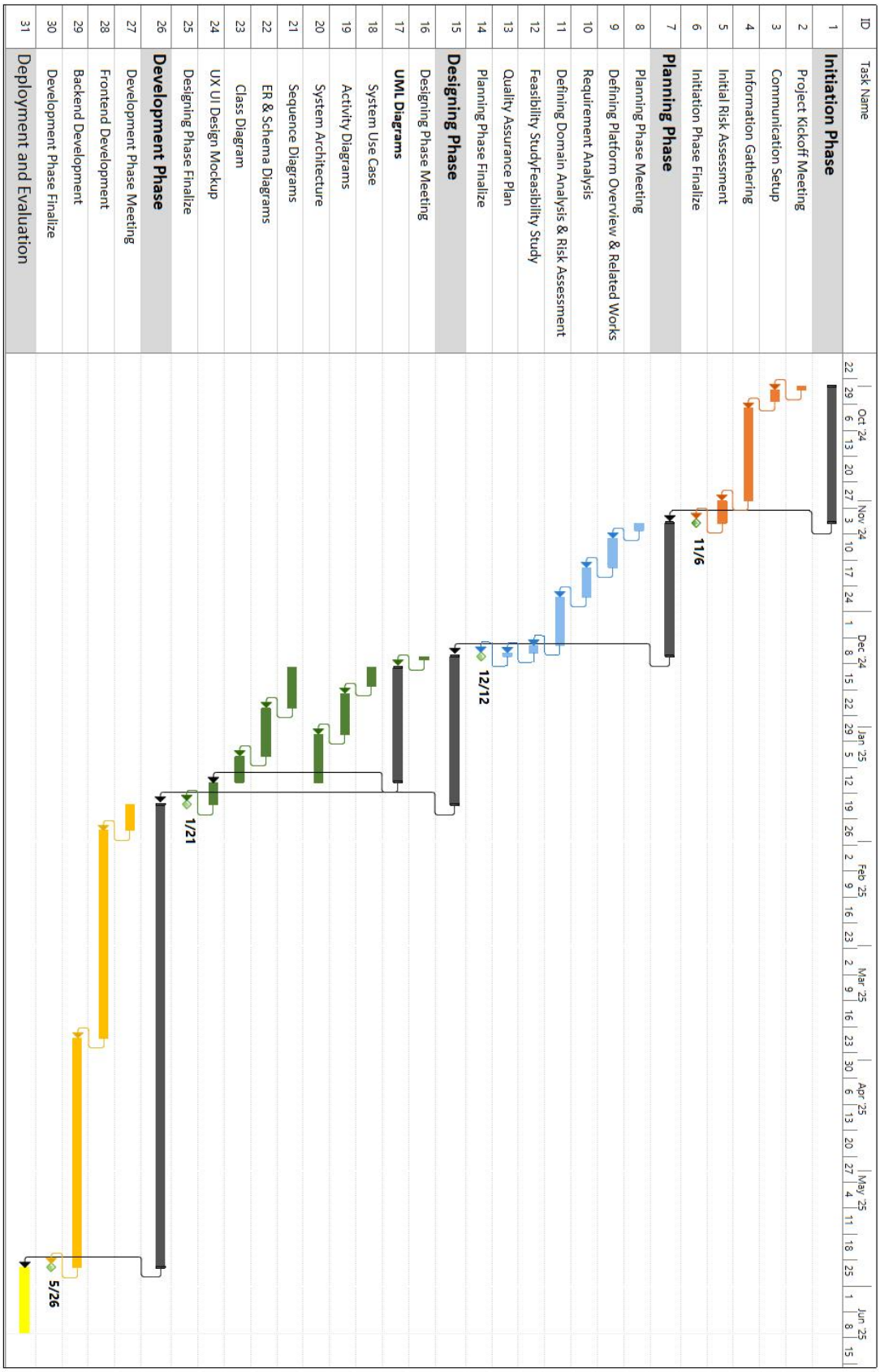


Figure 6: Project plan Gannt chart

3.4.2 Project plan Timeline

Start			
Tue 10/1/24			
Name	**Start**	**Finish**	**Duration**
Initiation Phase	Tue 10/1/24	Wed 11/6/24	27 days
Planning Phase	Thu 11/7/24	Thu 12/12/24	26 days
Designing Phase	Fri 12/13/24	Tue 1/21/25	28 days
Development Phase	Wed 1/22/25	Mon 5/26/25	89 days
Deployment and Evaluation	Tue 5/27/25	Fri 6/13/25	14 days
Expected Finish			
Fri 6/14/25			

Table 2: Project plan Timeline

3.5 Feasibility Study

3.5.1 Technical feasibility

SECTION	DESCRIPTION
Software Requirements	The platform will utilize React.js and Tailwind for the frontend, .net for the backend, Sql server as the database system and Python for machine learning integrations. The system should run on servers with Window OS, supporting the latest versions of .net, Sql server and Python.
Hardware Requirements	The infrastructure requirements include scalable cloud hosting services like Firebase storage.
Technology Assessment	React.js and .net are selected for their robustness, scalability, and extensive community support, enabling rapid development and a responsive user interface. Sql server is chosen due to its flexibility in handling structured data and its scalability for accommodating future data growth. Python for machine learning integrations and simplicity.
Technical Expertise	We have a team of skilled developers experienced in React.js, .net, Sql server, and python minimizing the need for additional recruitment or training.
Development Tools	Leveraging development tools like Visual Studio Code, Git for version control, and Visual Studio will facilitate efficient development workflows.
Security Measures	Implementing SSL encryption and regular security patches will safeguard user data, meeting industry standards and compliance regulations.

Table 3: Technical feasibility

3.5.2 Operational feasibility

SECTION	DESCRIPTION
User Acceptance Rate	Foresee a high user acceptance rate of 90% based on the platform's intuitive interface and user-friendly functionalities, allowing easy navigation and utilization for tenants, landlords, and customer service.
System Integration Period	Estimate a 5-month integration timeline to ensure seamless adoption and utilization of Symsary's diverse functionalities.
Throughput Enhancement	Project a 40% improvement in operational efficiency, expecting improving the system's ability to process a higher volume of requests, transactions, or operations efficiently within a given time.
Information Accessibility	Anticipate a 50% increase in timely access, provide real-time information on availability, pricing, and rental status to ensure accuracy. Use notifications for important updates like rental confirmations or policy changes.
Adaptation Period	Expect a 2-month adaptation period for users to acclimate to Symsary's unique features, leveraging interactive guides and forums to facilitate a smooth transition from conventional practices.
User Satisfaction	Aim for a 91% satisfaction rate among users post implementation, focusing on user-centric design and continual feedback incorporation to enhance the platform's usability.

Table 4: Operational feasibility

3.5.3 Economic feasibility

SECTION	DESCRIPTION	COST (EGP)
Development Costs	**Software Development:** Anticipate initial software development costs based on consultations with development teams and market analysis.	60,000
	Infrastructure Setup: Estimate infrastructure setup costs including server hosting, database setup, and system integration.	40,000
	Resource Allocation: Allocate costs for skilled resources, project management, and training during the development phase.	20,000
Operational Costs (Annual)	**Maintenance:** Estimate annual maintenance costs considering software updates, bug fixes, and server maintenance.	15,000
	Staff salary: Estimate staff salary costs for customer service gents for each time to review houses annually.	40,000
	Hosting: Hosting expense for server hosting and cloud services.	15,000

Table 5: Economic feasibility

Initial Investment: 120,000 EGP
Project duration for analysis: 5 years

SECTION	DESCRIPTION
Cost Savings	Expect cost savings of approximately 20,000EGP annually through the increase of houses already available and confirmed on the platform, which not need to inspection.
Revenue Projections	Reviews Service: 20,000 EGP annually (Revenue generated from reviews of houses before being added to the platform) House Advertisement adding: 60,000 EGP annually (Revenue generated from landlords' subscription to different plans of advertising on platform)

Table 6: Revenue

Year	Cost Savings	Reviews Service	House Advertisement	Total Benefit	Total Costs	Net Cash Flow	Cumulative Net Cash Flow	ROI
0	-	-	-	-	120,000	-120,000	-120,000	- 100%
1	20,000	20,000	60,000	100,000	70,000	30,000	-90,000	25.0%
2	20,000	20,000	60,000	100,000	70,000	30,000	-60,000	25.0%
3	20,000	20,000	60,000	100,000	70,000	30,000	-30,000	25.0%
4	20,000	20,000	60,000	100,000	70,000	30,000	0,000	25.0%
5	20,000	20,000	60,000	100,000	70,000	30,000	30,000	25.0%

Table 7: The payback period is approximately between in the fifth year

The payback period is approximately between in the fifth year, where the cumulative net cash flow turns positive and exceeds the initial investment of 120,000 EGP. Therefore, the estimated payback period for this investment is about 4 years.

3.6 Quality Assurance Plan

Automated testing :

In the Platform the AT is a cornerstone of Semsary's QA process, focusing on unit and integration testing. Unit testing involves the creation of detailed tests for individual components, such as functions and methods, which are executed in isolation to maintain code integrity. Developers write detailed tests for individual components, functions, and methods in isolation. These tests are executed automatically with each code modification, ensuring changes do not disrupt existing functionality. Any code changes trigger automated execution of these tests, quickly identifying and resolving potential bugs. Integration testing, on the other hand, validates the interactions between various modules and components, ensuring smooth communication and proper functionality across the system. Tools like Jest and Postman are employed to verify the compatibility and behavior of integrated components. Automated tests are seamlessly integrated into the platform's CI/CD pipelines, allowing each code commit or deployment to trigger immediate testing, providing early detection of any regressions or inconsistencies. Developers receive immediate alerts about any failures.

Manual testing :

Manual testing complements automated testing by focusing on exploratory and usability testing. Exploratory testing allows testers to navigate the platform organically, simulating real-world scenarios to uncover hidden defects and unexpected behaviors. This unscripted approach ensures that potential issues missed by automated tests are identified and addressed. Usability testing evaluates the platform's interface, navigation, and user-friendliness by simulating user journeys. Feedback collected during these tests is used to refine the platform's design, aligning it more closely with user expectations and improving the overall user experience.

Regression testing :

It is critical to maintaining the stability and reliability of the Semsary platform. This phase ensures that recent updates or changes do not adversely affect the platform’s existing functionality. A selective suite of critical test cases, focusing on high-impact areas such as the booking system, user authentication, and payment processing, is executed automatically after each deployment. These regression tests are integrated into the CI/CD pipeline, with results compared against baseline outcomes to detect any deviations or inconsistencies. By prioritizing high-risk scenarios, the platform ensures continuous functionality while accommodating changes.

Performance testing :

It ensures that the Semsary platform is scalable, robust, and reliable under various load conditions. Load testing simulates expected user behavior to evaluate the platform's response times, resource utilization, and overall performance during regular operations. Automated tools are used to monitor key metrics like throughput and latency. Stress testing complements this by subjecting the system to extreme loads, identifying its breaking points, and observing recovery behavior after stress conditions are removed. This approach ensures the platform remains stable and performs optimally even during high-demand scenarios.

Security testing :

It is a critical aspect of the QA process, focusing on protecting the platform against vulnerabilities and ensuring data confidentiality. Vulnerability scanning is conducted using automated tools to systematically identify weaknesses, such as misconfigurations or outdated components, which are then prioritized and mitigated. Additionally, penetration testing simulates real-world cyberattacks to identify potential entry points for unauthorized access. This testing, performed by skilled security professionals, assesses the resilience of authentication mechanisms, payment systems, and data storage. By addressing identified threats promptly, Semsary ensures a secure environment for its users

This structured quality assurance plan ensures that Semsary consistently delivers a reliable, secure, and user-friendly experience, aligning with its mission to provide transparency, trust, and convenience for its users.

3.7 System Requirements

3.7.1 Functional Requirements

Tenants:

Account Management:

- **FR-1:** The system should enable tenants to create an account and get verification via email or SMS.
- **FR-2:** The system should allow tenants to securely login using an email or phone number and password, and reset their login credentials if forgotten.
- **FR-3:** Tenants should be able to edit their profile information.

Search and Booking:

- **FR-4:** The system should allow tenants to search for properties and apply filters to find suitable listings.
- **FR-5:** Tenants should be able to send a booking request to a landlord if they have enough balance on the website.

Communication:

- **FR-6:** The system should enable tenants and landlords to communicate through an integrated chat feature.

Financial Transactions:

- **FR-7:** The system should allow tenants to deposit and withdraw money using credit cards or electronic wallets.
- **FR-8:** Tenants should be able to get back their money if they canceled the renting request during 4 days from date of request but after this period, the money will be transferred to landlord balance.
- **FR-9:** Tenants should be able to confirm their arriving to house to get their warranty money back.

Feedback and complaint:

- **FR-10:** System should enable tenants to request for a house preview in case the house specifications do not match the advertisement.
- **FR-11:** Tenants should have the ability to give a rate of houses that they had already rented.

Landlord:

Account Management:

- **FR-12:** The system should allow landlords to create an account and get verification via email or SMS.
- **FR-13:** The system should enable landlords to securely login using their email or phone number and password and reset their credentials if forgotten.
- **FR-14:** Landlords should be able to update their profile information.

House Advertisement Management:

- **FR-15:** The system should allow landlords to request an On-the-ground preview for their houses.
- **FR-16:** The system should provide landlords ability to accept house specifications added with customer service after the review and access all of them through their profiles.
- **FR-17:** Landlords should be able to update specific details in their houses advertisements.
- **FR-18:** Landlords should be able to publish their houses if they have enough balance and have the ability to delete them.

Financial Transactions:

- **FR-19:** The system should allow landlords to deposit and withdraw money using credit cards or electronic wallets.

Rental Management:

- **FR-20:** Landlords should be able to review, accept, or reject rental requests from tenants.
- **FR-21:** The system should enable landlords to confirm tenants arrival requests to enable tenants get their warranty money back.

Customer Service:

Account Management:

- **FR-22:** The system should allow customer service to securely login using an email and password, and reset their login credentials if forgotten.
- **FR-23:** customer service should be able to edit their profile information.

Reviews Management:

- **FR-24:** Customer service should be able to receive landlords requests for reviews and enter the house advertisement information.
- **FR-25:** Customer service should be able to receive tenants' complaints requests.

Admin:

FR-26: The system should allow admins to securely login using their emails and passwords, and reset their login credentials if forgotten.

FR-28: The system should allow the admin to manage customer service agents and add new agents by entering their details then send the generated credentials to the agents via email.

3.7.2 Non-Functional Requirements

Performance Requirements:

- **NFR-1:** The system should be able to handle up to 7,000 simultaneous user requests without performance degradation.
- **NFR-2:** Page load time should not exceed 3 seconds under normal load conditions.
- **NFR-3:** The platform should process payment transactions in under 5 seconds.

Scalability Requirements:

- **NFR-4:** The system should scale dynamically to accommodate an increase in users or data volume without performance degradation.

Availability Requirements:

- **NFR-5:** The platform should have an uptime of at least 98% annually.
- **NFR-6:** The system should provide redundancy mechanisms to minimize downtime during maintenance or unexpected failures.

Security Requirements:

- **NFR-7:** All user data must be encrypted during transmission using HTTPS and TLS protocols.
- **NFR-8:** Passwords should be hashed and stored securely using an industry-standard hashing algorithm.
- **NFR-9:** The system must be safeguarded against common web vulnerabilities like XSS, SQL injection, etc.

Usability Requirements:

NFR-10: The user interface should follow accessibility standards and be user-friendly.

Reliability Requirements:

NFR-11: The system should recover from critical failures within 10 minutes.

NFR-12: The database should maintain consistency and integrity even during unexpected crashes or system failures.

Maintainability Requirements:

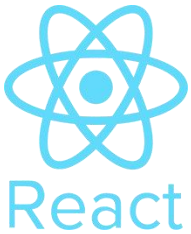
NFR-13: Maintain a clean, modular, and well-documented codebase to facilitate future enhancements and maintenance.

NFR-14: System updates or patches should be deployable with minimal downtime (preferably zero-downtime deployments).

3.8 Techniques and tools

3.8.1 Frontend Framework

1.React JS:



React.js was selected as the core framework for developing the frontend of the apartment rental platform due to its modular and dynamic nature. Key features used include:

Component-Based Architecture:
Built reusable components such as property cards, search bars, filters, and user authentication modals, Ensured consistency and reduced redundancy by organizing components logically within feature-based folders.

React Hooks:
Used useState to manage states like filter selections, form inputs, and user authentication,Implemented useEffect for handling side effects, such as fetching property data from Firebase in real-time and monitoring changes.

React Router:
Integrated React Router to enable navigation across pages (e.g., Home, Apartment Listings, User Profile, and About Us) without full page reloads, creating a seamless user experience

2. Styling Framework:

Tailwind CSS:
Tailwind CSS was used for its efficiency in designing modern, responsive, and visually appealing interfaces tailored for real estate listings.

Custom Themes:
Configured Tailwind's theme in tailwind.config.js to reflect a professional color palette, such as warm neutrals and blues, representing trust and reliability in real estate.

Responsive Design:
Ensured the website was mobile-friendly by designing layouts with responsive utility classes to accommodate all screen sizes, from smartphones to large monitors.

3. State Management

To manage data flow and application state:

React Context API: Used for sharing global states like active filters (e.g., price range, location, and amenities) and user login status across the application without prop drilling.

Local State Management: Managed component-specific states for dynamic features like search queries and form validations.

Redux : Redux was used to handle the global state of the apartment rental platform efficiently. Key features include:

Centralized State: Managed filters, apartment listings, user authentication, and reviews in a single global store.

Actions and Reducers: Defined reusable actions (e.g., FILTER_APARTMENTS, ADD_REVIEW) and reducers to update the state predictably.

Middleware: Used redux-thunk for asynchronous tasks like fetching apartment data and submitting reviews.

React Integration: Connected Redux to React using react-redux, utilizing useSelector for accessing state and useDispatch for dispatching actions.

Improved Performance: Cached data like apartment listings to minimize API calls and enhance responsiveness.

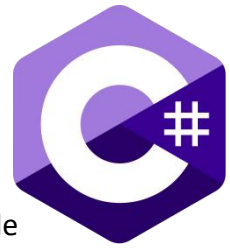
Build Tools: Vite and npm

Vite: Used for fast development with Hot Module Replacement (HMR) and optimized production builds. Custom configurations in vite.config.js supported Tailwind CSS and Firebase integration.

npm: Managed dependencies (e.g., React, Redux, Firebase) and provided scripts for development (npm run dev), production build (npm run build), and preview (npm run preview).

3.8.2 Backend Development

1. C#



C# was chosen as the programming language for backend development due to its versatility and strong integration with .NET. Key features include:

Object-Oriented Programming: Enabled modular, reusable, and maintainable code, essential for implementing business logic for user management, apartment listings, and reviews.

Asynchronous Programming: Used async/await for handling asynchronous tasks like database queries and API calls, improving server responsiveness.

Robust Error Handling: Implemented try-catch blocks and custom exception handling to ensure stability and provide detailed error messages for debugging.

Integration Support: C#'s rich library ecosystem facilitated seamless integration with Firebase for image storage and third-party tools like Google Analytics.

2. NET Framework

ASP.NET Core, a modern and high-performance web framework, powered the backend with the following features:

RESTful API Development: Designed endpoints for CRUD operations on apartments, user accounts, and reviews. Supported HTTP methods like GET (retrieve apartments), POST (add reviews), PUT (update user info), and DELETE (remove listings).

Middleware:Used custom middleware for logging, error handling, and request validation.I mplemented authentication middleware to validate tokens and secure API access.

Dependency Injection: Integrated DI to manage services like database contexts and external API clients, enhancing code modularity and testability.

Authentication and Authorization: Implemented JWT (JSON Web Token) for user authentication and role-based authorization to differentiate access for normal users, premium users, and admins.

1. SQL Server

SQL Server served as the relational database for managing structured data in the application. Key aspects include:

Database Design:

Tables were designed with proper normalization to reduce redundancy and improve data integrity.
Users: Stored user details, authentication information, and roles.
Apartments: Included details like location, price, amenities, and image URLs.
Reviews: Managed user feedback with fields for ratings, comments, and timestamps.

Stored Procedures:

Created stored procedures for complex queries like fetching apartments with filters applied and calculating average ratings.

Improved performance by reducing the load on the application layer and optimizing query execution.

Indexes:
Added indexes on frequently queried fields (e.g., apartment location, price) to speed up search operations.

Transactions:

Used transactions to ensure consistency during critical operations, such as booking apartments or submitting reviews, to prevent partial updates.

3.8.3 Machine Learning Model

1. Python

Python served as the primary programming language for implementing the recommendation system due to its:

Rich Ecosystem: Extensive libraries and tools for data manipulation, visualization, and machine learning.

Ease of Use: Simple syntax that accelerates development and experimentation.
Community Support: Access to a vast range of resources, tutorials, and pre-built models.

2. TensorFlow

TensorFlow, an open-source machine learning framework, was used for building and training the recommendation model.

Key Features Utilized:

Deep Learning Support: Designed a neural network architecture optimized for collaborative filtering or content-based recommendations.

Ease of Deployment: TensorFlow's model export capabilities allowed easy integration of the trained model with the backend.

TensorFlow Recommenders (TFRS): A specialized library within TensorFlow, TFRS provided prebuilt tools for creating recommendation systems, reducing development time.

Advantages:

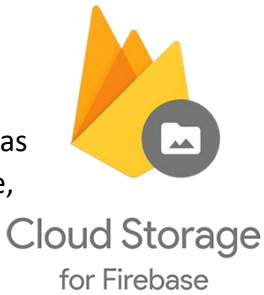
Scalability: Handled large datasets of user reviews, apartment details, and preferences efficiently.

Performance: TensorFlow’s GPU support accelerated training times for large-scale models.

Customizability: Allowed fine-tuning of the model to cater to unique aspects of the rental platform, such as prioritizing recent reviews or higher-rated apartments.

3.8.4 Third Party Services

1. Firebase Storage



Firebase Storage was utilized for storing and managing user-uploaded images, such as apartment photos. Its integration simplified handling large files and ensured reliable, scalable storage. Key features include:

Seamless Integration:I
ntegrated with the Firebase SDK to easily upload, retrieve, and manage image files directly from the application.

Scalability:
Supported storing high-resolution images without impacting app performance, ensuring the platform can handle increasing user activity.

Security Rules:
Configured Firebase security rules to control access, ensuring only authenticated users can upload images and restricting downloads as per user roles.

Real-Time URL Generation:
Generated secure download URLs after each upload, which were stored in the database and used to display images on the platform.

Cost-Effectiveness:
Benefited from Firebase's pay-as-you-go pricing, making it a cost-efficient choice for storage needs.

2. SendGrid:
Email Delivery Service

SendGrid is a cloud-based email delivery platform that provides businesses with the tools to send transactional and marketing emails at scale. It offers a range of APIs and SMTP services for email delivery, along with analytics to track email performance, ensuring high deliverability rates and reliable service. SendGrid is popular among developers for its ease of integration with both small applications and large-scale enterprise systems.

Email API: Allows developers to send email messages from their applications, with options for both transactional and marketing emails.
SMTP Relay: Provides a reliable service for sending emails using the SMTP protocol, making it easy to integrate into existing systems.
Deliverability Tools: Includes tools to track email delivery, open rates, click rates, and more, to optimize performance.
Templates and Personalization: Users can create and send customized email templates for better engagement with recipients.
Scalability: Supports both small and large-scale email sending needs, suitable for businesses of any size.
Security: Offers features like email authentication and encryption to protect against phishing and unauthorized access.

Use Cases:
Sending transactional emails (e.g., order confirmations, password resets).
Running marketing campaigns with advanced email personalization.I
ntegrating email services with web applications via API.

---

3. PayMob: Payment Gateway

PayMob is a leading payment gateway in Egypt, designed to facilitate secure online transactions for businesses and consumers. It provides an API for processing payments, enabling businesses to accept payments via credit/debit cards, mobile wallets, and other local payment methods. PayMob is widely used by e-commerce businesses, subscription services, and retail merchants to offer a seamless and secure payment experience.

Key Features:

Payment Gateway API: Allows businesses to process payments from customers via a wide range of local and international payment methods.

Mobile Wallet Integration: Supports popular mobile wallets, including Fawry, Vodafone Cash, and others for easy mobile-based payments.

Recurring Payments: Enables subscription-based services to collect payments on a regular basis.

Fraud Detection: Provides advanced security features, including fraud prevention tools, to protect businesses from fraudulent transactions.

Invoicing: Facilitates generating and managing invoices directly from the PayMob platform.

Local Integration: Tailored specifically for the Egyptian market, with support for local banks and payment methods.

Use Cases:
Enabling e-commerce sites to accept payments securely.

Providing subscription-based businesses with recurring billing solutions.

Integrating mobile wallet payments for greater convenience.

---

4. Twilio: SMS Messaging API

Overview:
Twilio provides a powerful SMS API for businesses to send and receive text messages globally. It allows seamless integration of SMS capabilities into web and mobile applications, enabling businesses to automate notifications, alerts, reminders, and two-way messaging. With robust scalability and reliable delivery, Twilio ensures effective communication for organizations of all sizes.

Key Features:

Send SMS:
Quickly send text messages to customers worldwide through a simple API.

Receive SMS:
Enable two-way communication by receiving replies directly in your application.

Phone Number Management:
Purchase, configure, and manage virtual phone numbers to customize messaging.

Global Reach:
Deliver SMS to over 180 countries with high reliability and fast delivery rates.

Programmable Messaging:
Personalize messages with templates and variables for tailored communication.

Delivery Status Tracking:
Monitor the status of sent messages with detailed delivery reports.

Scalability:
Supports large-scale messaging for campaigns, promotions, or transactional alerts.

Use Cases:
Sending transactional messages like OTPs, confirmations, and alerts.
Running SMS-based marketing campaigns.
Automating customer notifications, reminders, and updates.

---

5. Google Analytics

Google Analytics was integrated into the project to track and analyze user interactions, providing valuable insights for improving the platform.

Key Features:

Real-Time Monitoring:
Tracked active users, pages visited, and interactions as they occurred.

User Behavior Analysis:
Collected data on how users navigated the site, including frequently visited pages, filters used, and time spent on listings.

Traffic Sources:
Identified where users originated from (e.g., search engines, social media, or referrals), helping optimize marketing strategies.

Event Tracking: Monitored specific actions, such as apartment searches, clicks on "Contact Owner," and image uploads.

Helped understand user preferences and behavior, enabling data-driven improvements to the platform.

Monitored conversion rates, such as users signing up or submitting apartment reviews.

6. Version Control: Git and GitHub

Git: Used locally for tracking code changes, managing feature branches, and enabling easy rollbacks.

GitHub: Hosted the repository for collaboration, pull requests, and code reviews, while serving as a secure backup.

3.9 Application components

1) **user authentication and management:**
This component handles user registration, login, and profile management. It provides secure access to the platform, allowing users to create accounts, log in, manage their profiles, and reset passwords as required. Different user roles, such as tenants, landlords, and administrators, are supported.

2) **property listing and management:**
This module enables landlords to list properties, including apartments, rooms, or beds, with detailed information such as features, prices, and proximity to amenities like markets and transportation. Landlords can update or manage listings, ensuring accurate and up-to-date information for potential renters.

3) **search and filtering:**
The search functionality allows users to explore rental properties based on their needs. Filters such as number of rooms, rental type, budget, and proximity to key facilities (e.g., schools, markets, transportation) help users find suitable accommodations efficiently.

4) **booking and payment management:**
This component supports direct booking of properties through the platform. Users must add funds to their site wallet to complete transactions. Secure payment options are integrated, ensuring a seamless and reliable booking process. The platform also handles refunds and cancellation fees as per defined policies.

5) **user review and feedback system:**
This feature enables tenants to share their experiences and reviews about rented properties. Ratings and feedback help other users make informed decisions and improve the overall transparency of the rental process.

6) **machine learning for price estimation:**
Machine learning models analyze property data, such as features, amenities, and market trends, to provide landlords with rental price suggestions. The system ensures fair pricing by offering feedback on whether the proposed prices align with market standards.

7) **administrative management:**

The admin panel allows platform administrators to manage reports, approve advertisements, resolve disputes, and ensure compliance with platform guidelines. Admins also oversee financial transactions and ensure system integrity.

8) **database management:**

The database stores crucial information, including user profiles, property details, transactions, reviews, and booking history. Efficient data management ensures scalability, security, and quick retrieval of data for all platform operations.

Chapter 4

Proposed System & Methodology

4.1 System Use Cases

4.1.1 Client side use case

4.1.1.1 Tenant

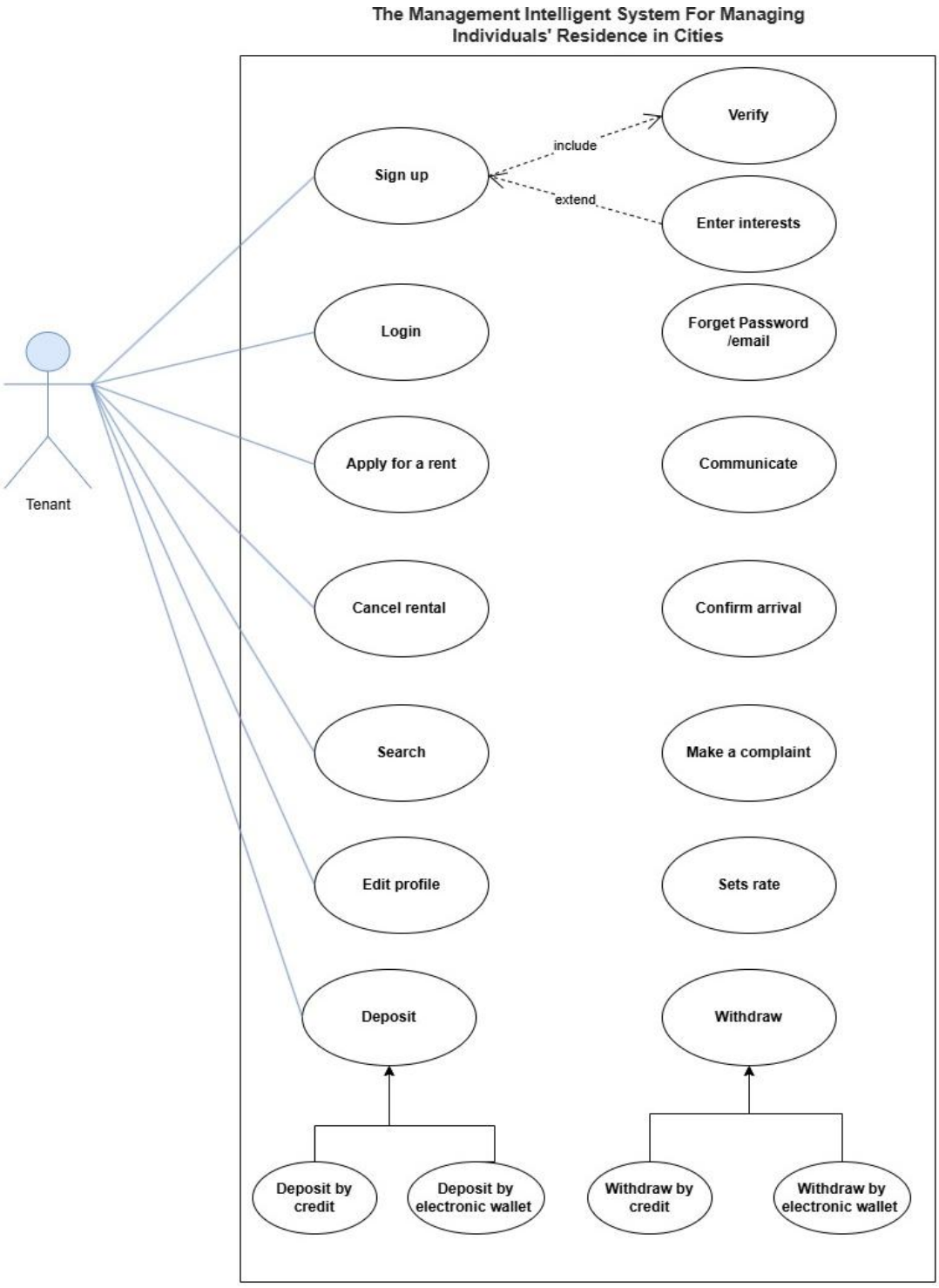


Figure 7: Use Case Diagram - Tenant

4.1.1.2 Landlord

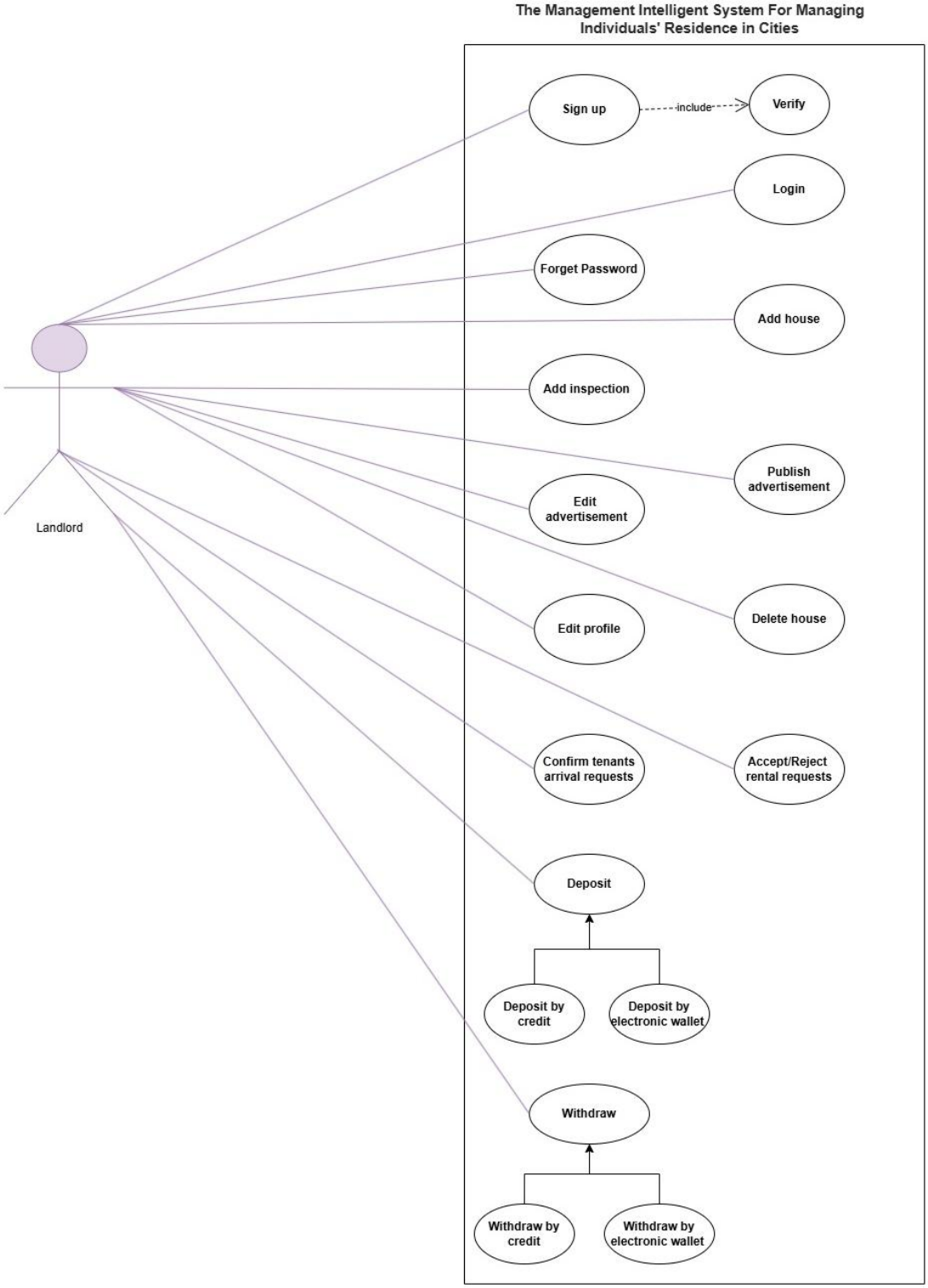


Figure 8: Use Case Diagram - Landlord

4.1.2 Server side use case

4.1.2.1 Customer Service

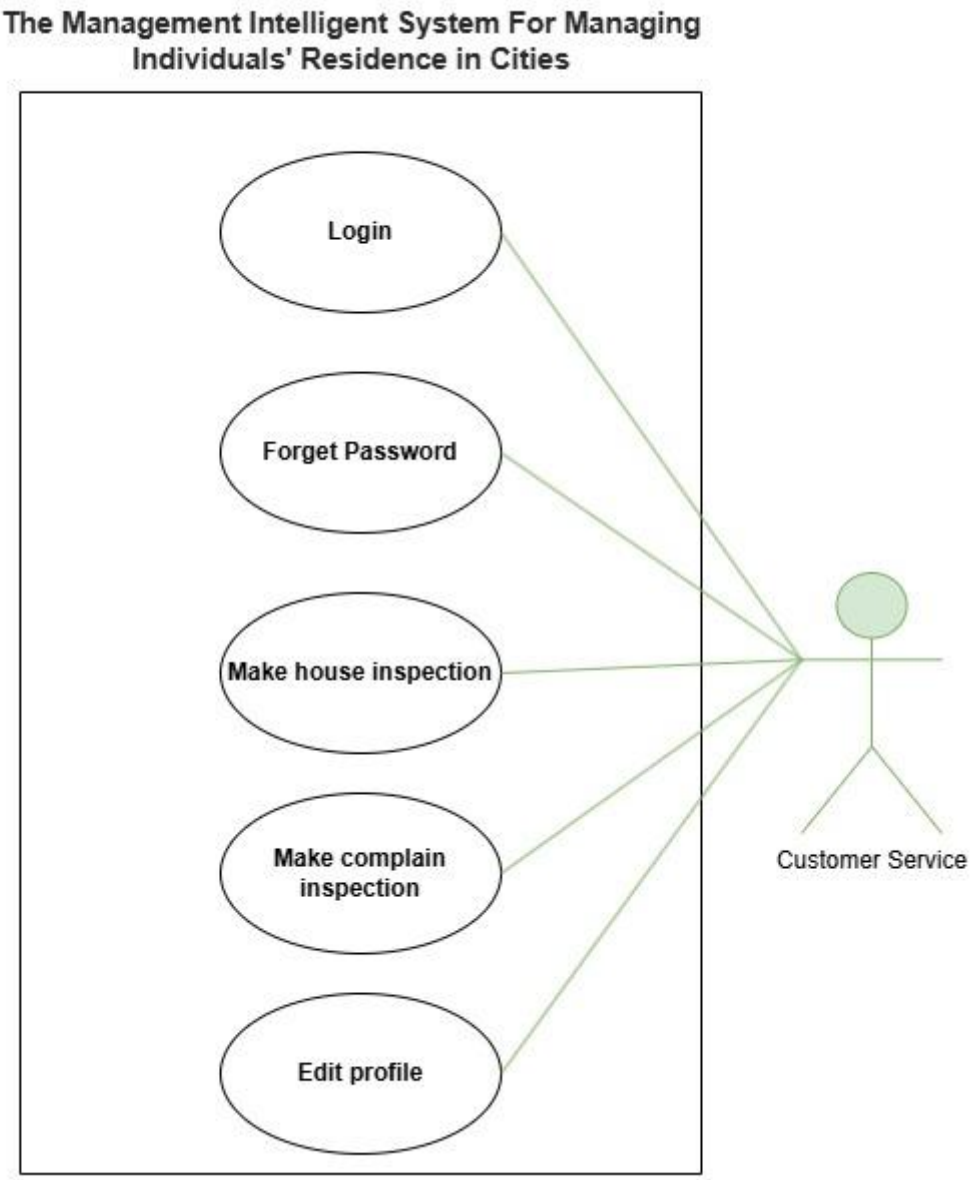


Figure 9: Use Case Diagram - Customer Service

4.1.3 Admin side use case

4.1.3.1 Admin Use Case

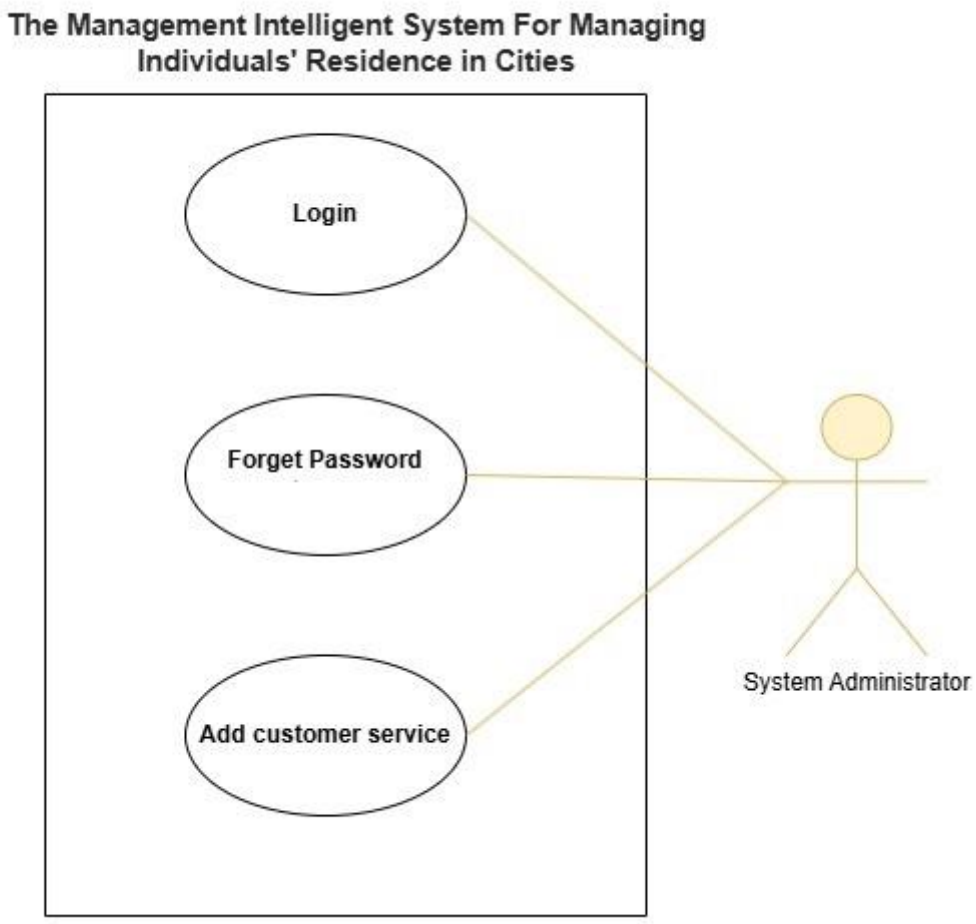
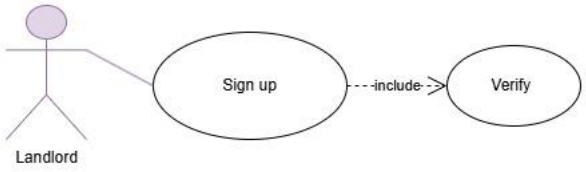


Figure 10: Use Case Diagram - Admin Use Case

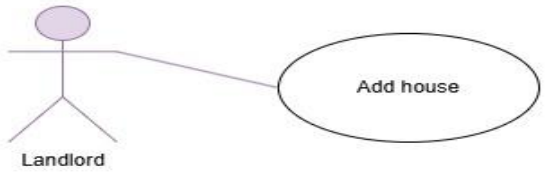
4.2 Use Case Description (Use case scenario)

4.2.1 Landlord



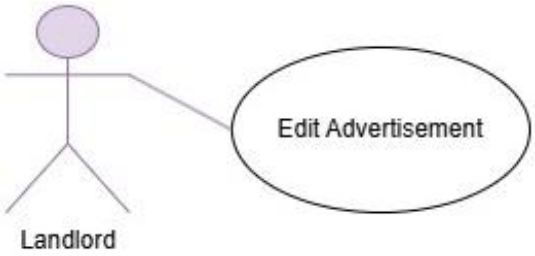
Case Name:	Landlord Sign Up
Case ID	LND-01
Actor(s):	Landlord
Description:	Registers a new landlord on the platform.
Triggering Event:	Landlord selects "Sign Up" option on the platform.
Preconditions:	Landlord is not yet registered. Landlord has required information (phone number or email, password, proof of identity).
Assumptions:	The platform is accessible and operational. Identity verification is automated and supported by a reliable third-party service.
Main Path:	Landlord enters phone number or email, password, and uploads identity document. Platform validates the information. If all data is valid, the system creates a new account for the landlord.
Alternate Path:	If the identity document is invalid, the system notifies the landlord and prompts them to upload a valid document.
Postcondition:	New landlord is registered and can access their account.
Priority:	High

Table 8: Use Case Scenario - Landlord Sign-Up



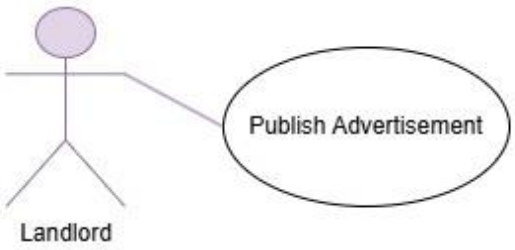
Case Name:	Add House
Case ID	LND-02
Actor(s):	Landlord
Description:	Allows landlords to submit a request for customer service to inspect the house.
Triggering Event:	Landlord selects "Add House."
Preconditions:	Landlord is logged in. Landlord has a valid proof of ownership document.
Assumptions:	Customer service agents are available for property inspections. Inspection scheduling occurs within an acceptable timeframe (e.g., 48 hours).
Main Path:	Landlord enters property address and uploads proof of ownership. Platform sends request to customer service for inspection. Customer service inspects property and defines specifications. Platform adds the house to landlord list of advertisements.
Alternate Path:	If proof of ownership is invalid, the system requests resubmission.
Postcondition:	Property is listed for rent on the platform.
Priority:	High

Table 9: Use Case Scenario - Add House



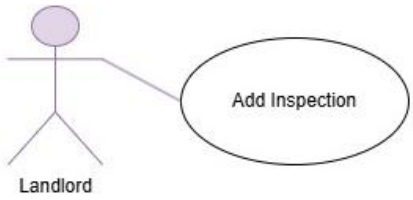
Case Name:	Edit Advertisement
Case ID	LND-03
Actor(s):	Landlord
Description:	Allows landlord to update availability and pricing information for an existing listing.
Triggering Event:	Landlord selects"Edit Advertisement"on their listing
Preconditions:	Landlord is logged in. Landlord has enough balance for publishing the advertisement.
Assumptions:	The platform updates are instantaneous and reflect immediately on listings. Landlord has accurate, up-to-date information about availability and pricing.
Main Path:	Landlord updates availability status and/or price. Platform saves the changes to the listing.
Alternate Path:	If the listing is inactive, the platform notifies the landlord.
Postcondition:	Updated listing is displayed on the platform.
Priority:	Medium

Table 10: Use Case Scenario - Update House



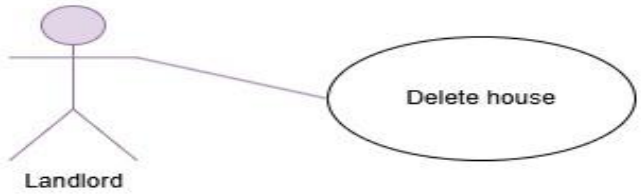
Case Name:	Publish Advertisement
Case ID	LND-04
Actor(s):	Landlord
Description:	Allows landlord to publish house from their houses' advertisements list.
Triggering Event:	Landlord selects"Publish Advertisement"for advertisement he wants to publish.
Preconditions:	Landlord is logged in. The house has already been reviewed by customer service.
Assumptions:	Landlord already has houses in his list. Landlord has enough balance.
Main Path:	Landlord chooses the house he want to publish from his list. Landlord choose a suitable plan for the advertisement. The cost of the advertisement will be withdrawn from landlord balance.
Alternate Path:	If the landlord does not have enough balance he will be notified.
Postcondition:	Display the Advertisement on the platform.
Priority:	High

Table 11: Use Case Scenario - Publish House



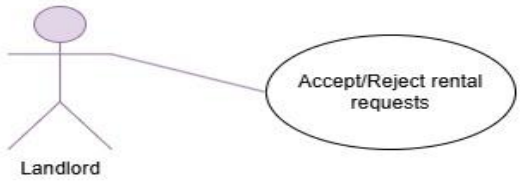
Case Name:	Add Inspection
Case ID	LND-05
Actor(s):	Landlord
Description:	Allows landlord to check houses inserted in his list.
Triggering Event:	Landlord selects"Add Inspection"
Preconditions:	Landlord is logged in.
Assumptions:	Landlord already has houses in his list.
Main Path:	Landlord enters the list of houses that has been inspected and added to his list. Landlord is able to choose any advertisement to check it details.
Alternate Path:	If the landlord does not have houses in his list he will be notified.
Postcondition:	Display the list of Advertisements for landlord.
Priority:	High

Table 12: Use Case Scenario - Review House



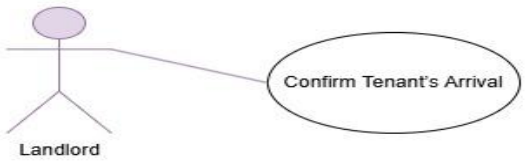
Case Name:	Delete House
Case ID	LND-06
Actor(s):	Landlord
Description:	Allows landlord to Delete house advertisement from his list.
Triggering Event:	Landlord selects "Delete House".
Preconditions:	Landlord is logged in.
Assumptions:	There is already advertisement to be deleted.
Main Path:	Landlord chooses any advertisement to delete. Platform saves the changes to the listing.
Alternate Path:	If the landlord does not have houses in his list he will be notified.
Postcondition:	Advertisement will be removed from landlord list.
Priority:	Medium

Table 13: Use Case Scenario - Delete House



Case Name:	Accept or Reject Rental Requests
Case ID	LND-07
Actor(s):	Landlord
Description:	Landlord reviews and approves or declines rental requests.
Triggering Event:	Landlord receives a rental request.
Preconditions:	Landlord has an active rental listing. Rental request exists with specified dates.
Assumptions:	Platform messaging system facilitates prompt communication between landlord and tenant. Tenant has sufficient funds for the guarantee if the request is accepted.
Main Path:	Landlord reviews rental request details. Landlord either accepts or rejects the request. Platform updates the request status and notifies the tenant.
Alternate Path:	If the rental dates are unavailable, platform displays an error message.
Postcondition:	Request status is updated, and tenant is notified.
Priority:	High

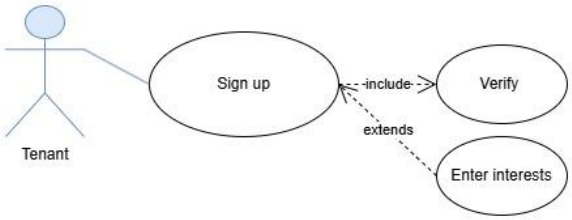
Table 14: Use Case Scenario - Accept or Reject Rental Requests



Case Name:	Confirm Tenant’s Arrival
Case ID	LND-08
Actor(s):	Landlord
Description:	Confirms tenant arrival at the rental property.
Triggering Event:	Tenant informs platform of their arrival.
Preconditions:	Rental request is approved, and tenant is scheduled to arrive.
Assumptions:	Platform reliably tracks tenant arrival and departure dates. Landlord can access and confirm tenant arrival without delay.
Main Path:	Landlord receives arrival notification. Landlord confirms tenant’s arrival on the platform.
Alternate Path:	If tenant does not arrive within a set timeframe, platform marks request as “Cancelled.”
Postcondition:	Rental officially begins, and system updates status.
Priority:	High

Table 15: Use Case Scenario - Confirm Tenant’s Arrival

4.2.2 Tenant

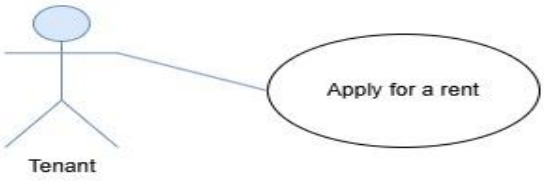


Case Name:	Tenant Sign Up
Case ID	TEN-01
Actor(s):	Tenant
Description:	Registers a new tenant on the platform.
Triggering Event:	Tenant selects "Sign Up" on the platform.
Preconditions:	Tenant has an email or phone number, password, and proof of identity.
Assumptions:	The platform's identity verification process is swift and reliable. Tenant's device supports document upload functionality.
Main Path:	Tenant enters email or phone number, password, and uploads identity document. Platform verifies the information. Tenant answer questions about the type of houses he prefer to live in. Account creation is completed if data is valid.
Alternate Path:	If identity verification fails, tenant is prompted to resubmit valid proof.
Postcondition:	New tenant account is created.
Priority:	High

Table 16: Use Case Scenario - Tenant Sign Up

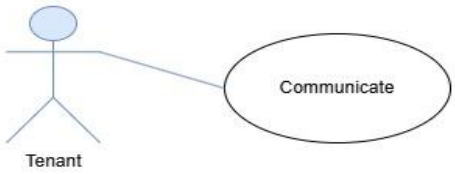
Case Name:	Search for Suitable House
Case ID	TEN-02
Actor(s):	Tenant
Description:	Tenant searches for available houses based on preferences.
Triggering Event:	Tenant accesses search functionality.
Preconditions:	Tenant is logged in.
Assumptions:	Search filters function accurately and are up-to-date. Platform has enough rental listings to provide relevant results.
Main Path:	Tenant selects search filters (e.g., location, price range, number of bedrooms). Platform displays listings that match the criteria.
Alternate Path:	If no listings match, platform suggests alternative options.
Postcondition:	Tenant views a list of suitable properties.
Priority:	Medium

Table 17: Use Case Scenario - Search for Suitable House



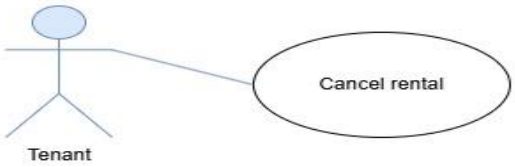
Case Name:	Apply for Rent
Case ID	TEN-03
Actor(s):	Tenant
Description:	Tenant submits a rental request to the landlord.
Triggering Event:	Tenant selects "Apply for Rent."
Preconditions:	Tenant is logged in. Tenant has sufficient funds for the guarantee amount.
Assumptions:	Guarantee amount covers initial rental period and any fees. Landlord responds within a reasonable time to tenant requests.
Main Path:	Platform forwards request to landlord and updates status. The warranty money required is withdrawn from landlord balance.
Alternate Path:	If funds are insufficient, platform prompts for deposit.
Postcondition:	Rental request is created, and landlord is notified.
Priority:	High

Table 18: Use Case Scenario - Apply for Rent



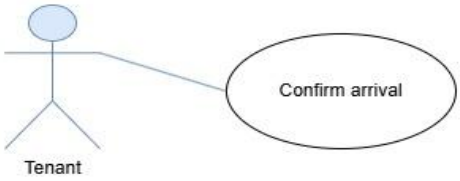
Case Name:	Communicate
Case ID	TEN-04
Actor(s):	Tenant
Description:	Tenant could be communicate with the landlord.
Triggering Event:	Tenant selects "Communicate."
Preconditions:	Tenant is logged in.
Assumptions:	Landlord will reply for the tenant message.
Main Path:	Tenant send messages to landlord. The platform will send notifications to tenant went the landlord reply.
Alternate Path:	If communication failed, the tenant will be notified to try again.
Postcondition:	Tenant and landlord be connected together.
Priority:	Medium

Table 19: Use Case Scenario - Communicate



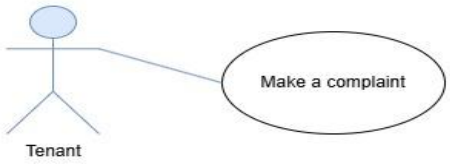
Case Name:	Cancel Rental
Case ID	TEN-05
Actor(s):	Tenant
Description:	Tenant could be cancel his rent request.
Triggering Event:	Tenant selects "Cancel Rental Request."
Preconditions:	Tenant is logged in.
Assumptions:	Tenant already has sent a rental request.
Main Path:	The landlord will be notified and state of house will be changed automatically. If it has not been 4 days from the date of the request, the warranty money will be returned to tenant account. If tenant exceeded the permitted period, the warranty money will be added to landlord balance.
Alternate Path:	If cancel failed, the tenant will be notified to try again.
Postcondition:	Tenant renting request be canceled and advertisement state be changed.
Priority:	Medium

Table 20: Use Case Scenario - Cancel Rental



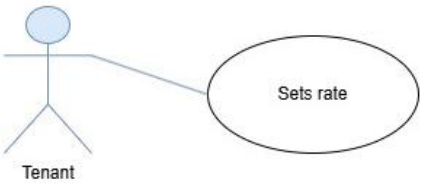
Case Name:	Confirm Arrival
Case ID	TEN-06
Actor(s):	Tenant
Description:	Tenant could be able to confirm his arrival.
Triggering Event:	Tenant selects "Confirm Arrival."
Preconditions:	Tenant is logged in.
Assumptions:	Tenant arrived to rented house within the specified period and found that the house specification are identical to the advertisement. Landlord confirmed the user arrival request.
Main Path:	Tenant warranty money will returned to his balance. Balance amount will be updated.
Alternate Path:	If landlord did not confirm the arrival through 3 days, the warranty money will return back automatically to the tenant.
Postcondition:	Warranty money will return back to tenant balance.
Priority:	High

Table 21: Use Case Scenario - Confirm Arrival



Case Name:	Make A Complaint
Case ID	TEN-07
Actor(s):	Tenant
Description:	Tenant request customer service to review the house.
Triggering Event:	Tenant selects "Complain"
Preconditions:	Tenant is logged in.
Assumptions:	Tenant arrived to rented house within the specified period and found that the house specification does not match the advertisement.
Main Path:	Tenant warranty money will returned to his balance. Balance amount will be updated. Landlord will take a ban for 4 months.
Alternate Path:	If the complaint is false, the warranty money will be transformed to our system.
Postcondition:	Warranty money will return back to tenant balance and the landlord will be banned for 4 months.
Priority:	High

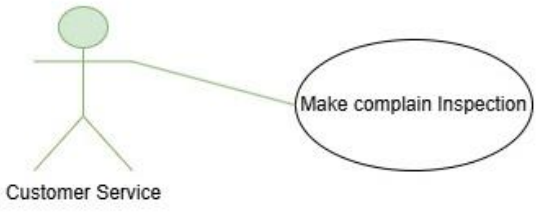
Table 22: Use Case Scenario - Make A Complaint



Case Name:	Sets Rate
Case ID	TEN-08
Actor(s):	Tenant
Description:	Allows tenants to submit specific ratings and reviews about the house, the landlord, and a general overall rating of their rental experience.
Triggering Event:	Tenant completes a rental period, or the landlord finalizes the rental agreement.
Preconditions:	Tenant has completed a stay at the property. Both parties are logged in to their accounts.
Assumptions:	The review system includes fields to rate the house, the landlord, and a general experience score. Users are encouraged to provide honest, respectful feedback. Protections are in place to detect inappropriate or biased reviews.
Main Path:	Tenant accesses the "Rate and Review" section after completing their rental stay. Tenant provides a separate rating for the house, the landlord, and a general rating. • **House Rating**: Tenant rates the property's condition, amenities, and match with the listing description. • **Landlord Rating**: Tenant rates the landlord's responsiveness, professionalism, and communication. • **General Rating**: Tenant provides an overall rating for the entire experience. Tenant submits additional comments or feedback. Platform publishes the ratings and review, displayed on the profiles of the tenant, landlord, and property listing.
Alternate Path:	If a review is flagged for inappropriate content, it is temporarily hidden and sent for review by customer service.
Postcondition:	Ratings and reviews are saved, helping future users make informed decisions based on house, landlord, and general experience ratings.
Priority:	Medium

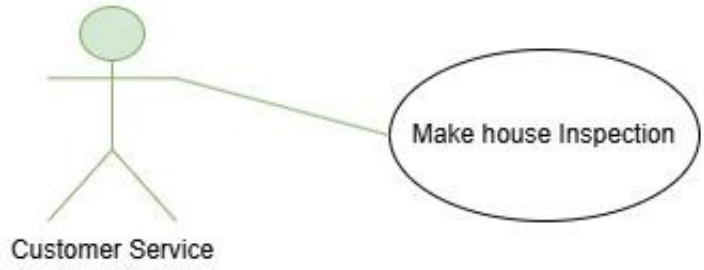
Table 23: Use Case Scenario - Sets Rate

4.2.3 Customer services



Case Name:	Make Complain Inspection
Case ID	CS-01
Actor(s):	Customer Service
Description:	Customer service receives a review request from tenant.
Triggering Event:	Tenant selects"Make Complain Inspection"
Preconditions:	Customer service agent has been added to the system.
Assumptions:	Customer service is online to receive the notification.
Main Path:	If there is really a difference between the house and advertisement, the landlord will be blocked for 4 months any warranty money will be transformed back to tenant balance without landlord confirmation. If it is a false complaint, the warranty money will be transformed back to the tenant without blocking the landlord.
Alternate Path:	If a customer service agent is offline when the complaint occurs, they will be notified as soon as they are online again.
Postcondition:	Landlord be blocked and warranty money will be transformed back to the tenant.
Priority:	High

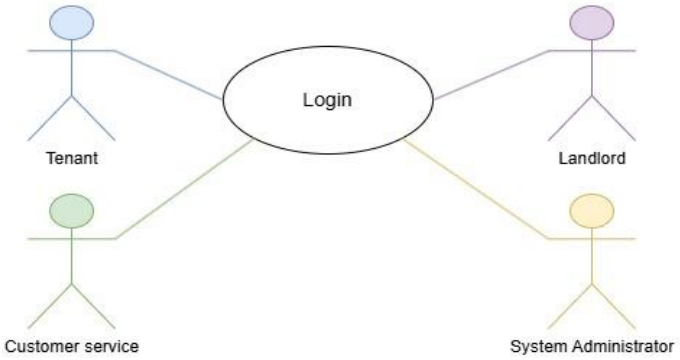
Table 24: Use Case Scenario - Reviews Complains



Case Name:	Make House Inspection
Case ID	CS-02
Actor(s):	Customer Service
Description:	Customer service inspects a submitted property and defines specifications.
Triggering Event:	Landlord submits a rental request.
Preconditions:	Landlord's property request is valid and verified.
Assumptions:	Customer service team has adequate resources for inspections. Inspection results are objective and accurate, preventing false listings.
Main Path:	Customer service schedules and performs inspection. They define property specifications (rooms, amenities, photos). Advertisement is published on the platform.
Alternate Path:	If additional proof is needed, customer service requests it from the landlord.
Postcondition:	Property is listed with verified details.
Priority:	High

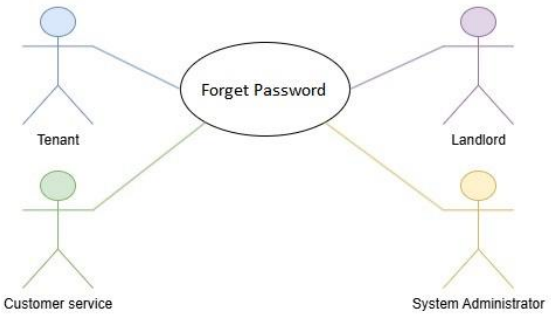
Table 25: Use Case Scenario - Review/Enter House Information

4.2.4 User



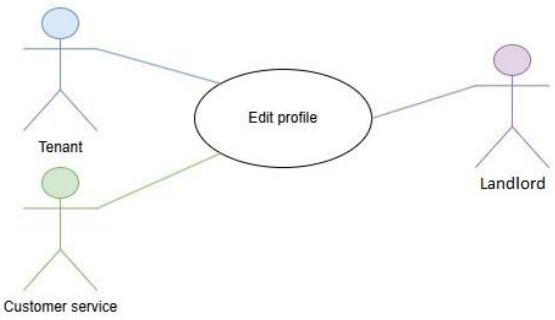
Case Name:	User Login
Case ID	GEN-01
Actor(s):	User (Tenant, Landlord, Customer Service Agent, or System Administrator)
Description:	Allows any user (tenant, landlord, customer service agent, or System Administrator) to securely log in to their account on the platform.
Triggering Event:	User accesses the login page and enters credentials.
Preconditions:	User has an existing account with registered information. The user's device is compatible with the platform.
Assumptions:	Login process includes security measures such as CAPTCHA or multi-factor authentication for authorized access. System enforces lockout policies after multiple failed login attempts.
Main Path:	User enters their logininfo on the login page. Platform validates credentials. If credentials are correct, the user is granted access to their account.
Alternate Path:	If credentials are incorrect, the user receives an error message and an option to retry.
Postcondition:	User successfully logs in and accesses their account dashboard.
Priority:	High

Table 26: Use Case Scenario - User Login



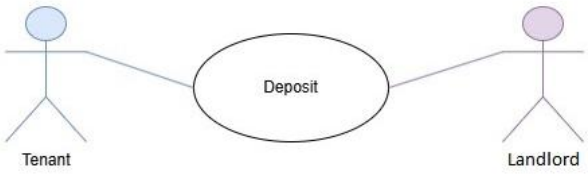
Case Name:	Forgot Password
Case ID	GEN-02
Actor(s):	(Tenant , Landlord, Customer Service Agent, Admin)
Description:	Allows users (tenants, landlords, Customer Service Agents) to reset their password if they cannot remember it.
Triggering Event:	User selects the "Forgot Password" option on the login screen.
Preconditions:	User has an account on the platform with a registered email or phone number. User can access their email or phone for verification.
Assumptions:	The email or phone number is correctly linked to the user's account. The user's registered email service or phone carrier is operational. The system has implemented proper security to handle password resets securely.
Main Path:	User selects "Forgot Password" on the login page. Platform prompts user to enter their registered email or phone number. Platform sends a password reset link (or verification code) to the provided email or phone. User receives the reset link or code and follows instructions (clicking link or entering code). Platform verifies the link or code and prompts user to enter a new password. User enters a new password and confirms it. Platform updates the user's password and confirms the reset.
Alternate Path:	If the entered email or phone number does not match any account, platform displays an error message. If the reset link or code expires, platform notifies the user to request a new one.
Postcondition:	User's password is successfully updated, and they can log in with the new password.
Priority:	High

Table 27: Use Case Scenario - Forgot Password



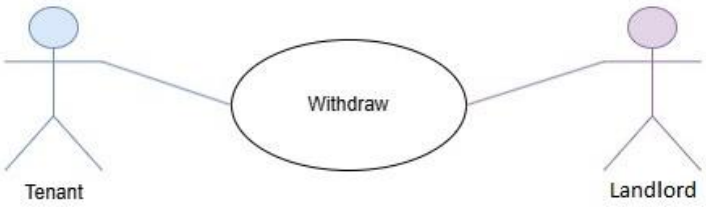
Case Name:	Edit profile
Case ID	GEN-03
Actor(s):	(Tenant , Landlord, Customer service)
Description:	Allows users (tenants, landlords, Customer Service Agents) to edit their profile information.
Triggering Event:	User selects the "Edit profile" option.
Preconditions:	User has an account on the platform with a registered email or phone number.
Assumptions:	User profile information is not updated.
Main Path:	User selects the field that he want to change. User enters the new information. Platform saves the changes to the profile.
Alternate Path:	If the user entered invalid information for the selected field, the system will notify him.
Postcondition:	User profile is up to date.
Priority:	High

Table 28: Use Case Scenario - Edit profile



Case Name:	Deposit
Case ID	GEN-04
Actor(s):	User (Tenant or Landlord)
Description:	Allows any user (tenant or landlord) to deposit money into his balance.
Triggering Event:	User selects "Deposit".
Preconditions:	User logged in.
Assumptions:	Payment Gateway is secure.
Main Path:	User chooses the deposit method (Credit or electronic wallet). User enter the required information. The amount of money is deposited to the use balance.
Alternate Path:	If the Credit or electronic wallet required information is wrong, the system will notify user.
Postcondition:	The deposit process is completed successfully.
Priority:	High

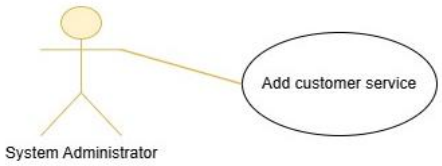
Table 29: Use Case Scenario - Deposit



Case Name:	Withdraw
Case ID	GEN-05
Actor(s):	User (Tenant or Landlord)
Description:	Allows any user (tenant or landlord) to withdraw money from his balance.
Triggering Event:	User selects "Withdraw".
Preconditions:	User logged in.
Assumptions:	Payment Gateway is secure.
Main Path:	User chooses the withdraw method (Credit or electronic wallet). User enter the required information. The amount of money is withdrawn from the use balance.
Alternate Path:	If the Credit or electronic wallet required information is wrong, the system will notify user. If the user balance is less than the amount to be withdrawn, the system will notify the user
Postcondition:	The withdrawal process is completed successfully.
Priority:	High

Table 30: Use Case Scenario - Withdraw

4.2.5 System Administrator



Case Name:	Add Customer Service
Case ID	ADM-01
Actor(s):	Admin
Description:	Add new customer service agent to the website.
Triggering Event:	User selects "Add new Customer Service".
Preconditions:	User logged in.
Assumptions:	Admin has all required data about the customer service.
Main Path:	Admin enter data of the new customer service agent Admin save the data
Alternate Path:	If entered data is invalid, the system will notify the admin.
Postcondition:	Sending email to the customer service agent with his password.
Priority:	High

Table 31: Use Case Scenario - Add Customer Service

4.3 System Architecture

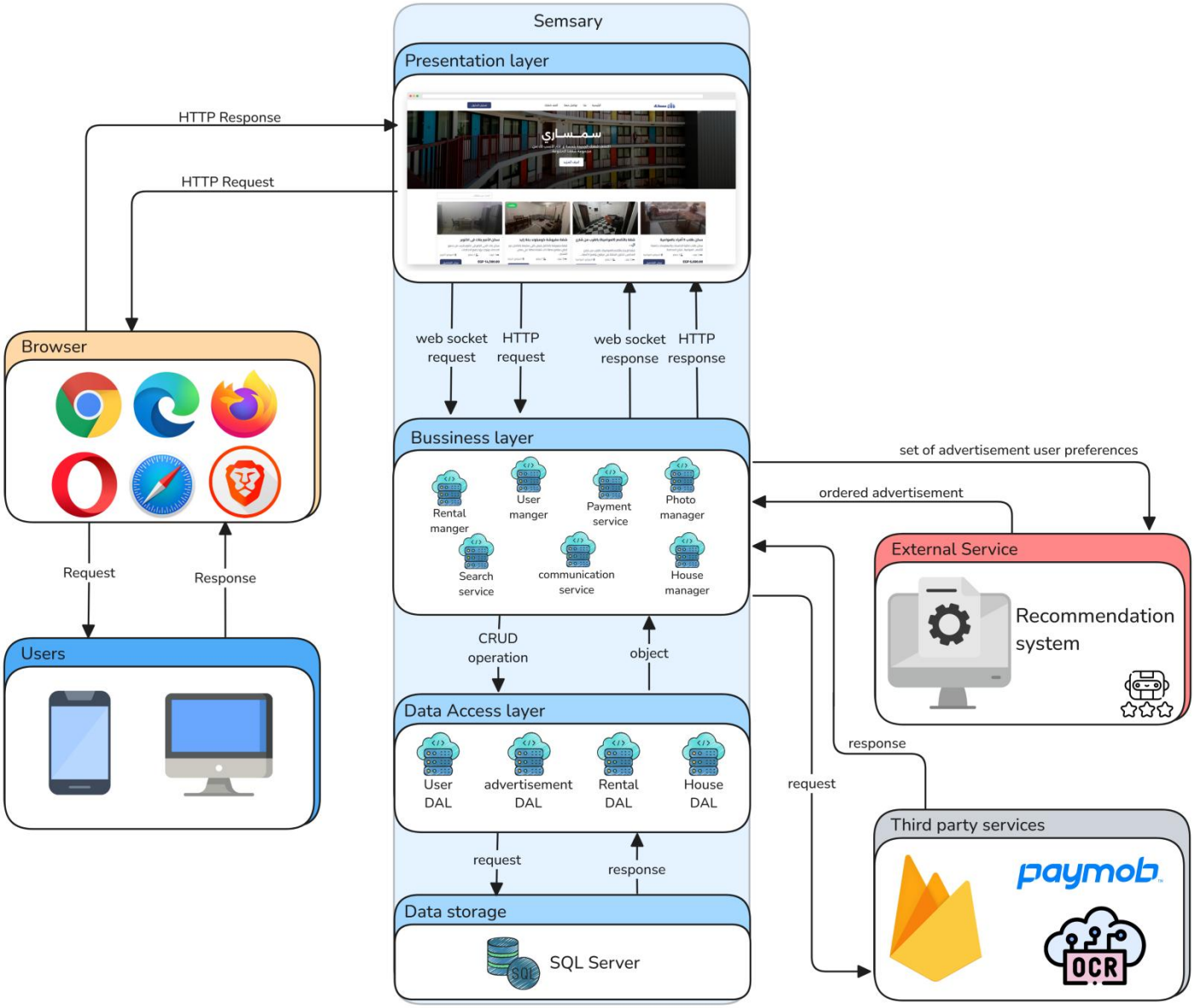


Figure 11: System Architecture diagram

4.4 Analysis Class

This section provides an in-depth examination of the primary classes identified during the analysis phase of the system design. Each class is defined based on its responsibilities, attributes, and interactions with other classes. The following elements are covered:

Class Identification: A list of core classes that form the foundation of the system.

Attributes and Methods: Key attributes and associated methods for each class, emphasizing their role in fulfilling system requirements.

Relationships: A detailed explanation of the associations, aggregations, or inheritance relationships between classes.

Use Case Mapping: How each class aligns with the functional requirements and use cases of the system.

Scalability Considerations: Insights into how the class design supports future system growth and extensibility.

This section lays the groundwork for transitioning from high-level requirements to the detailed design and implementation phases.

4.4.1 Context Diagram (Level Zero diagram)

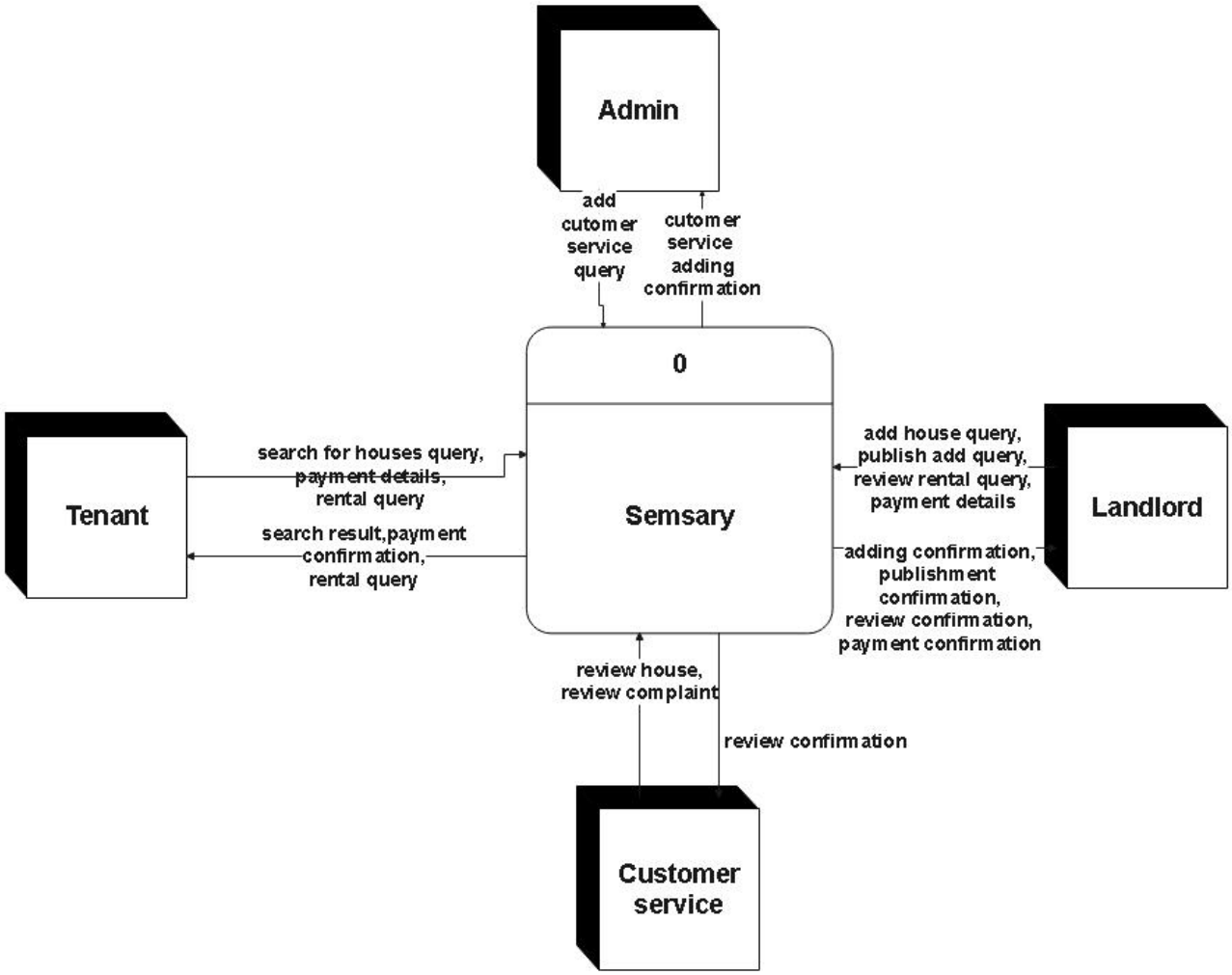


Figure 12: Context Diagram (Level Zero diagram)

4.4.2 Data flow diagram

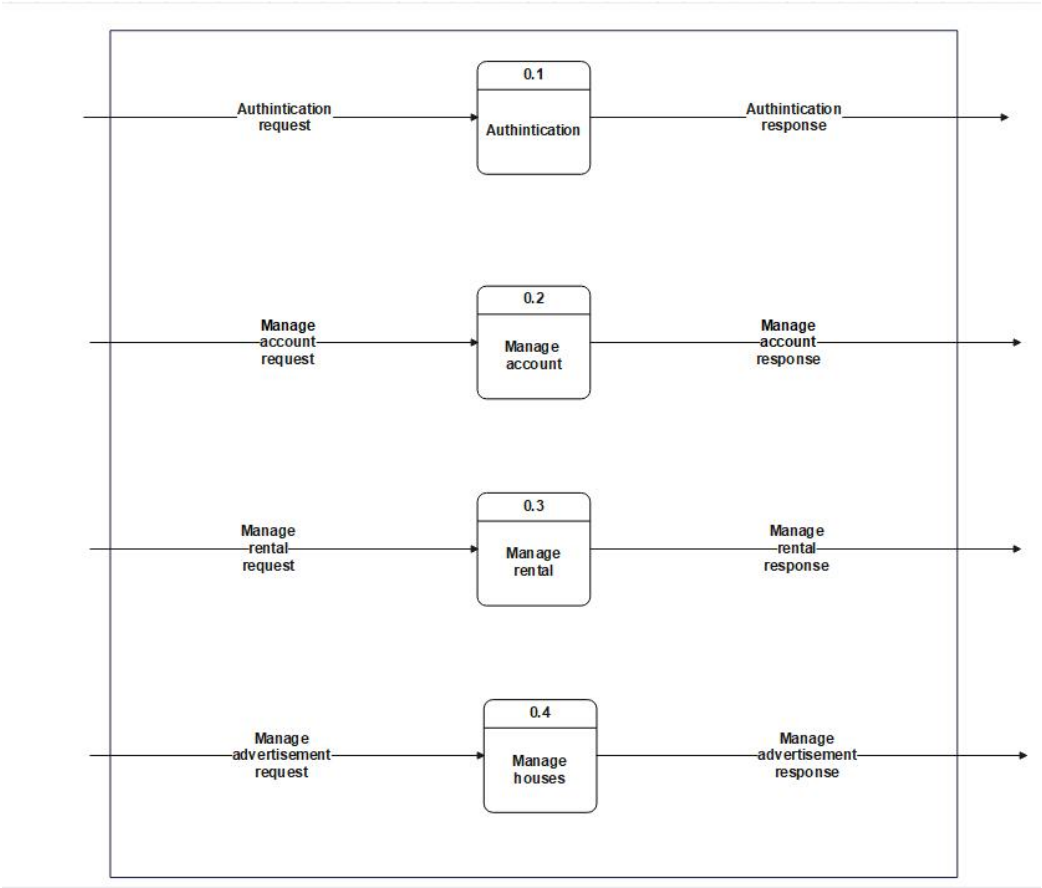


Figure 13: Data flow diagram

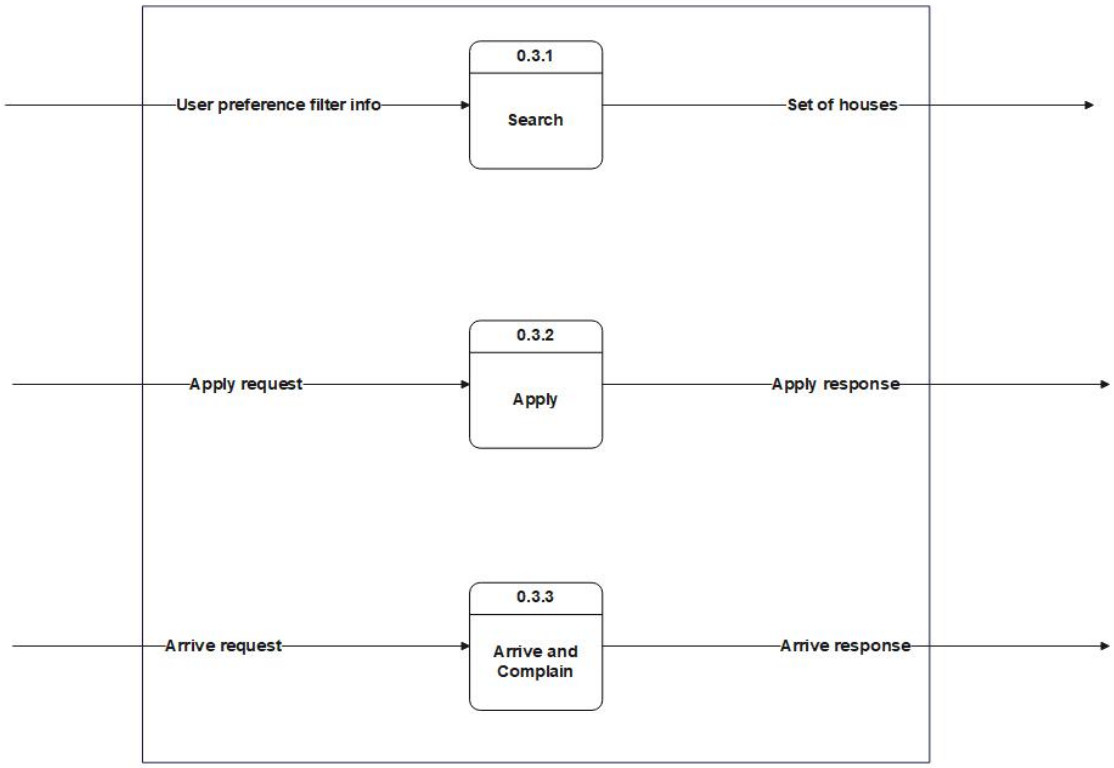


Figure 14: Data Flow Diagram 1

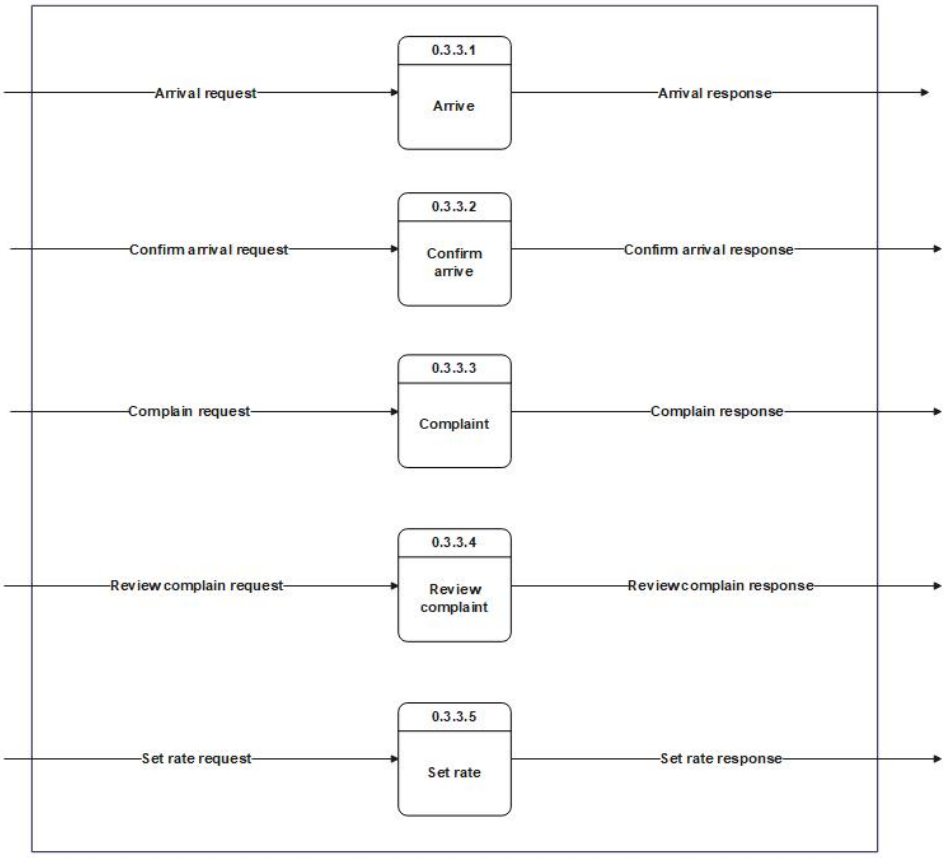


Figure 15: Data Flow Diagram 2

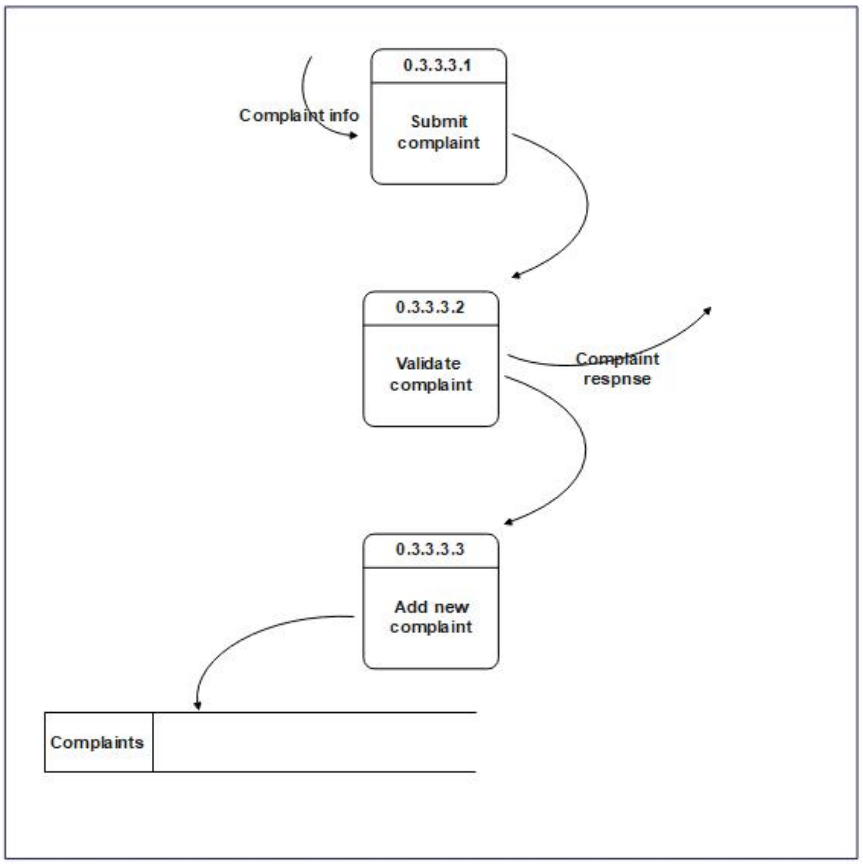


Figure 16: Data Flow Diagram 3

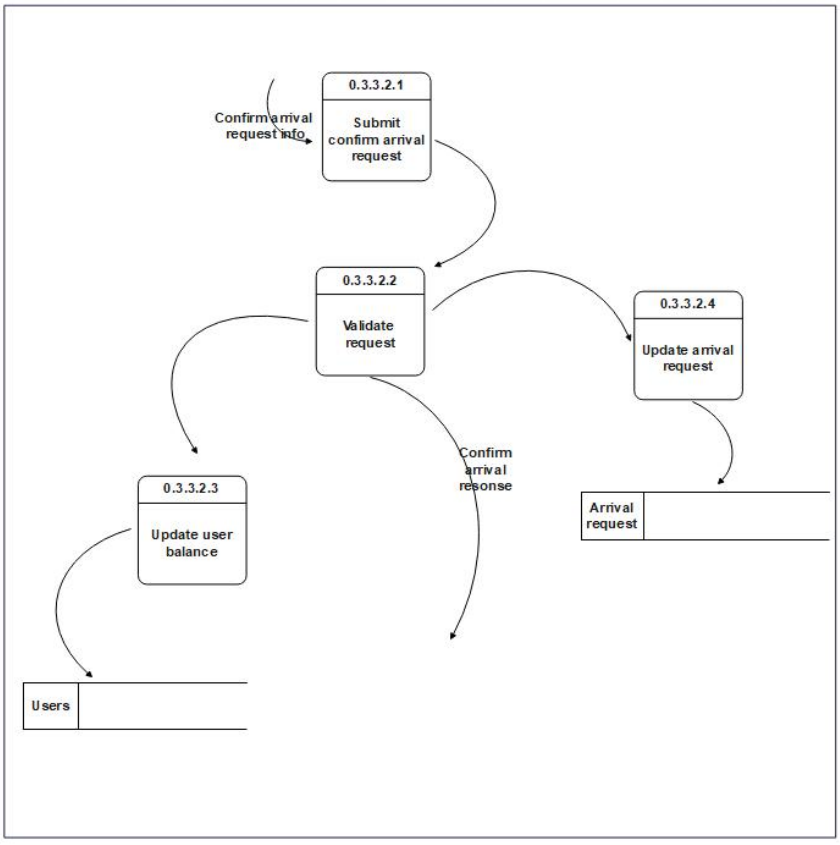


Figure 17: Data Flow Diagram 4

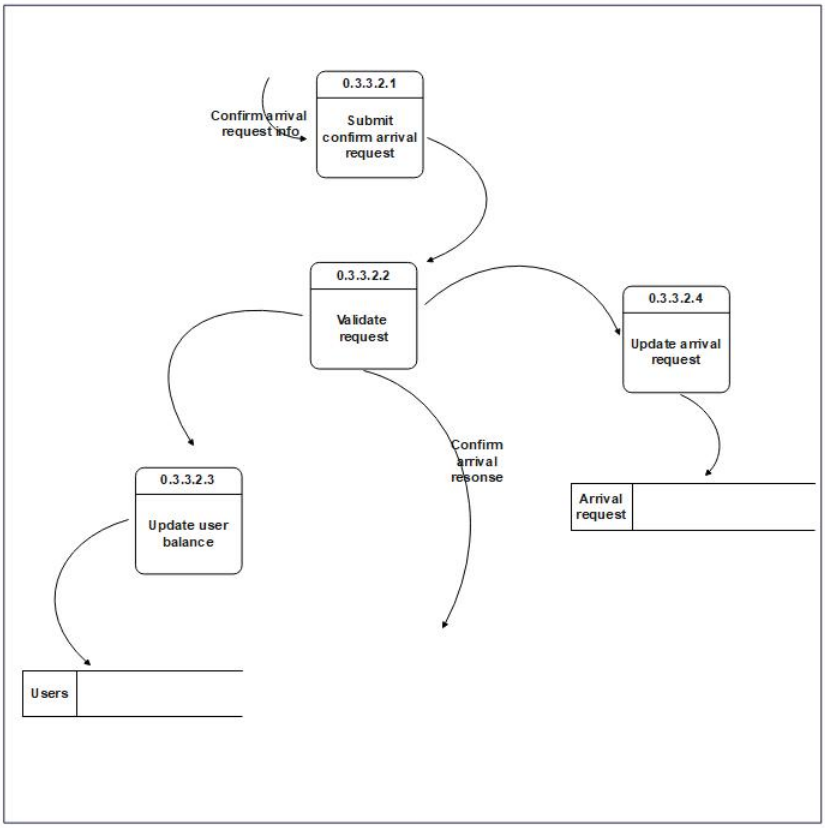


Figure 18: Data Flow Diagram 5

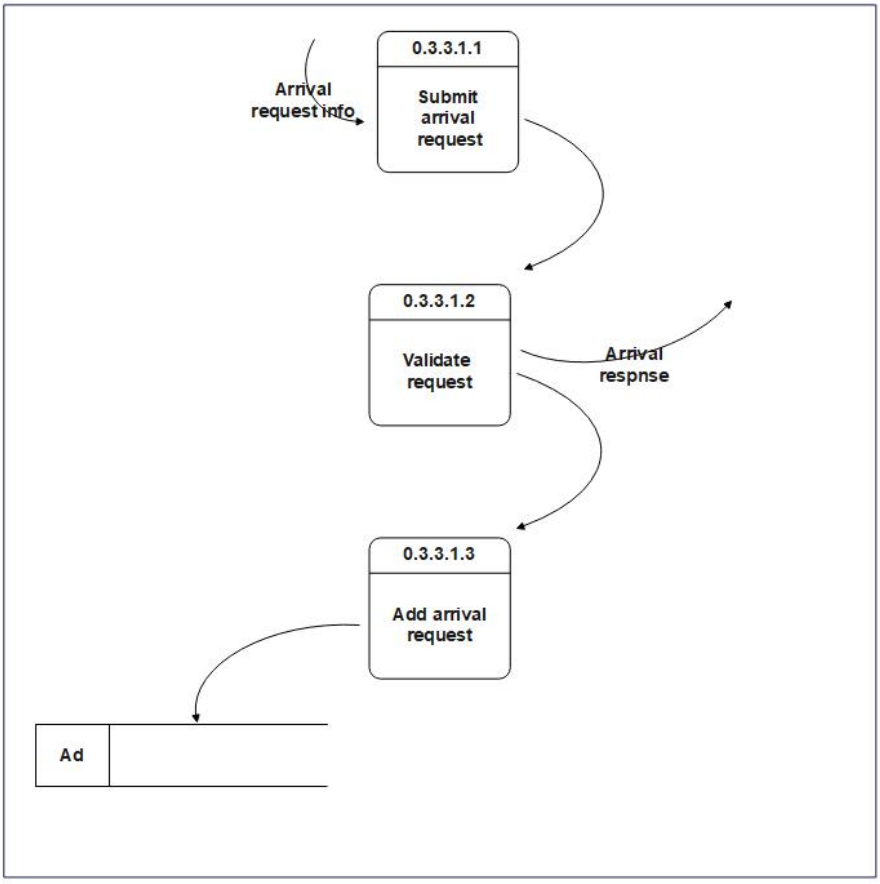


Figure 19: Data Flow Diagram 6

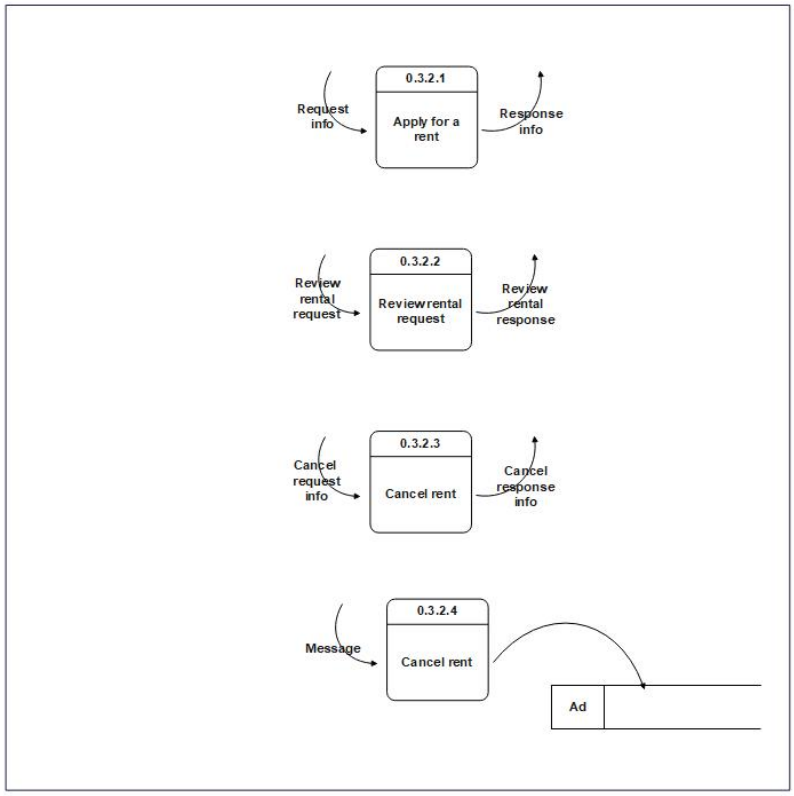


Figure 20: Data Flow Diagram 7

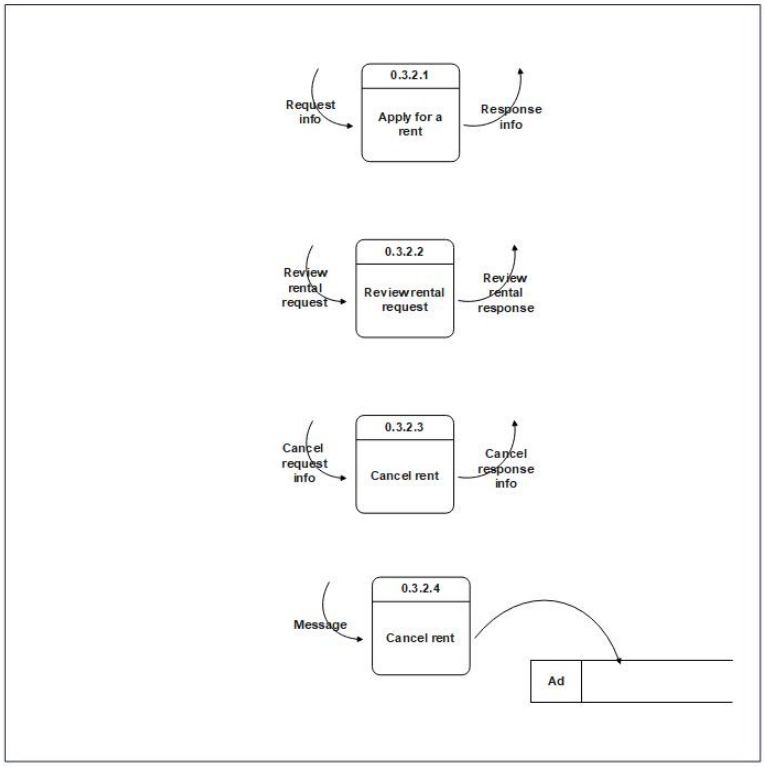


Figure 21: Data Flow Diagram 8

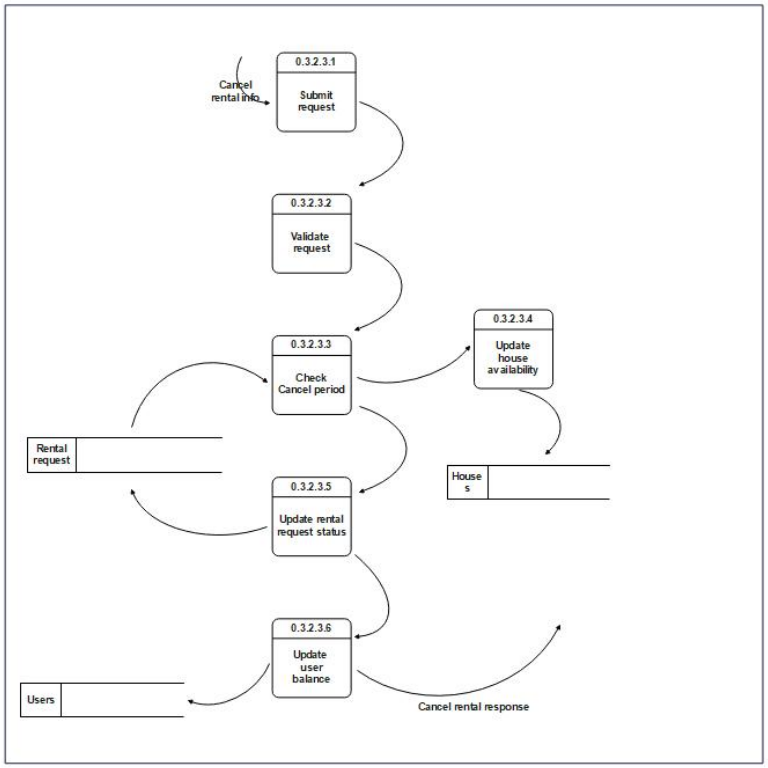


Figure 22: Data Flow Diagram 9

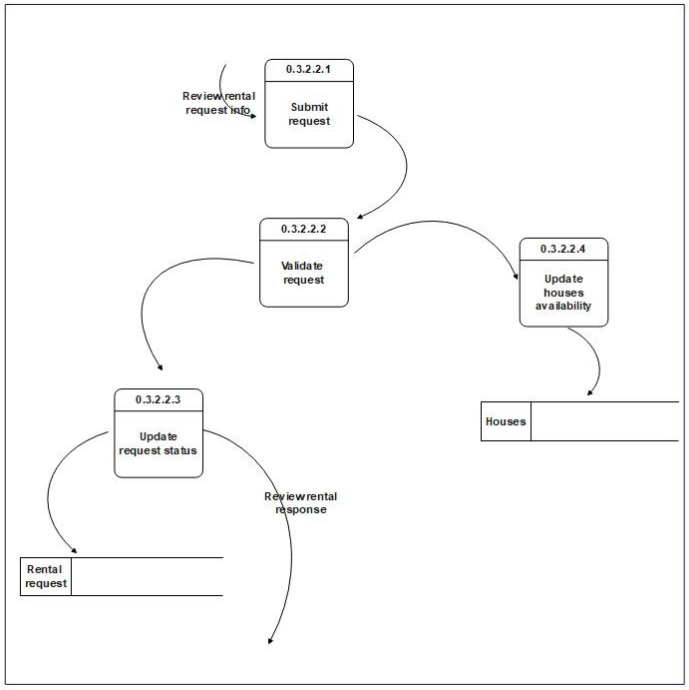


Figure 23: Data Flow Diagram 10

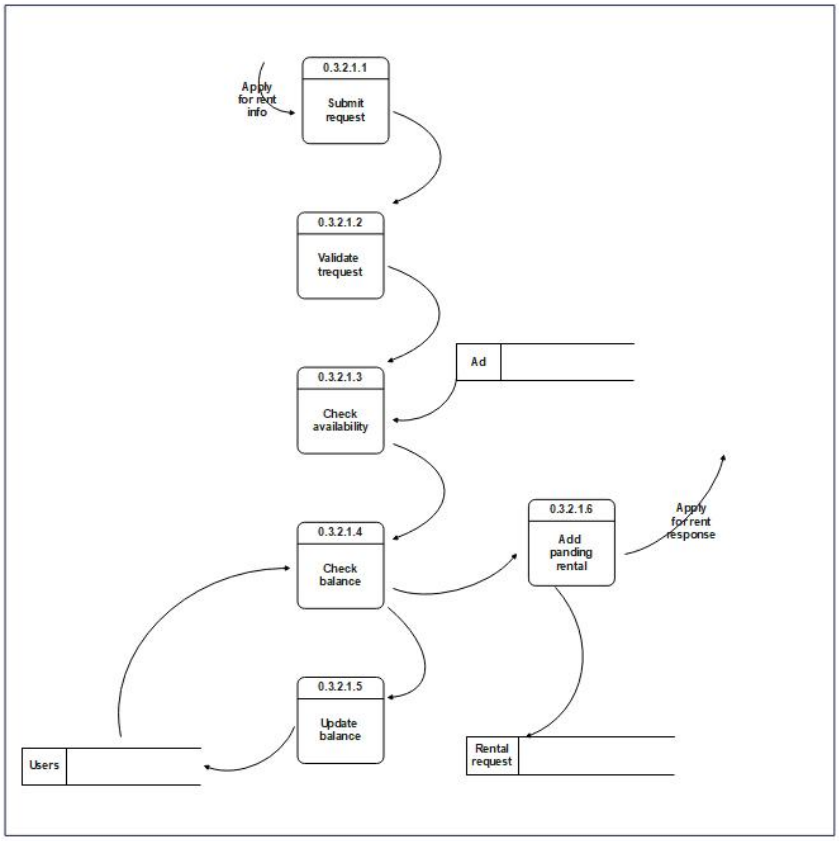


Figure 24: Data Flow Diagram 12

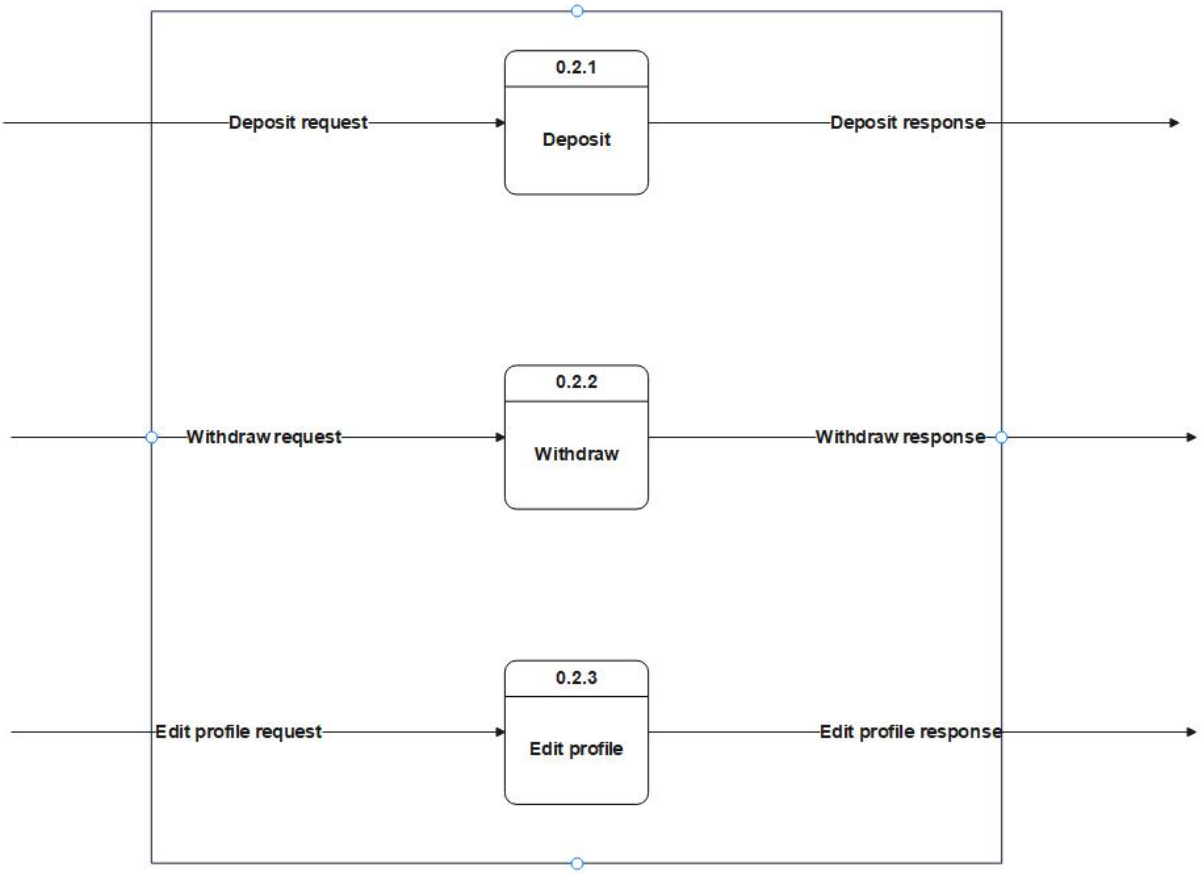


Figure 25: Data Flow Diagram 13

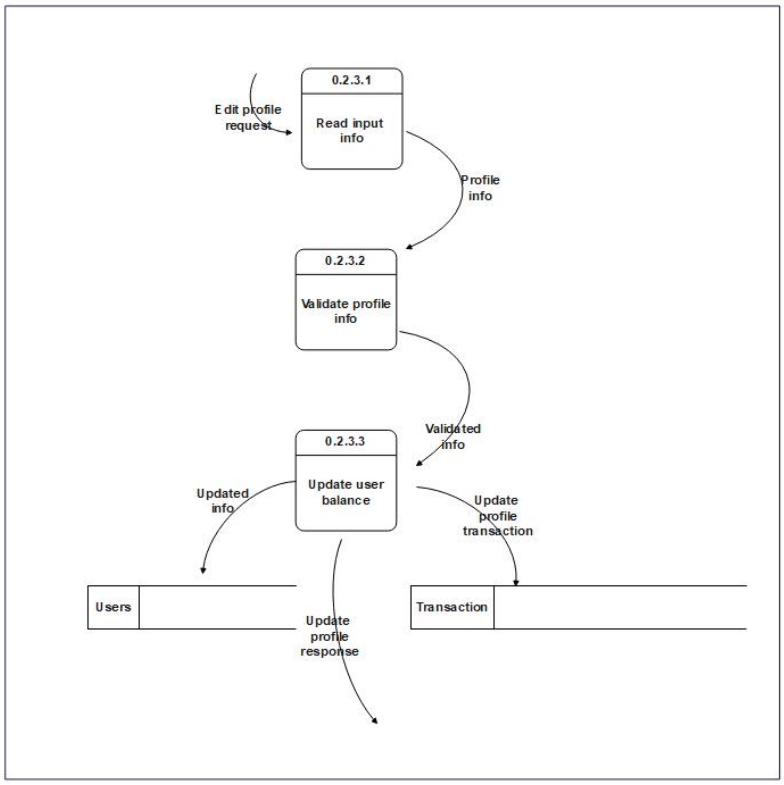


Figure 26: Data Flow Diagram 14

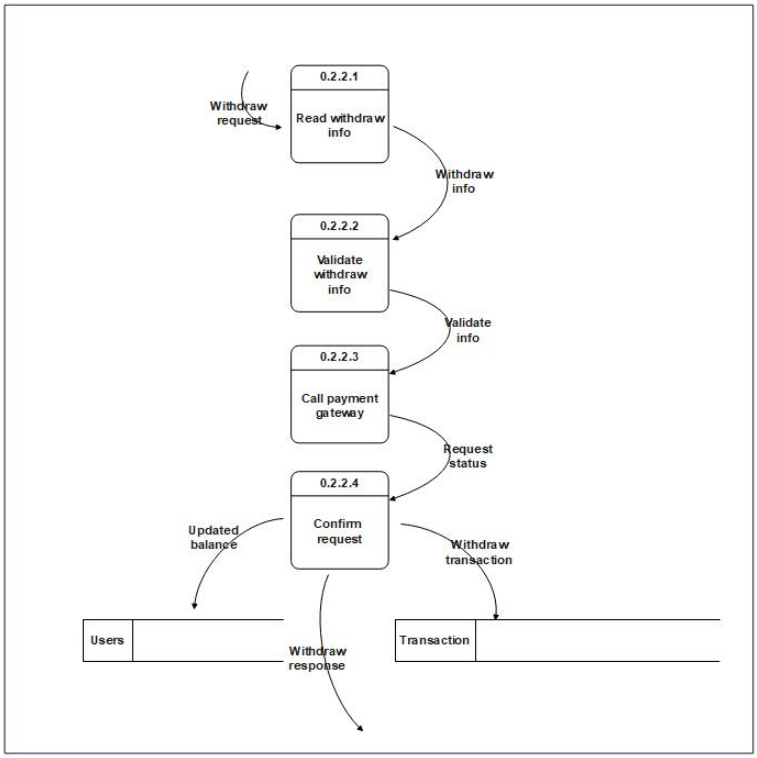


Figure 27: Data Flow Diagram 15

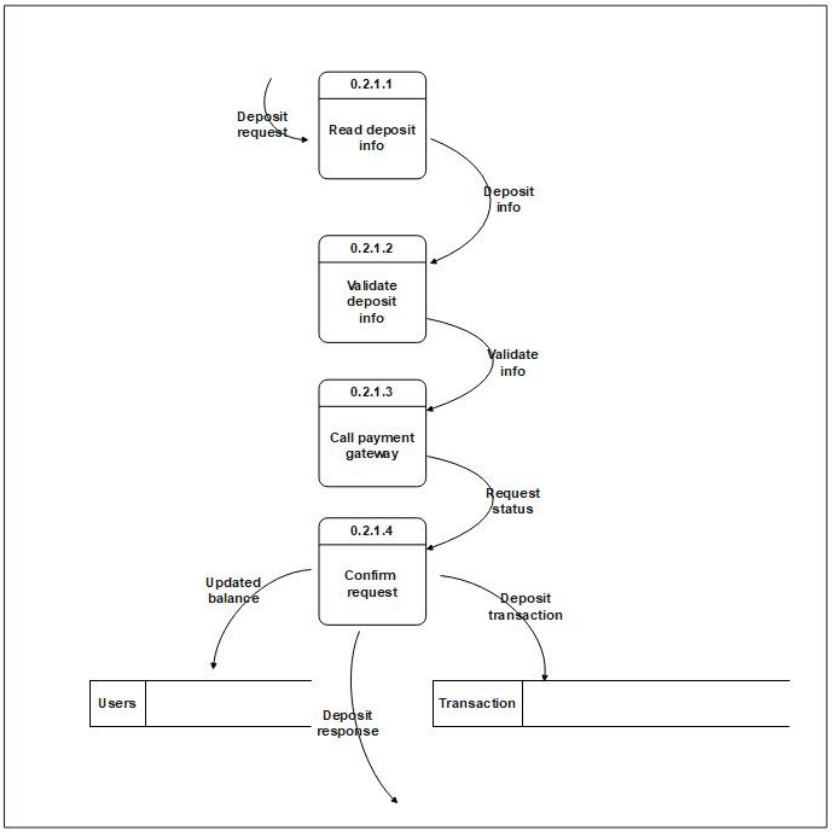


Figure 28: Data Flow Diagram 16

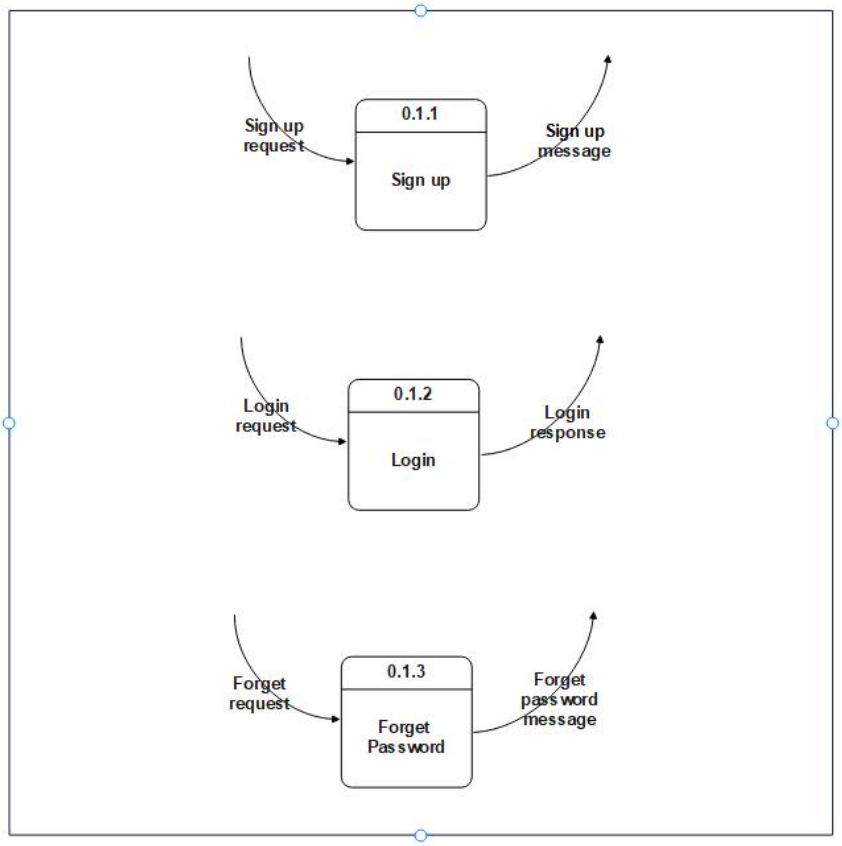


Figure 29: Data Flow Diagram 17

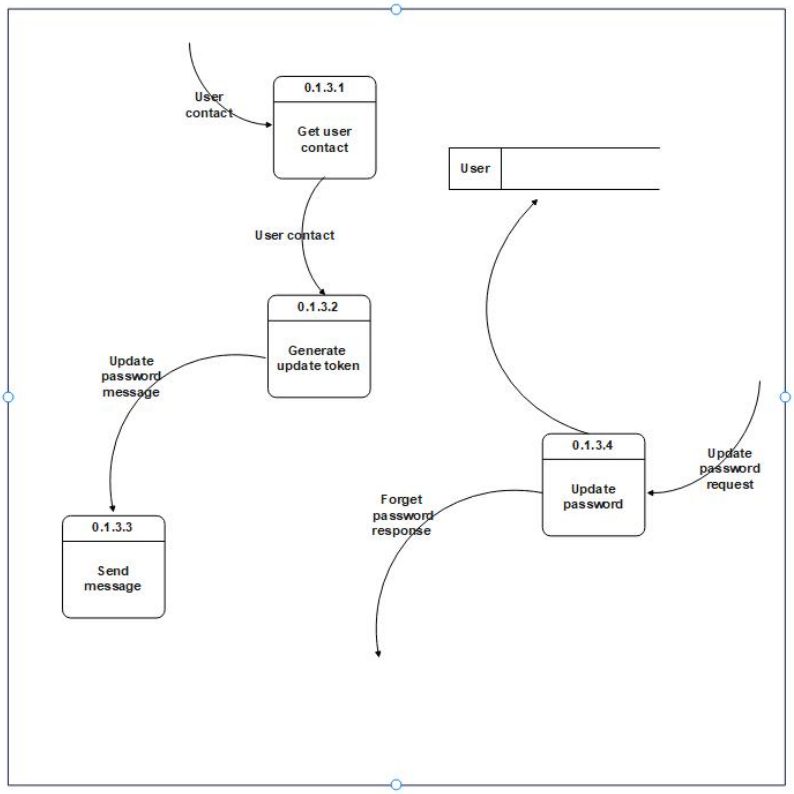


Figure 30: Data Flow Diagram 18

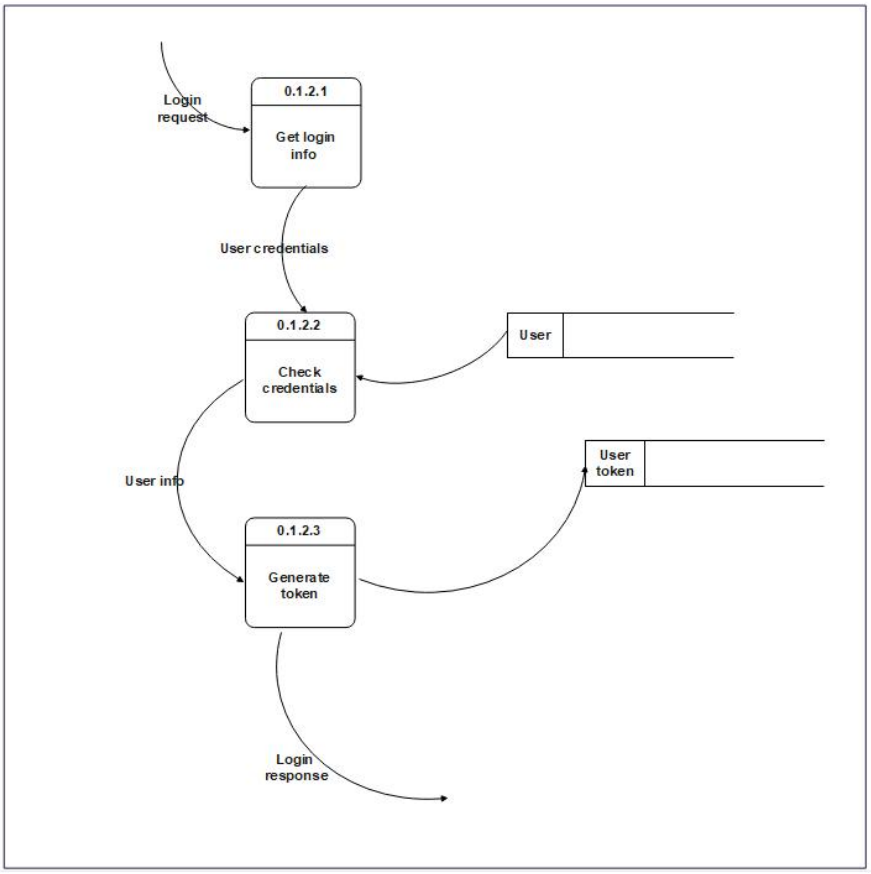


Figure 31: Data Flow Diagram 19

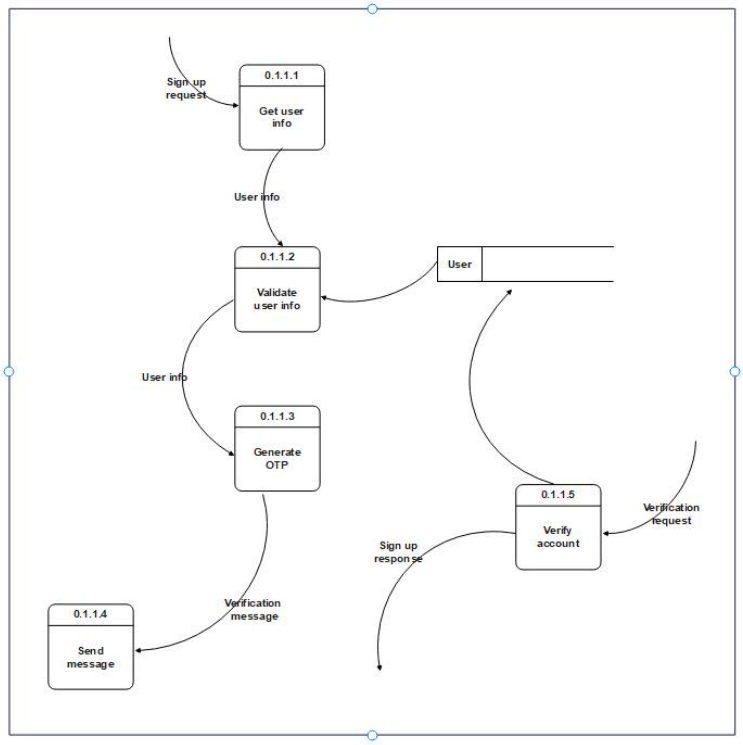


Figure 32: Data Flow Diagram 20

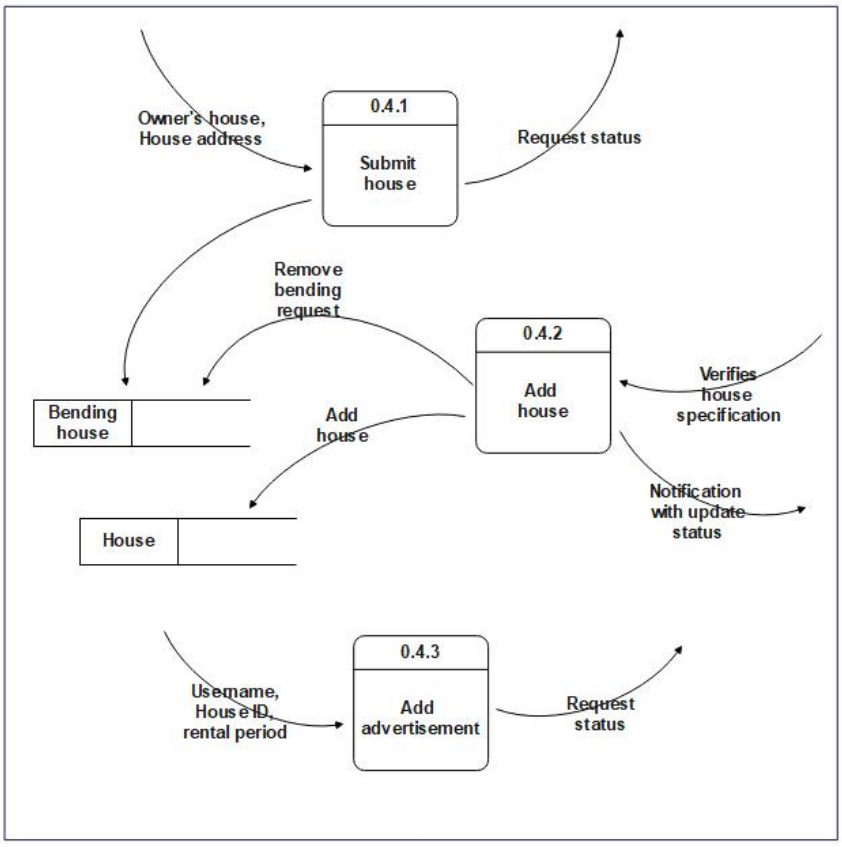


Figure 33: Data Flow Diagram 21

4.5 Activity diagrams

4.5.1 Clinet side activity

4.5.1.1 Landlord

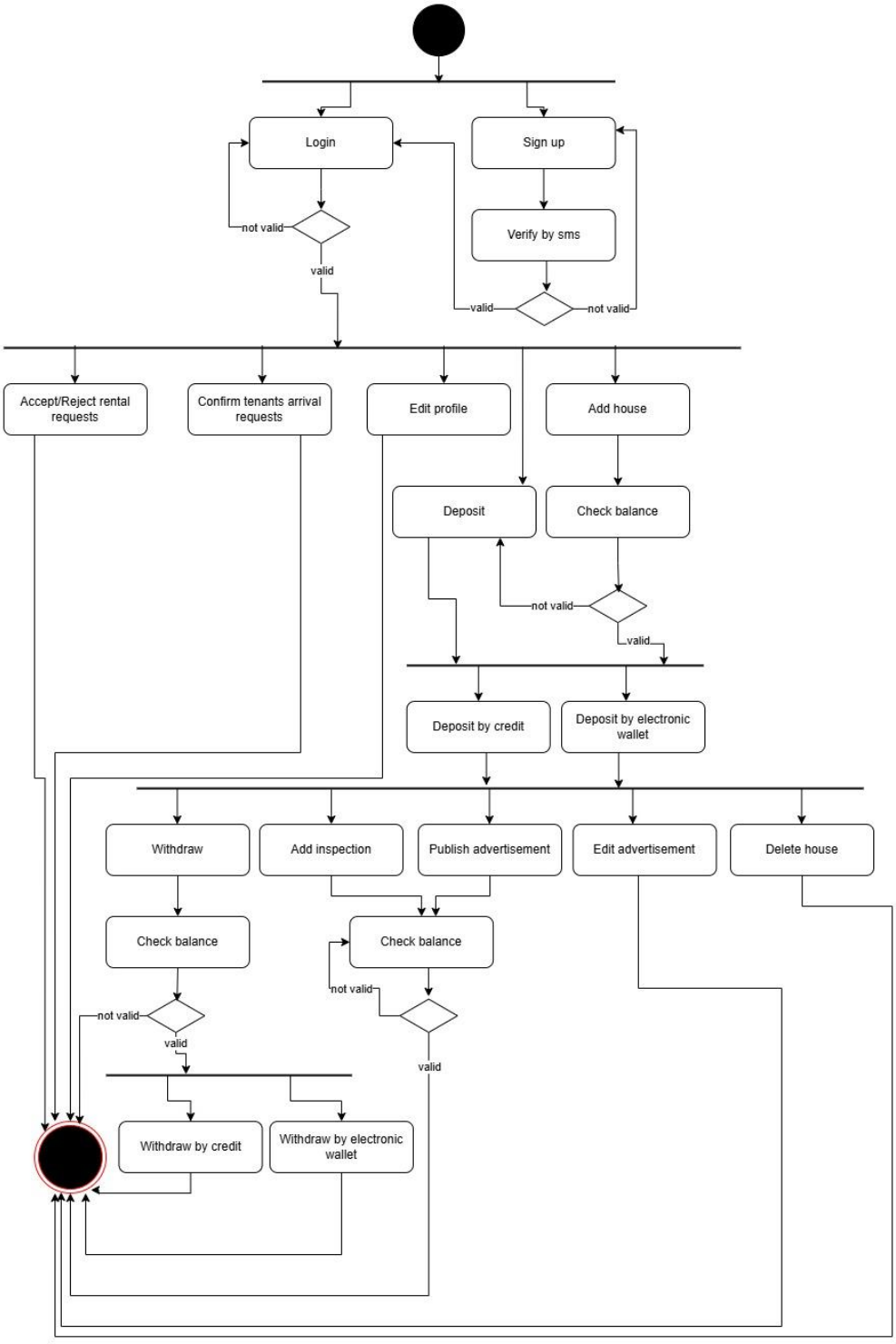


Figure 34: Activity diagrams - Landlord

4.5.1.2 Tenant

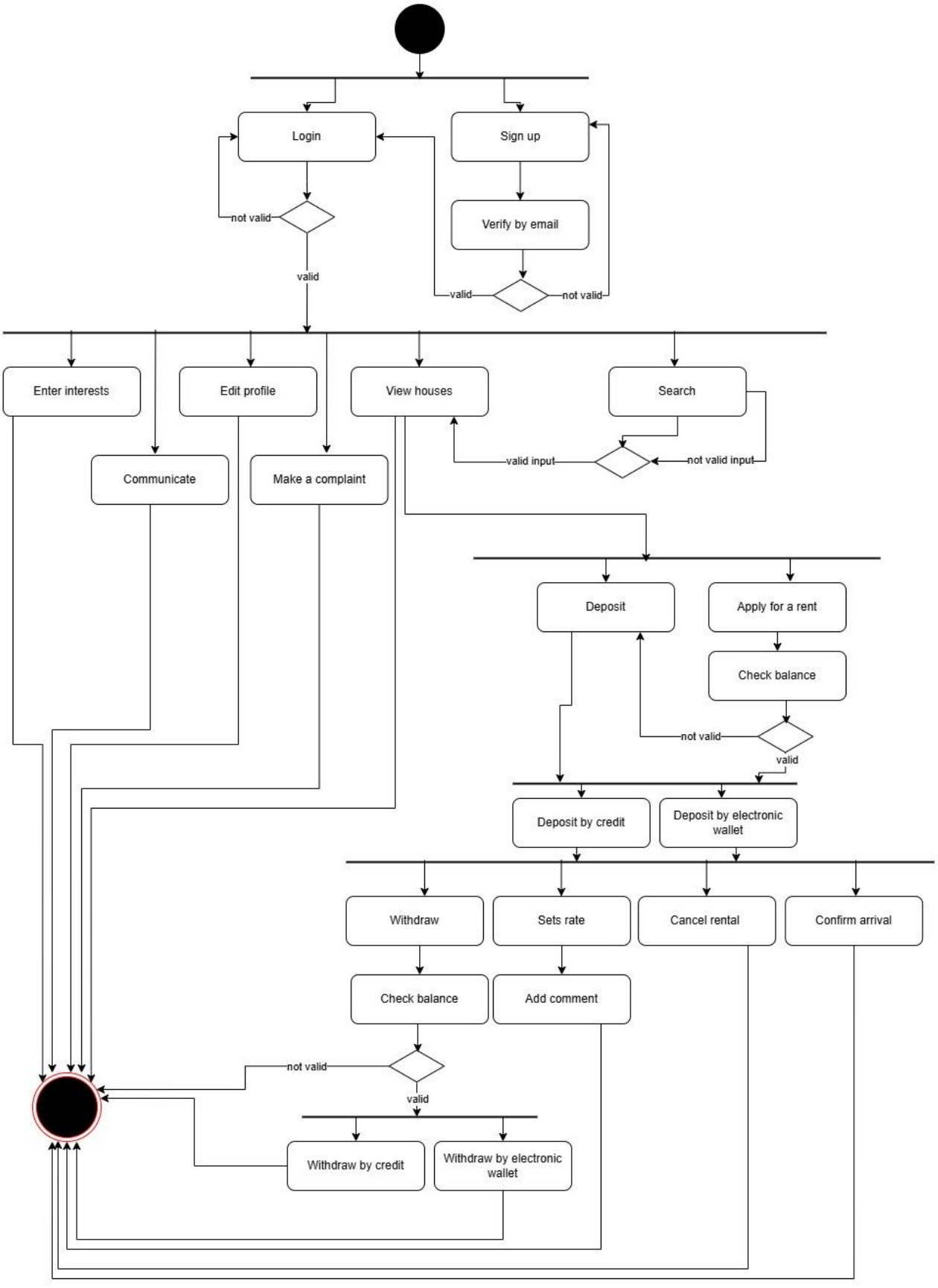


Figure 35: Activity diagrams - Tenant

4.5.2 Server side activity

4.5.2.1 Admin and Customer service

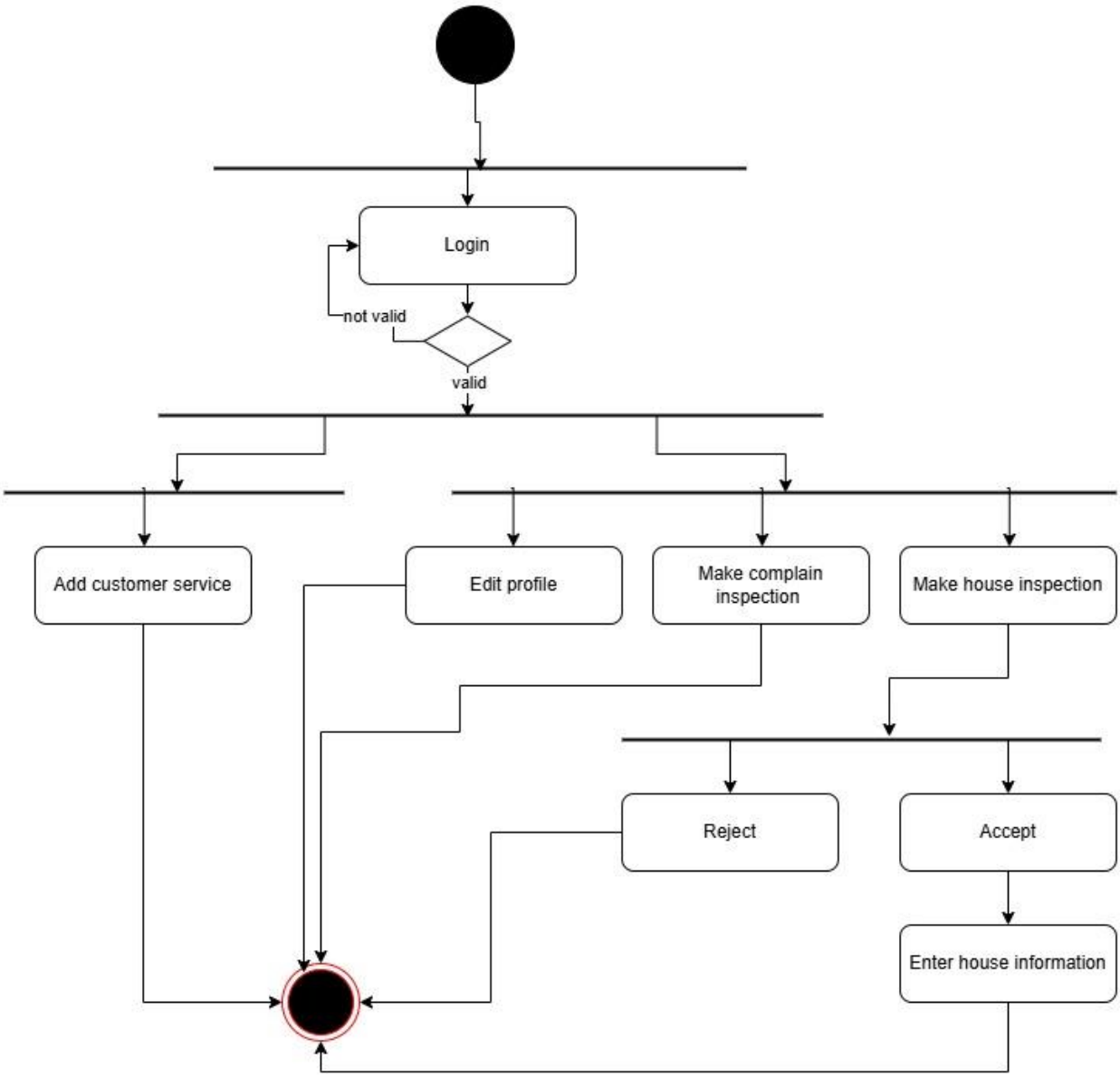


Figure 36: Activity diagrams - Admin

4.6 interaction class diagram

4.6.1 Sequence Diagram

4.6.1.1 **Add Customer Service**

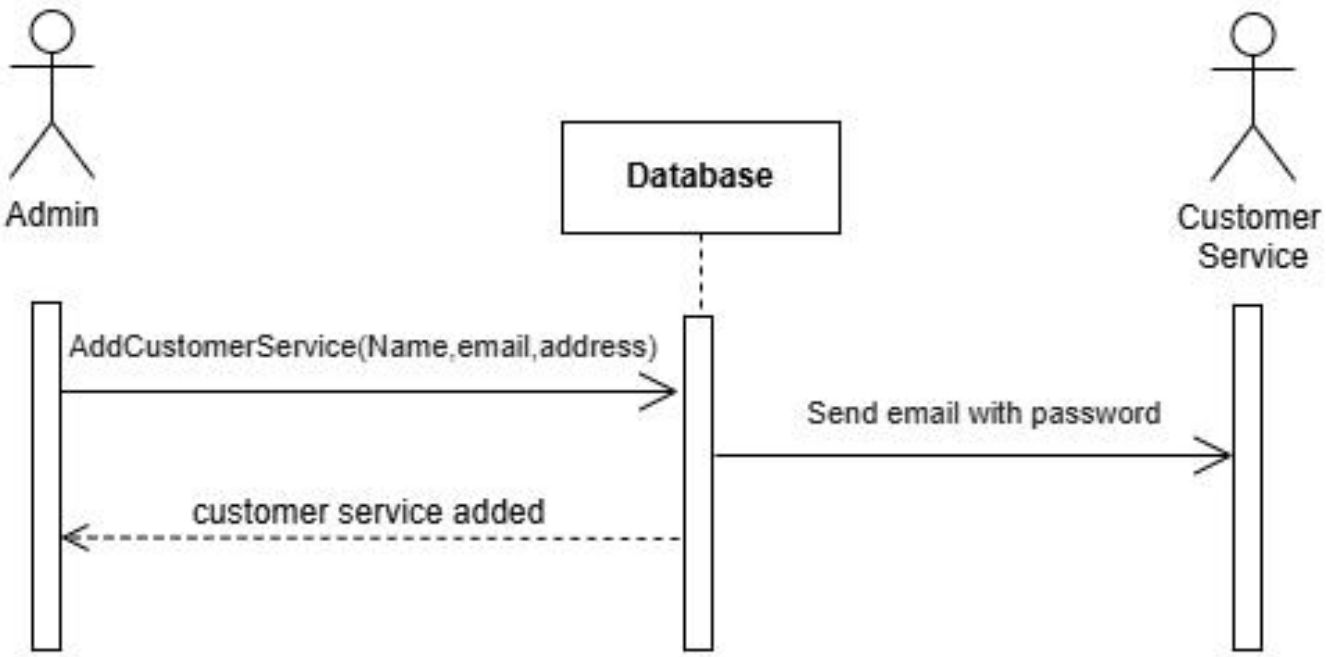


Figure 37: Sequence Diagram - Add Customer Service

4.6.1.2 **Add / Publish / Inspect House**

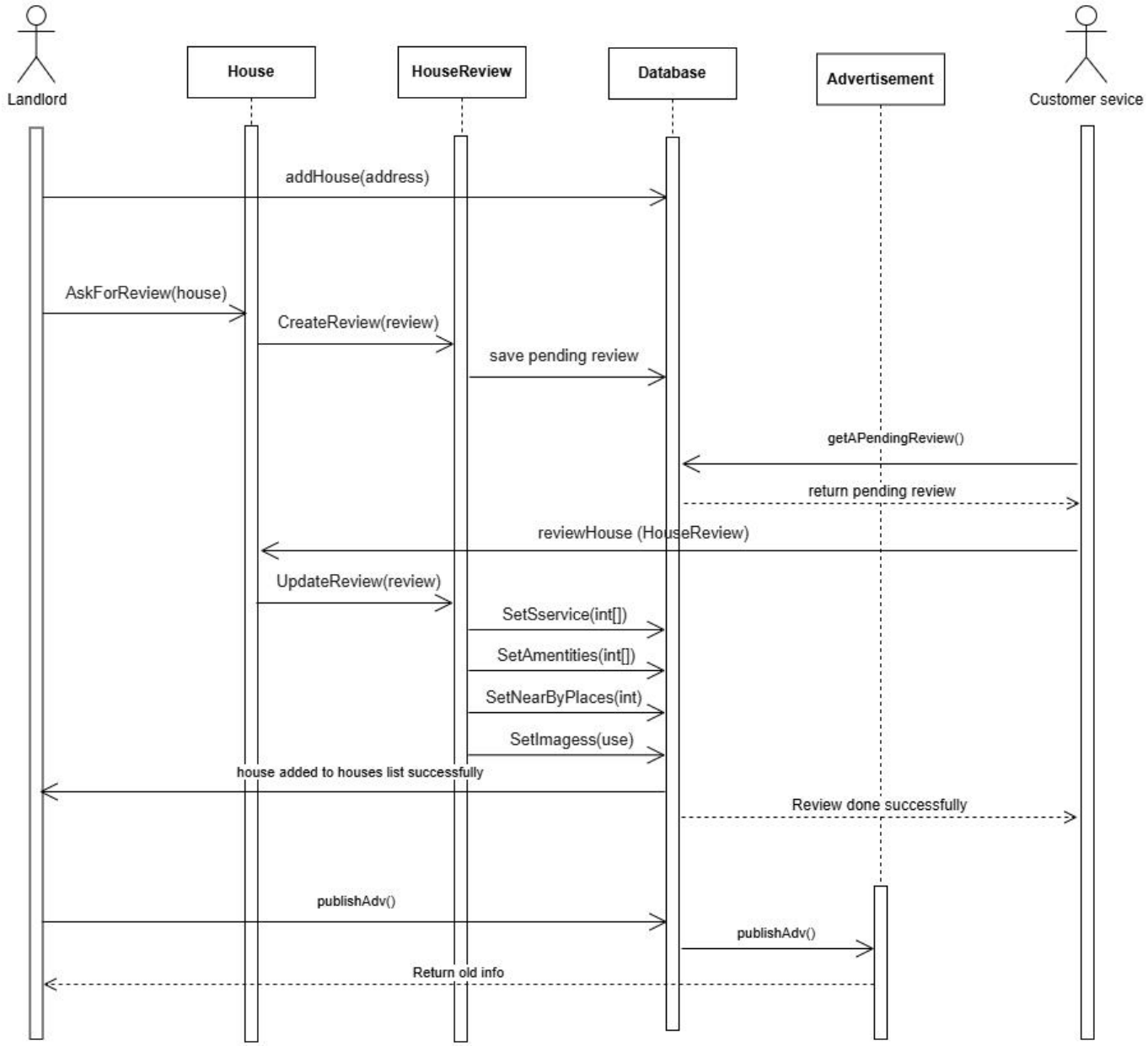


Figure 38: Sequence Diagram - Add / Publish / Inspect House

4.6.1.3 **Admin Edit Profile**

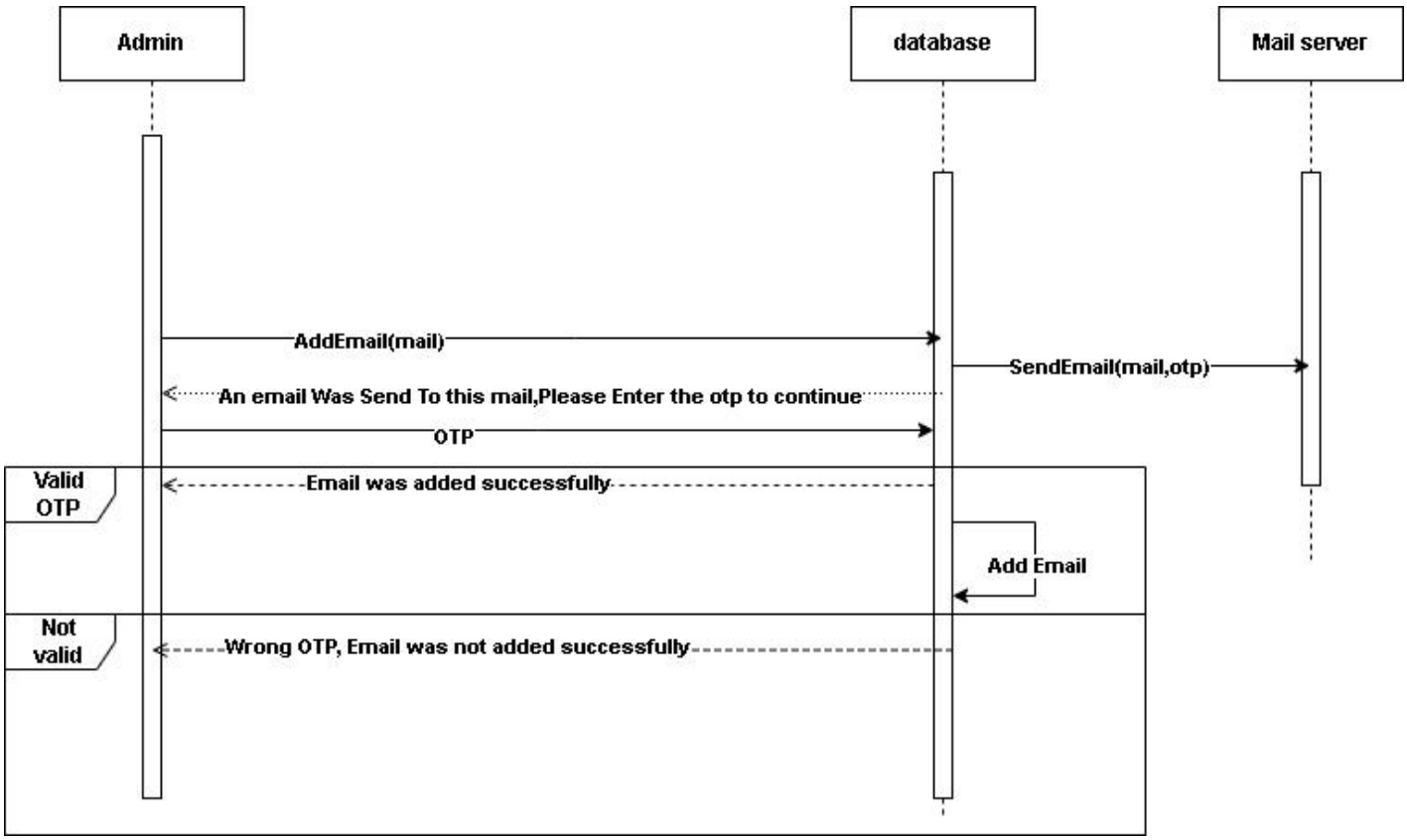


Figure 39: Sequence Diagram - Admin Edit Profile

4.6.1.4 **Cancel Rental / Confirm Arrival**

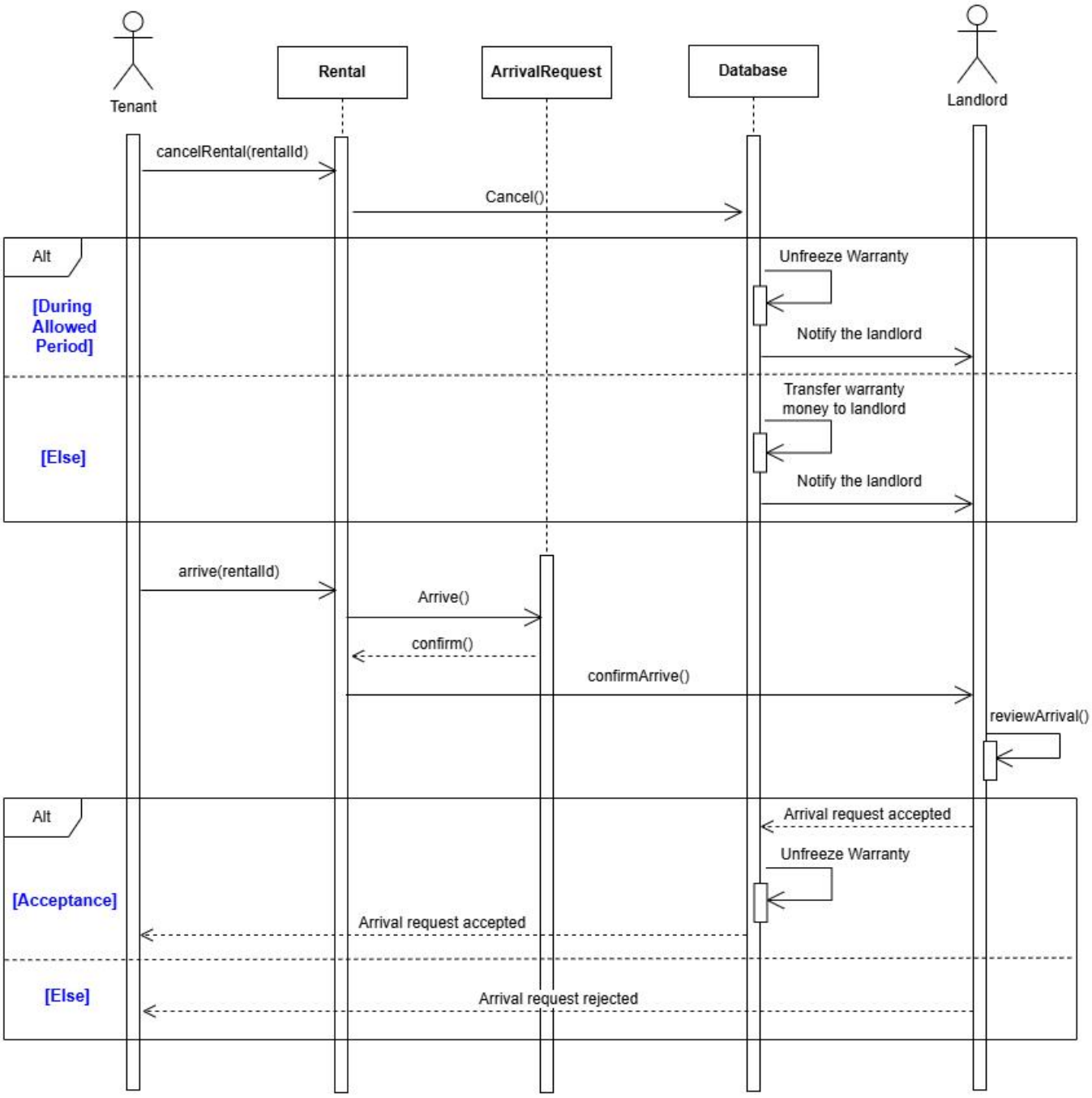


Figure 40: Sequence Diagram - Cancel Rental / Confirm Arrival

4.6.1.5 **Landlord & Customer Service communication**

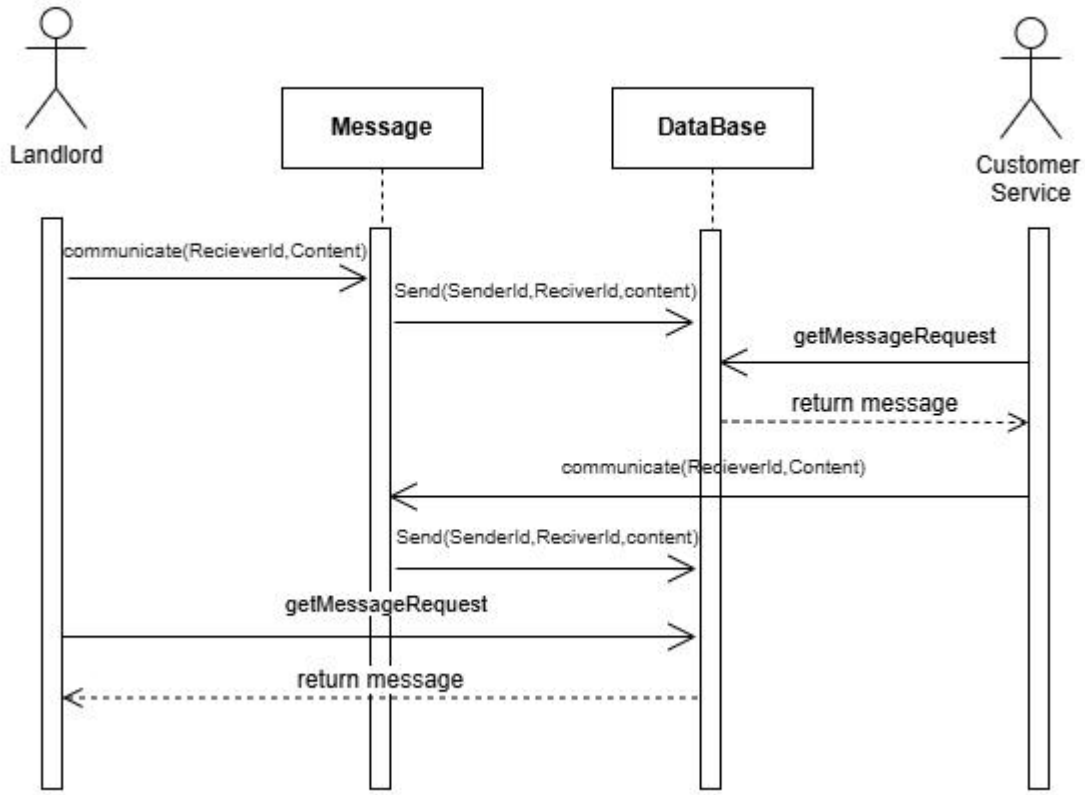


Figure 41: Sequence Diagram - Landlord & Customer Service communication

4.6.1.6 **Tenant & Landlord Communication**

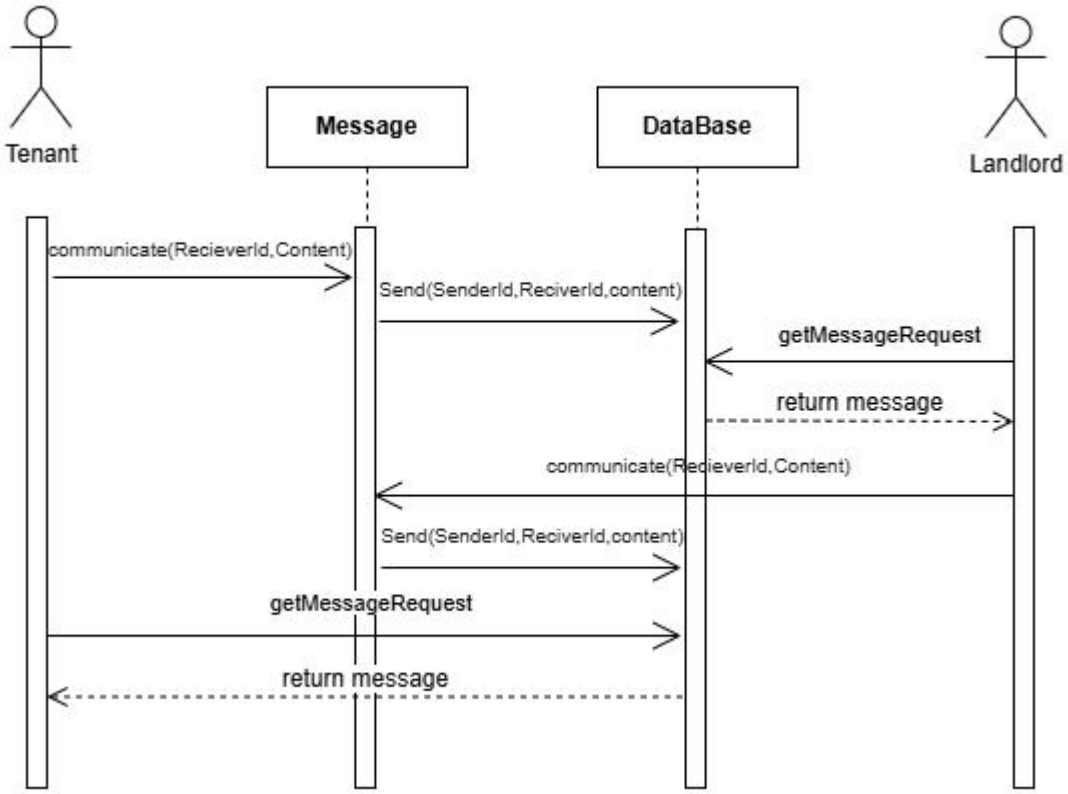


Figure 42: Sequence Diagram - Tenant & Landlord Communication

4.6.1.7 **Tenant & Customer service communication**

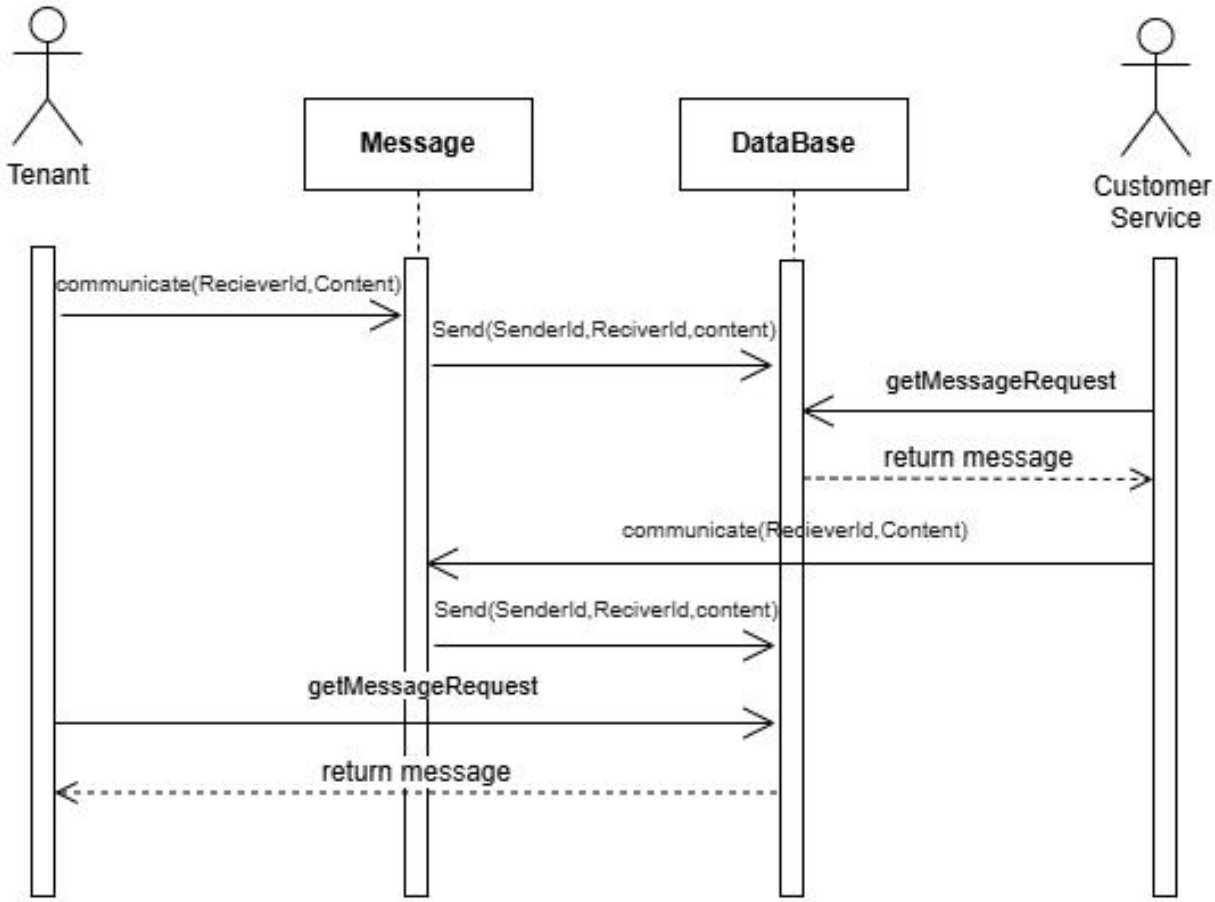


Figure 43: Sequence Diagram - Tenant & Customer service communication

4.6.1.8 **Customer Service Edit Profile**

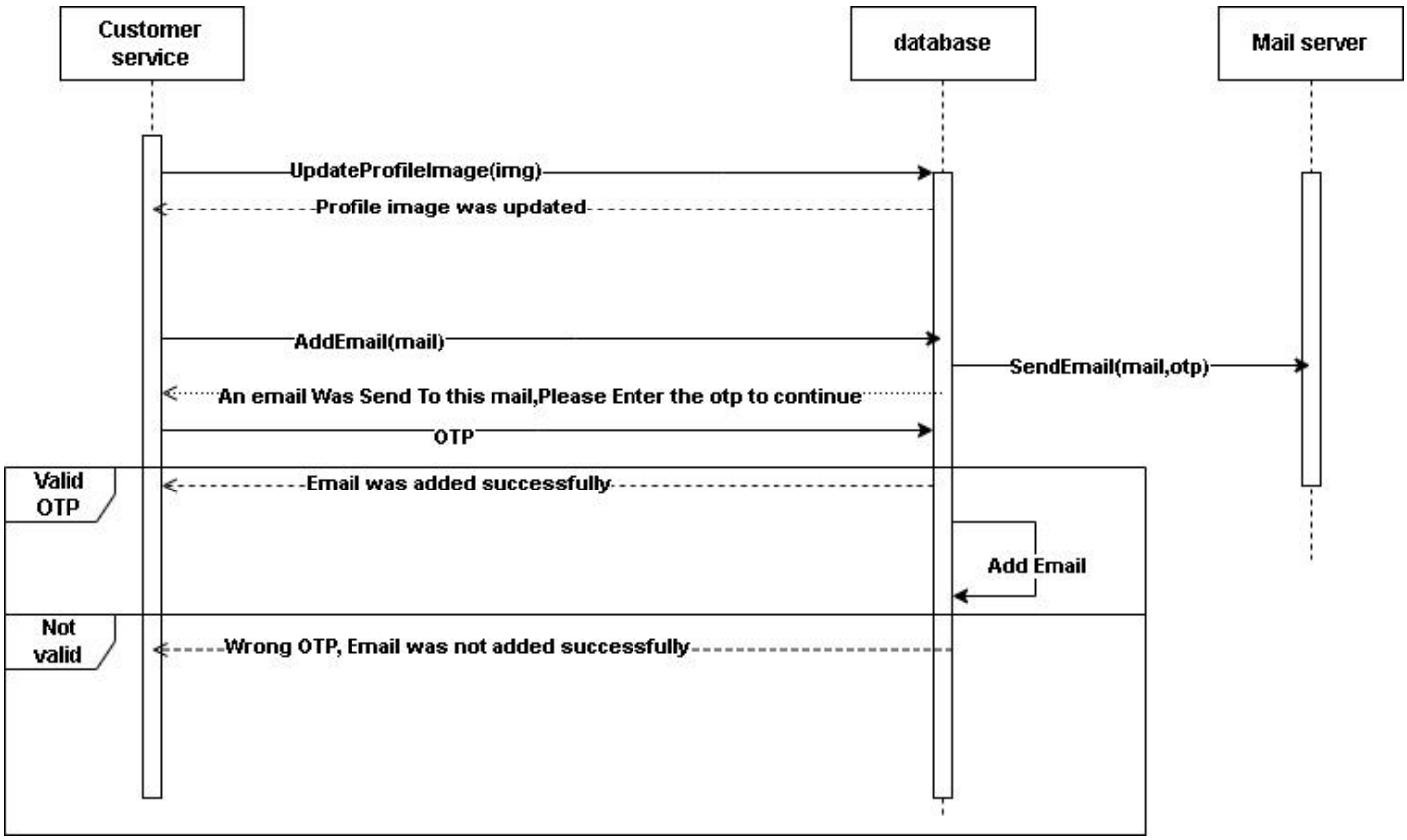


Figure 44: Sequence Diagram - Customer Service Edit Profile

4.6.1.9 **Delete House**

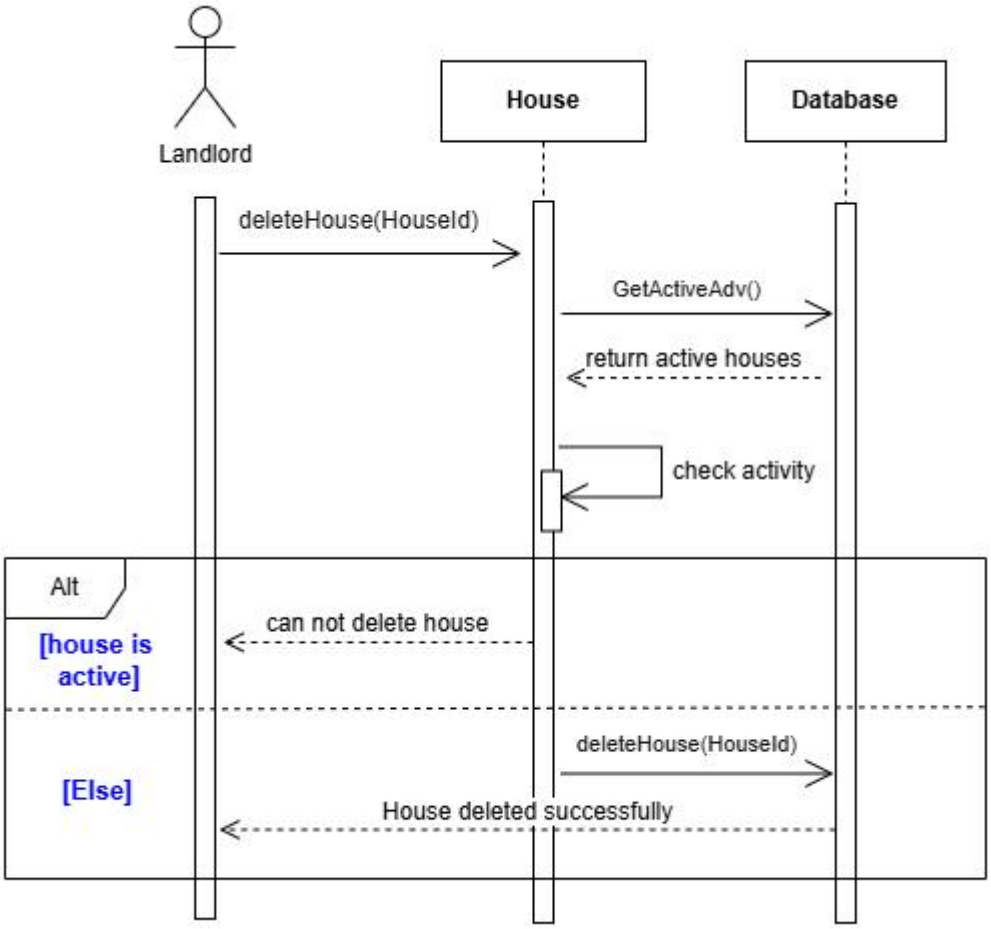


Figure 45: Sequence Diagram - Delete House

4.6.1.10 **Deposite / Withdraw**

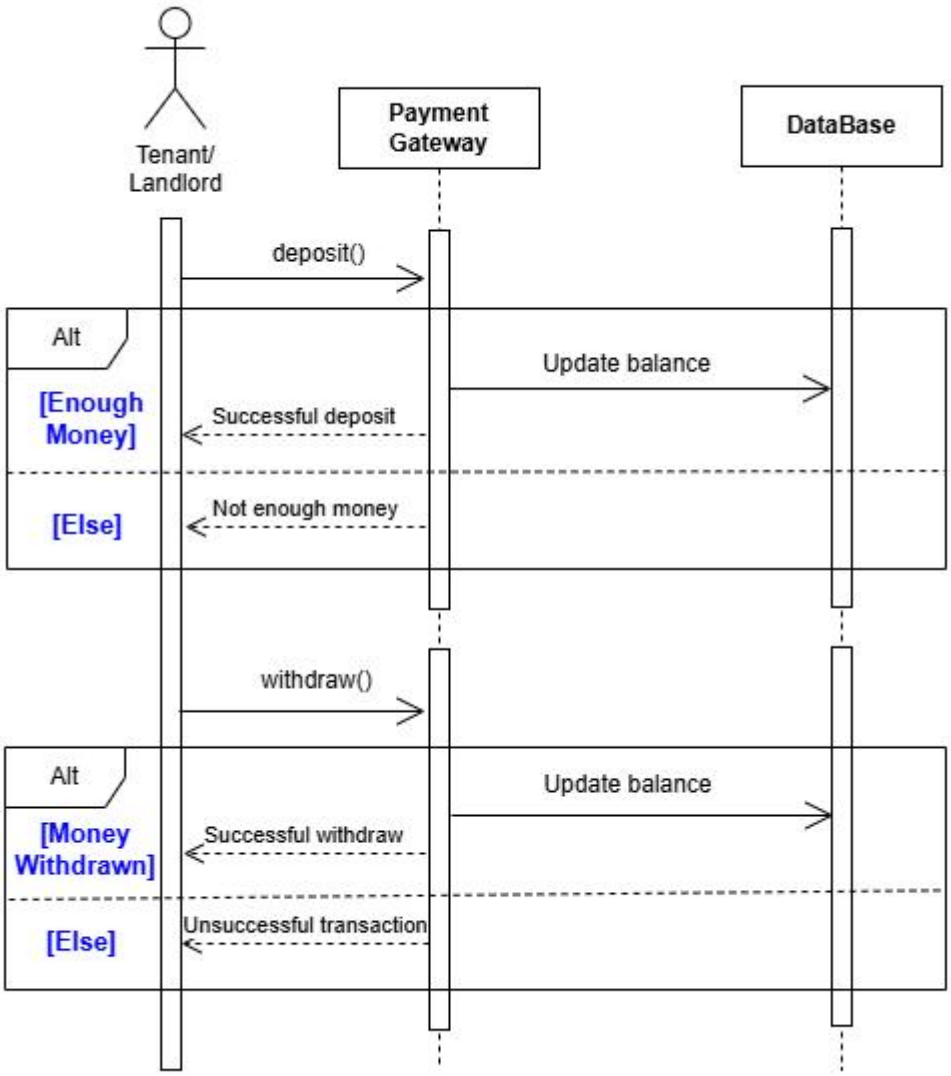


Figure 46: Sequence Diagram - Deposite / Withdraw

4.6.1.11 LandLord Edit Profile

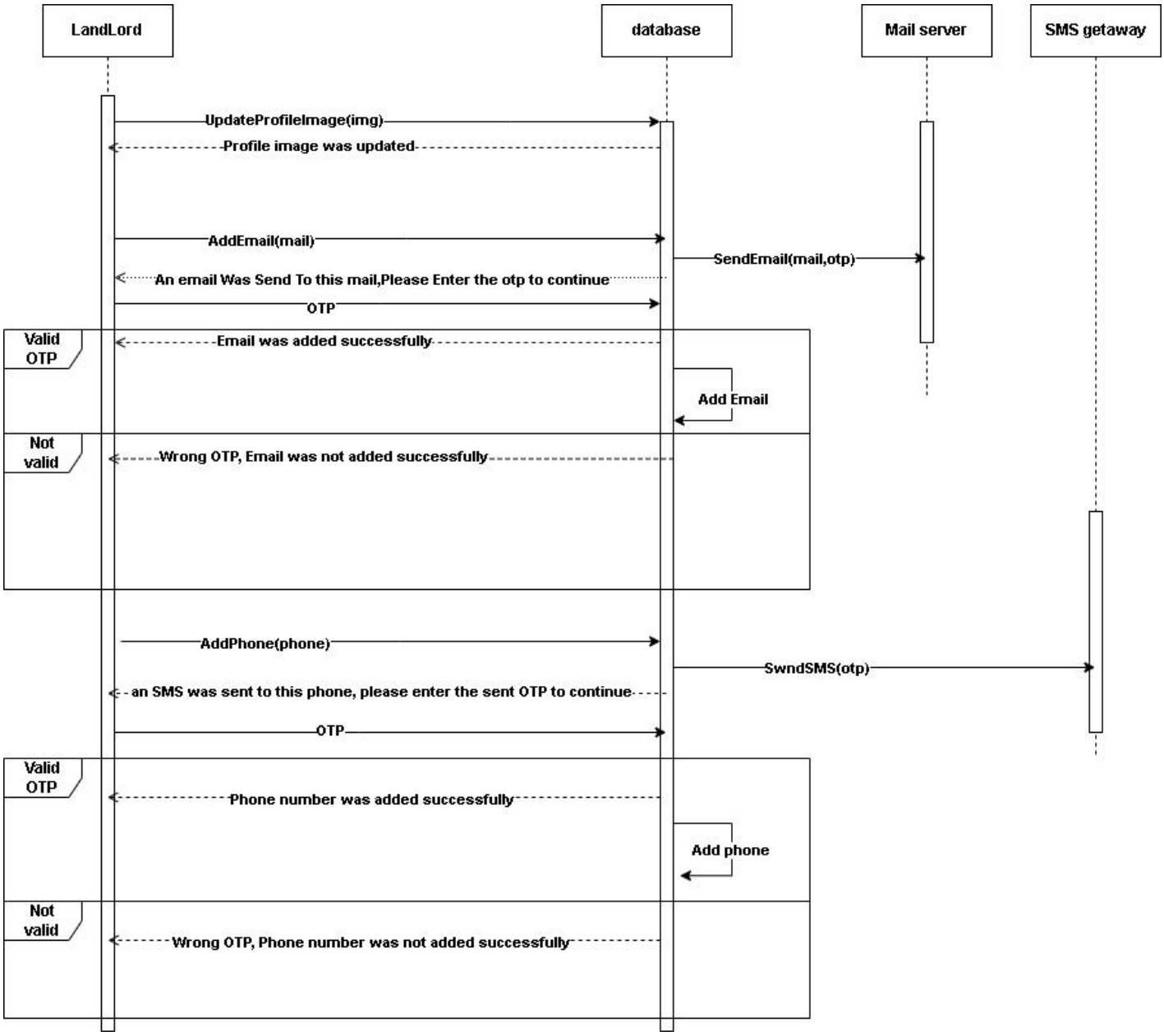


Figure 47: Sequence Diagram - LandLord Edit Profile

4.6.1.12 **Landlord SignUp**

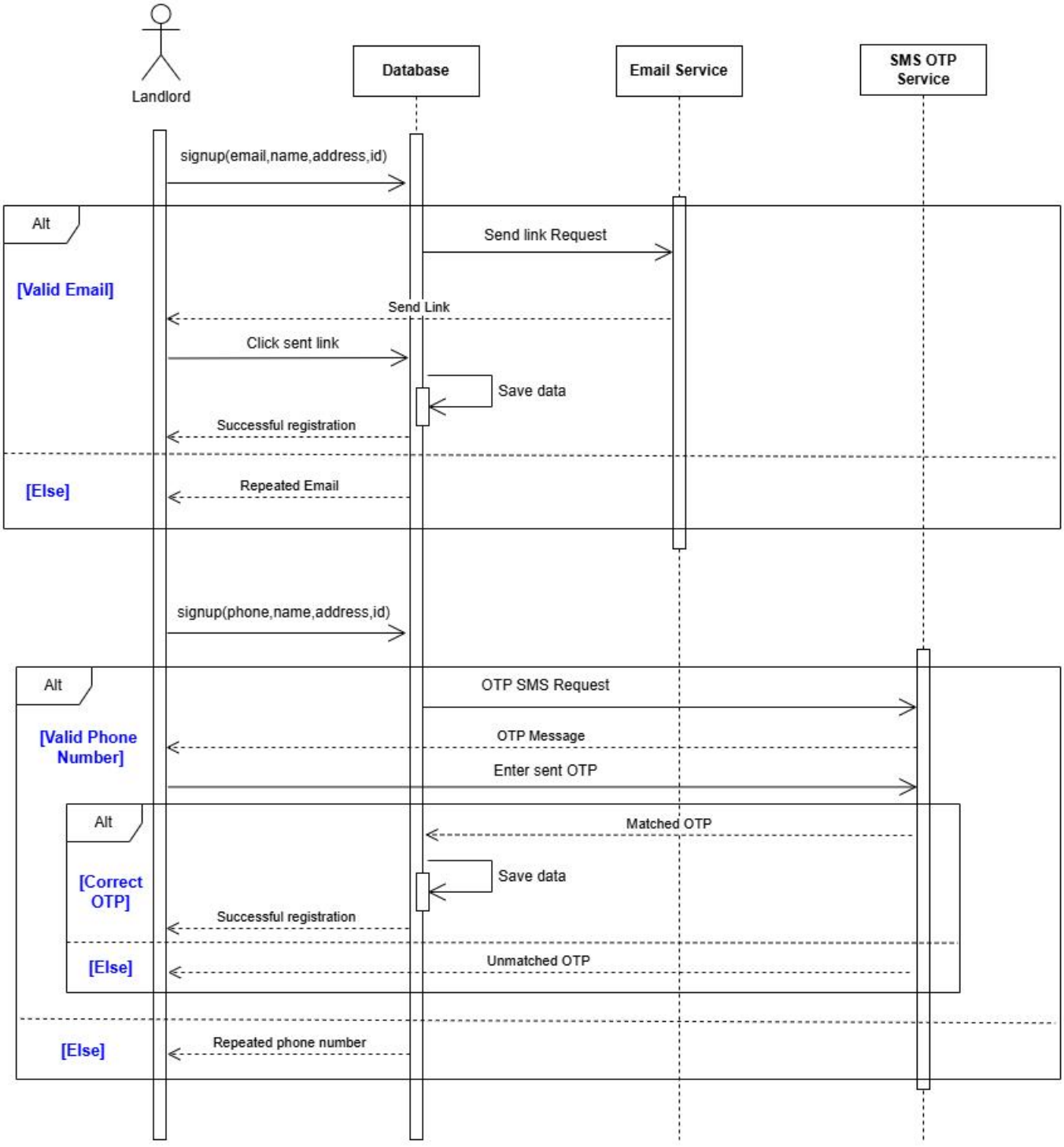


Figure 48: Sequence Diagram - Landlord SignUp

4.6.1.13 **Login**

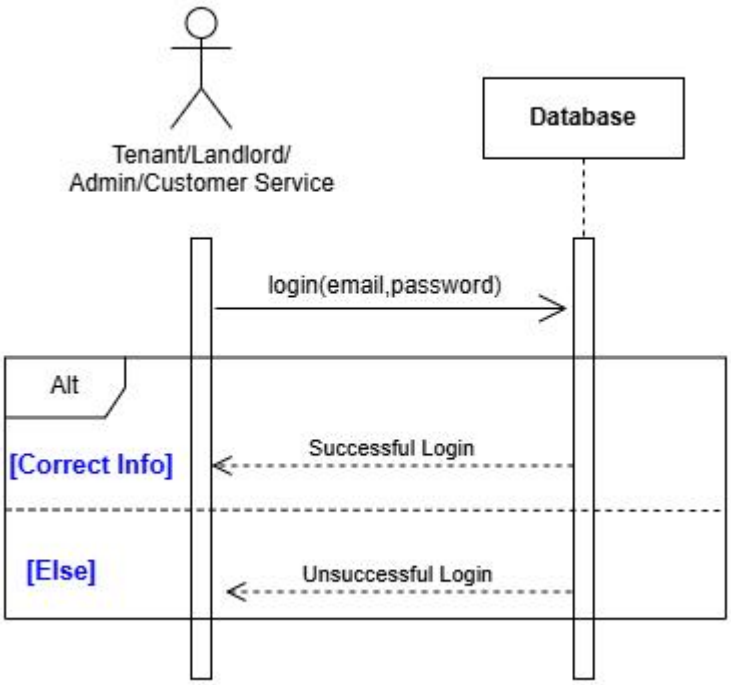


Figure 49: Sequence Diagram - Login

4.6.1.14 **Tenant Edit Profile**

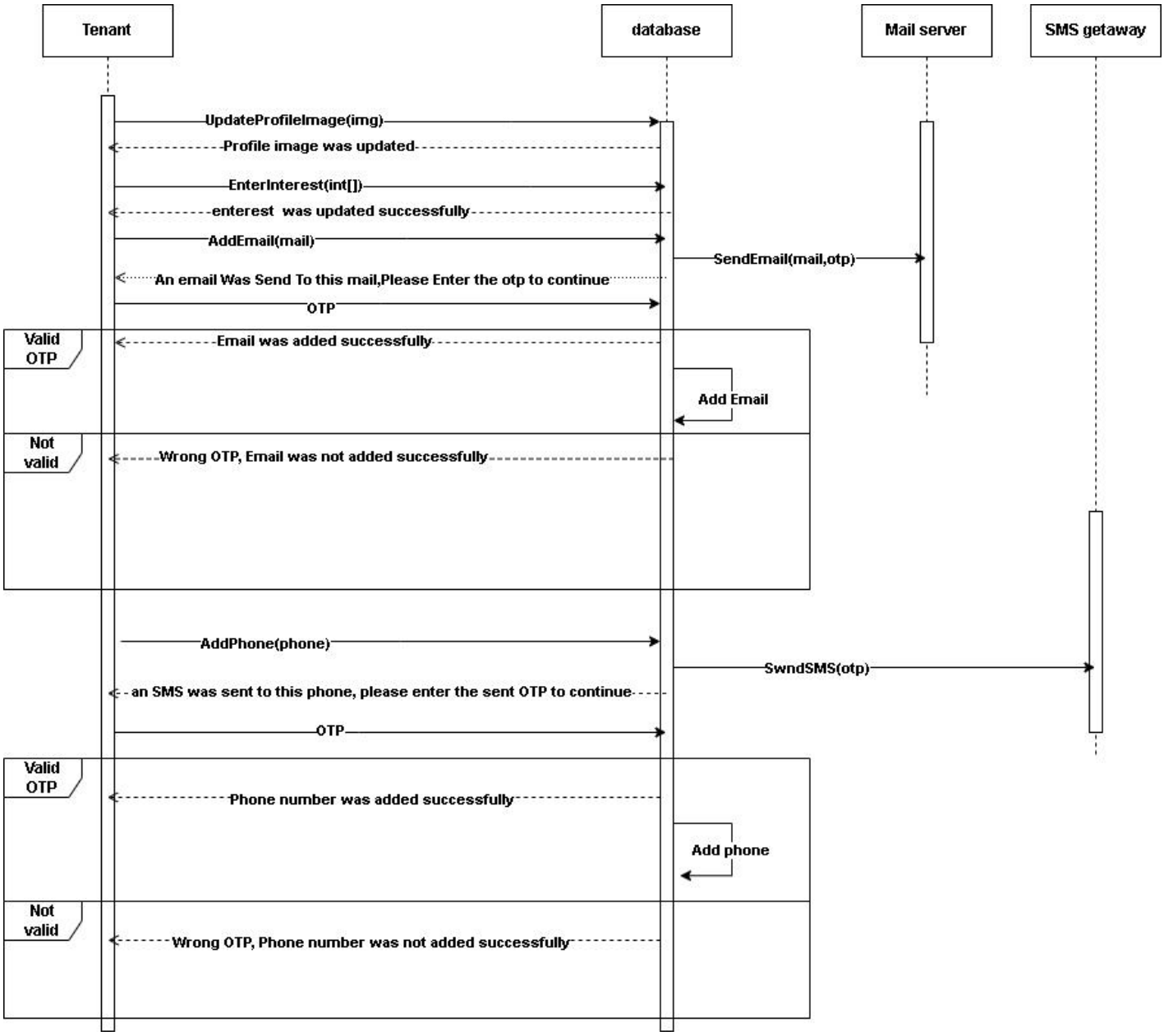


Figure 50: Sequence Diagram - Tenant Edit Profile

4.6.1.15 **Make Complain Inspect /Make Complaints**

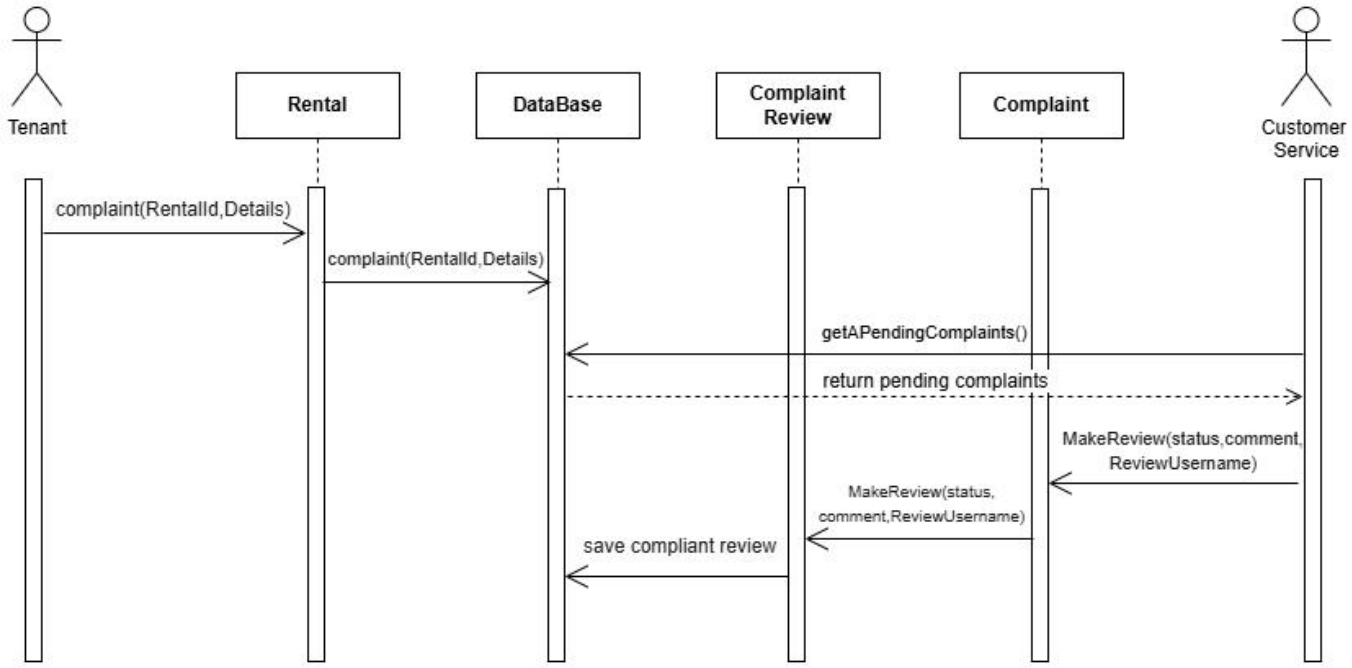


Figure 51: Sequence Diagram - Make Complain Inspect /Make Complaints

4.6.1.16 **Set Rate**

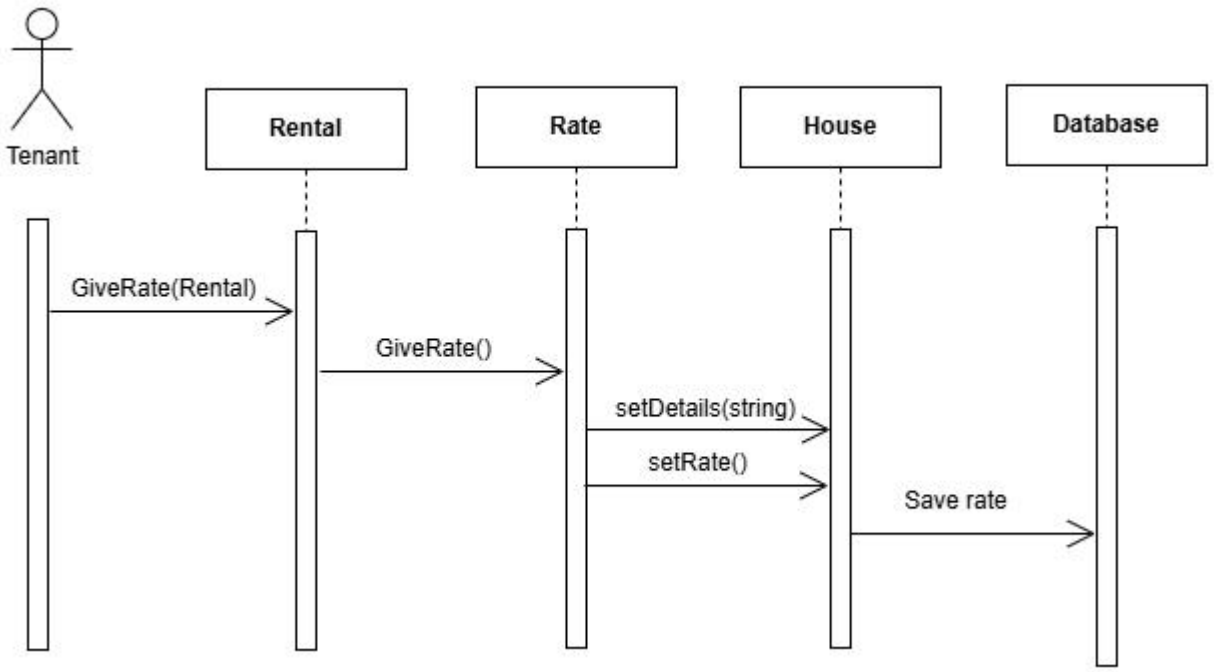


Figure 52: Sequence Diagram - Set Rate

4.6.1.17 **Rental Request / Rental Response**

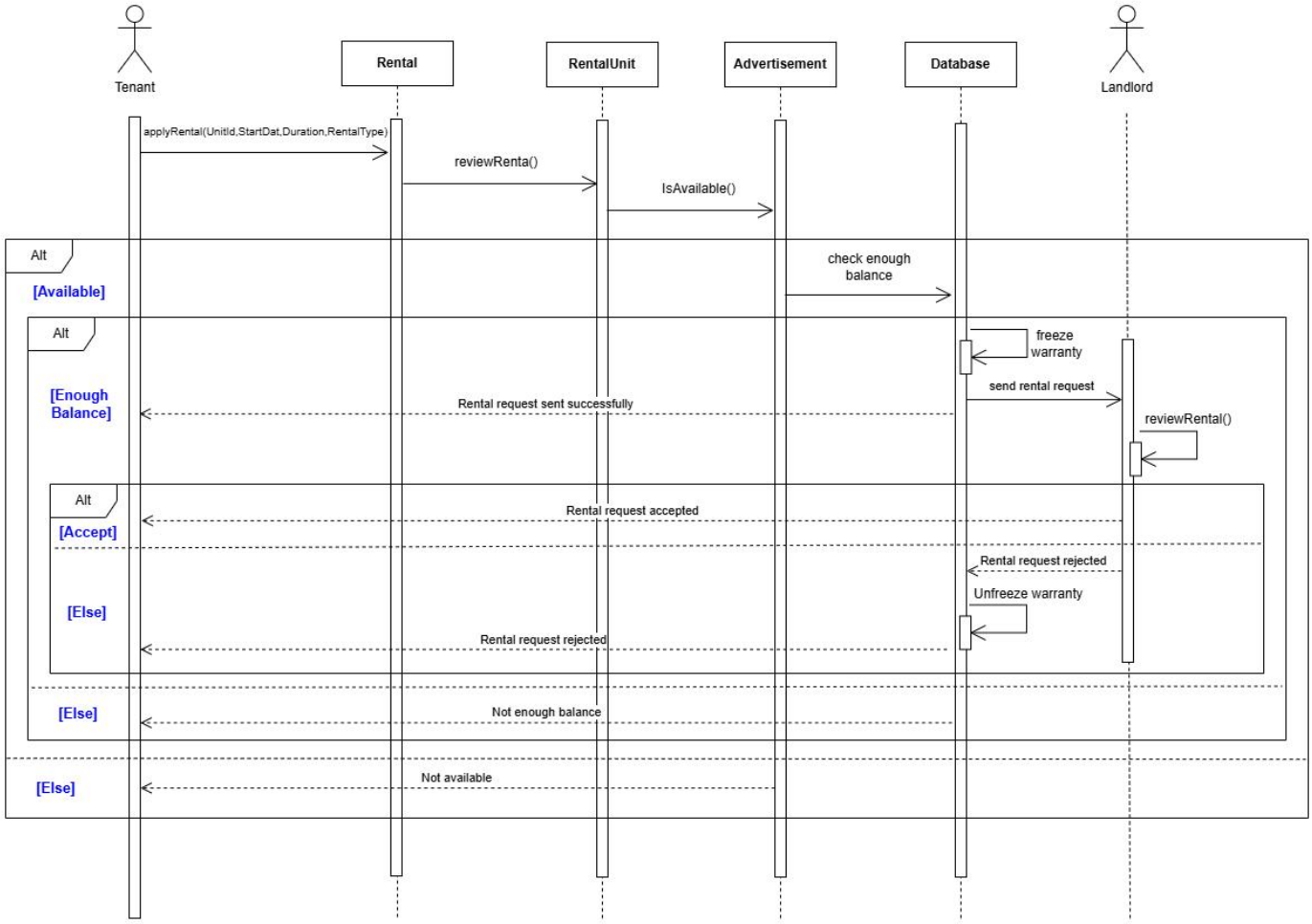


Figure 53: Sequence Diagram - Rental Request / Rental Response

4.6.1.18 **Tenant SignUp With Email**

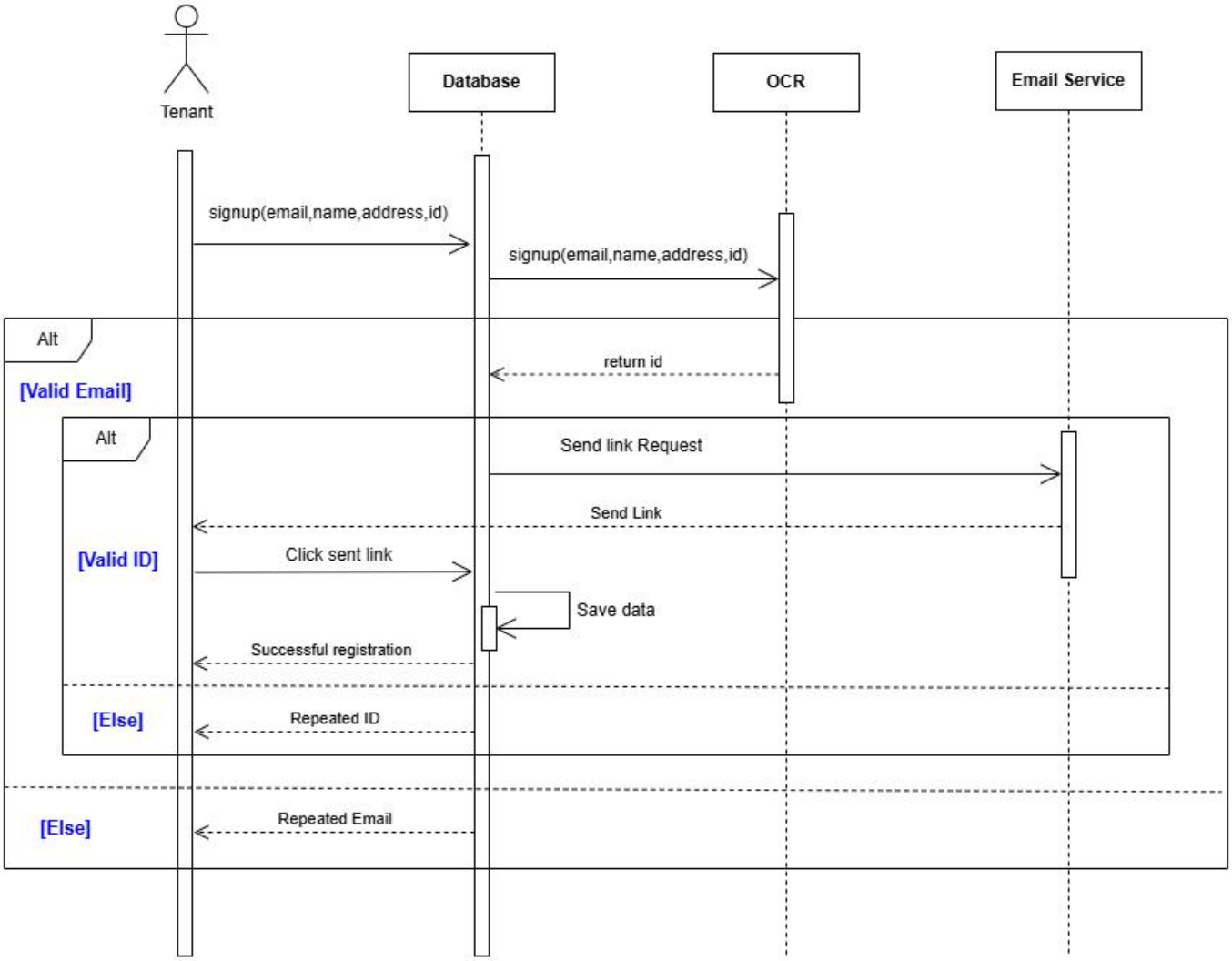


Figure 54: Sequence Diagram - Tenant SignUp With Email

4.6.1.19 **Tenant SignUp With Phone Number**

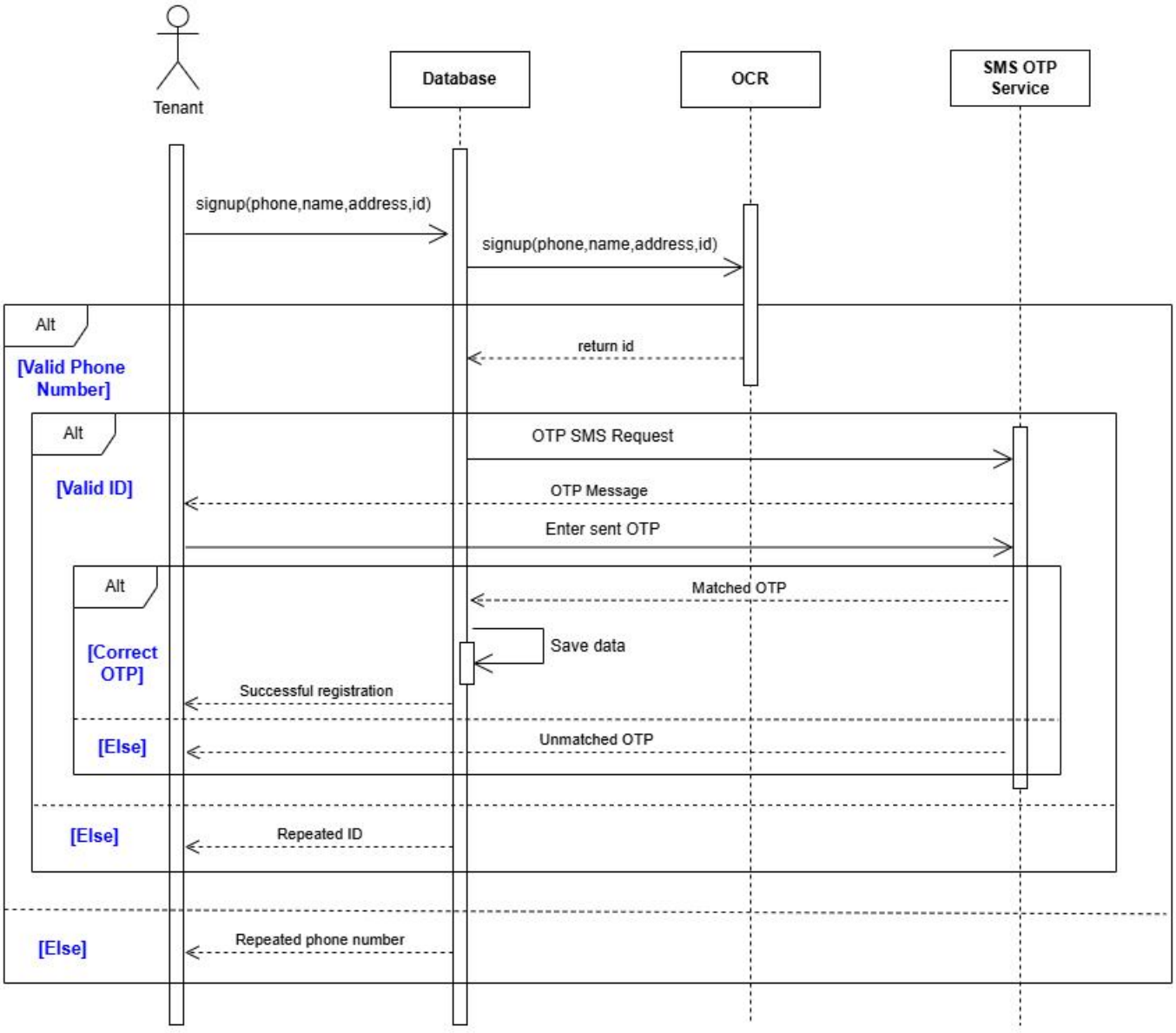


Figure 55: Sequence Diagram - Tenant SignUp With Phone Number

4.6.1.20 **Update Password Using Phone Number**

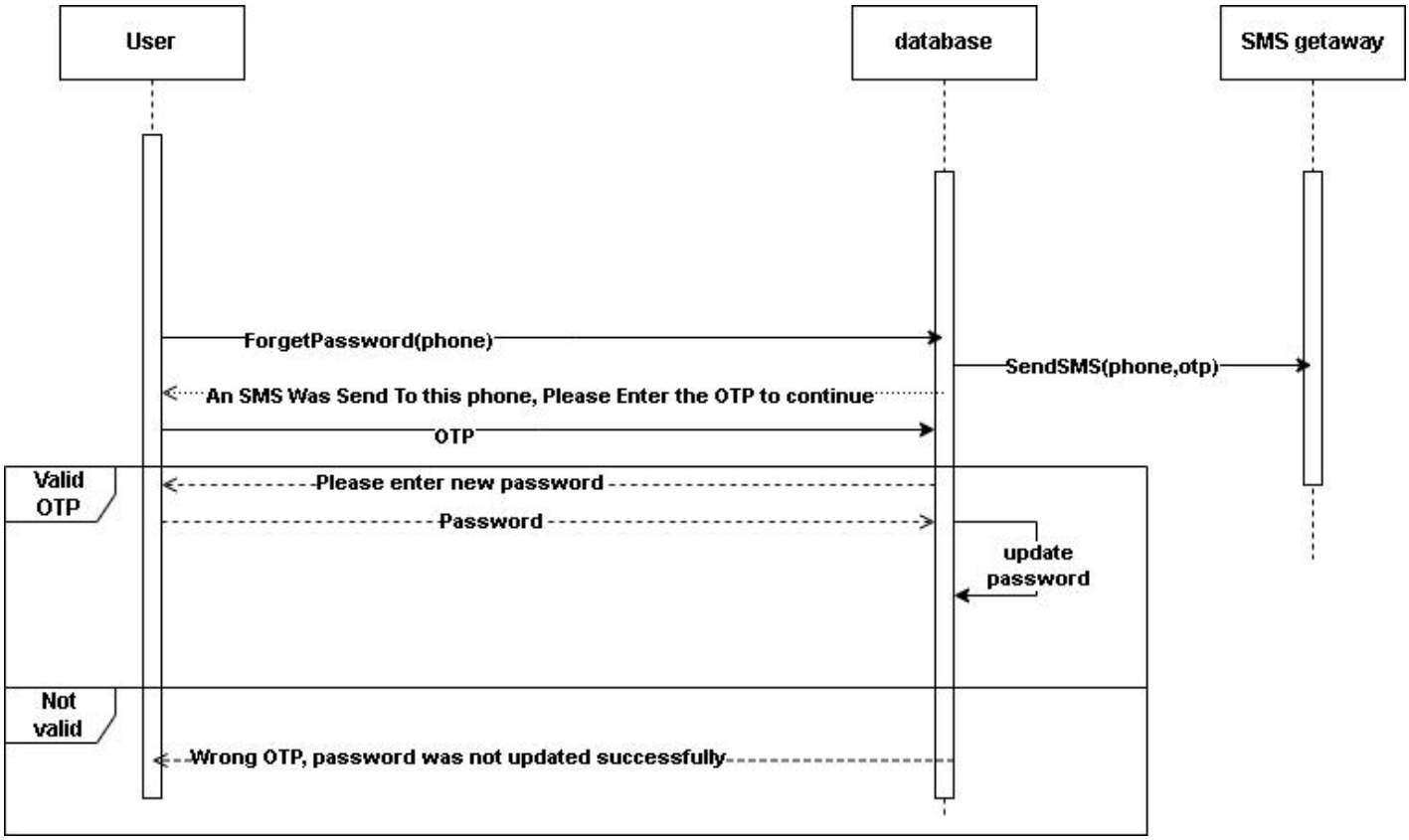


Figure 56: Sequence Diagram - Update Password Using Phone Number

4.6.1.21 Update Password Using Email

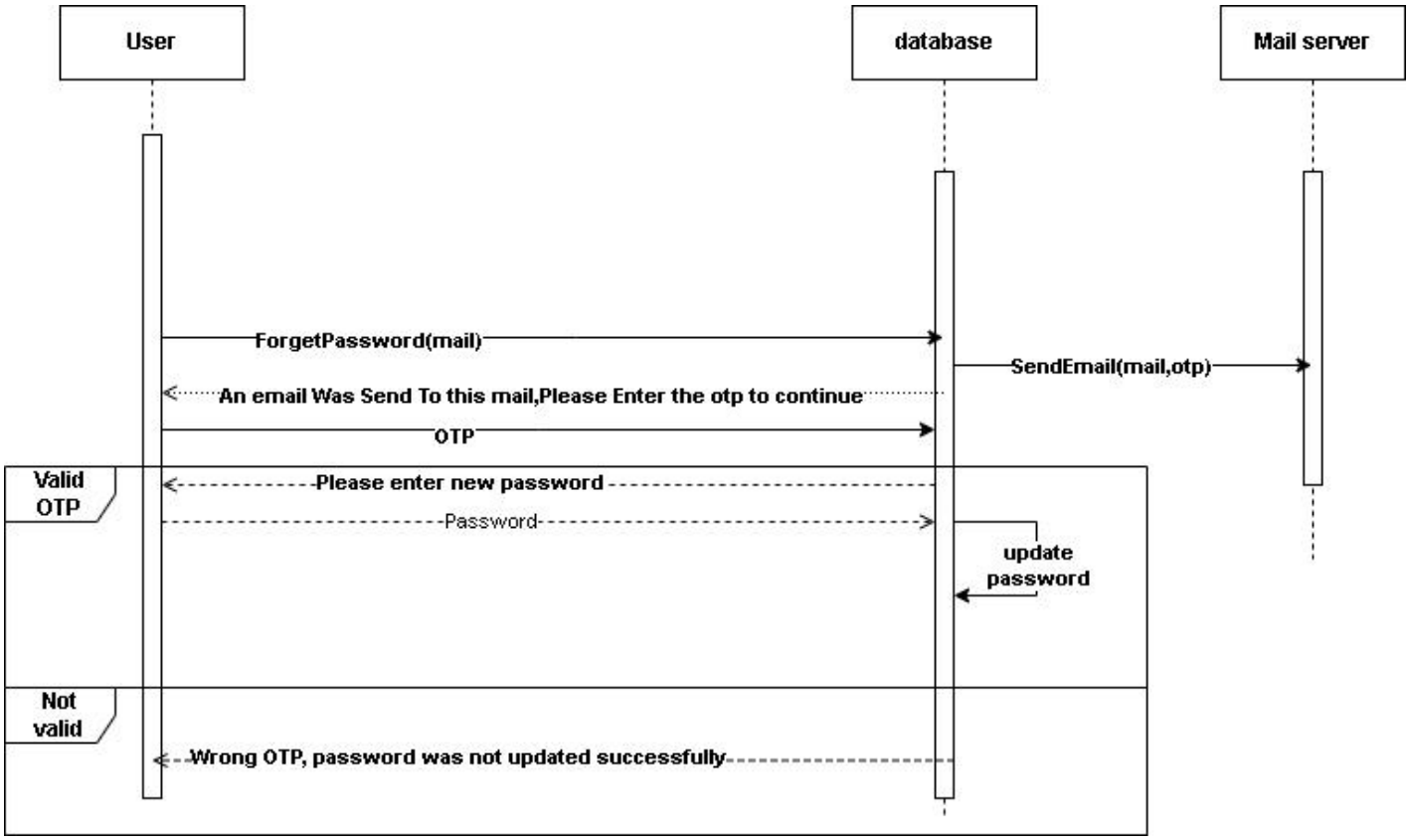


Figure 57: Sequence Diagram - Update Password Using Email

4.7 Design Class

4.7.1 Class Diagram

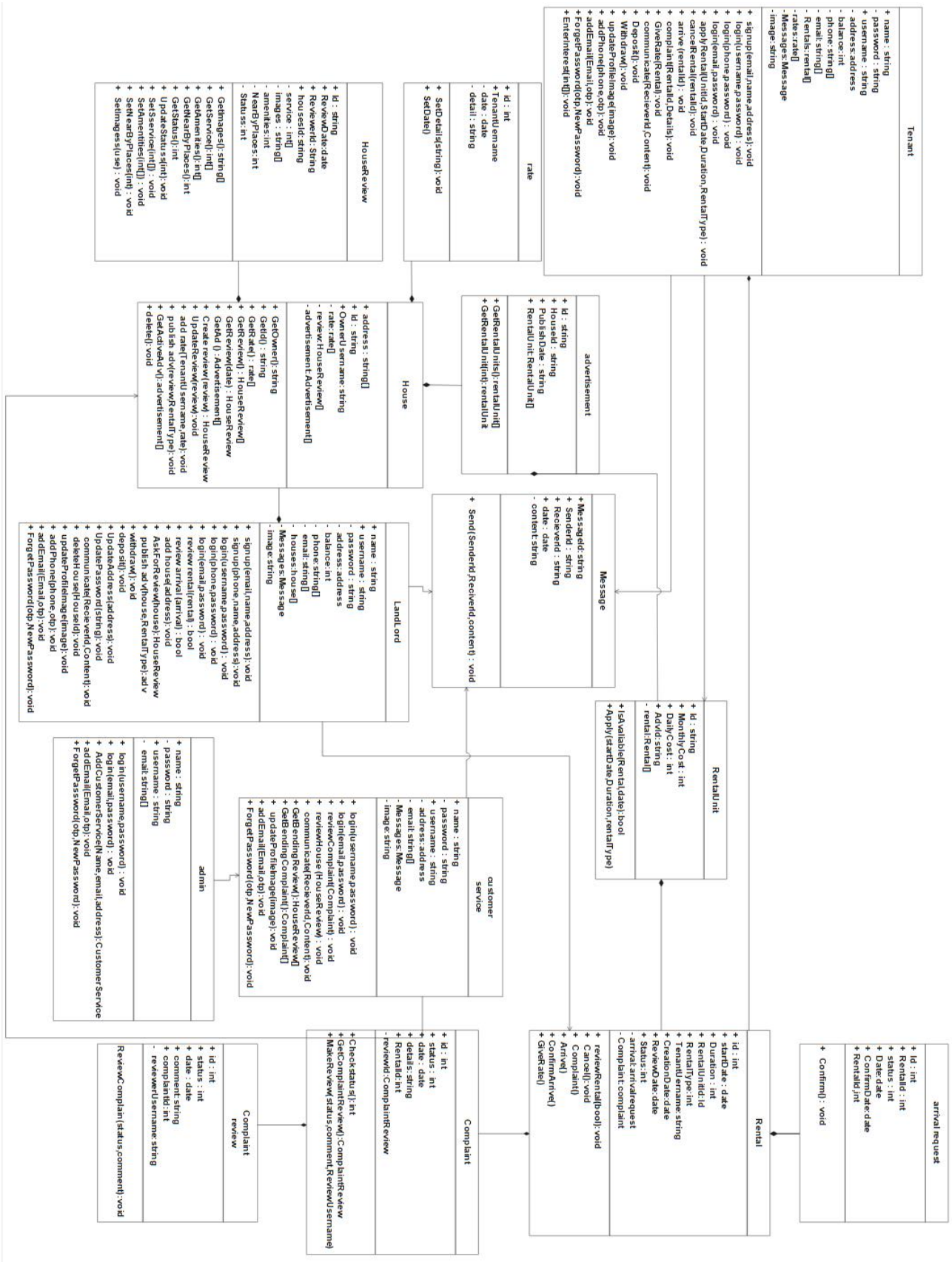


Figure 58: Class Diagram

4.8 ER Diagram

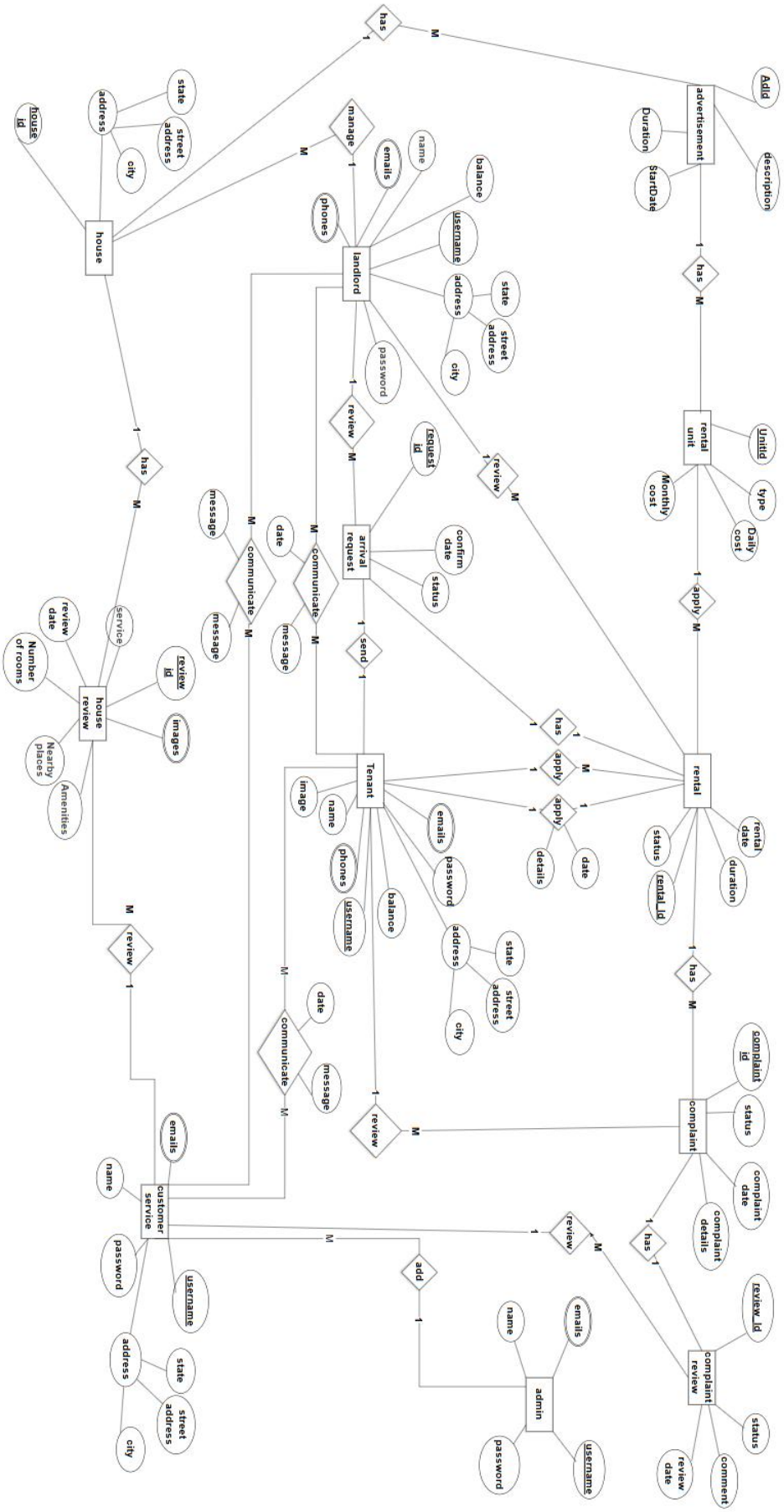


Figure 59: ER Diagram

4.9 Database Schema

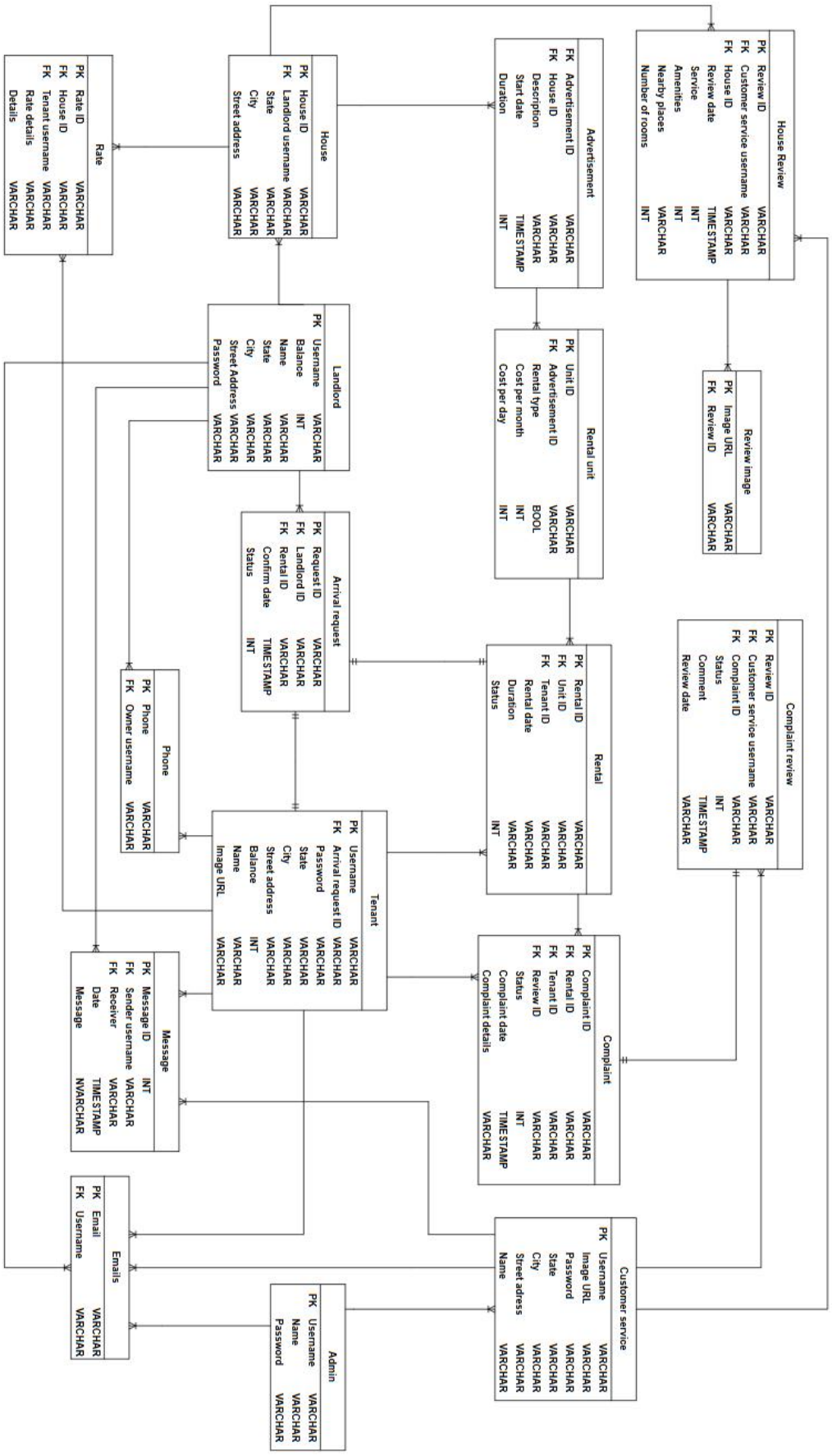


Figure 60: Database Schema

4.10 Design Mockup

4.10.1 Users Authentication

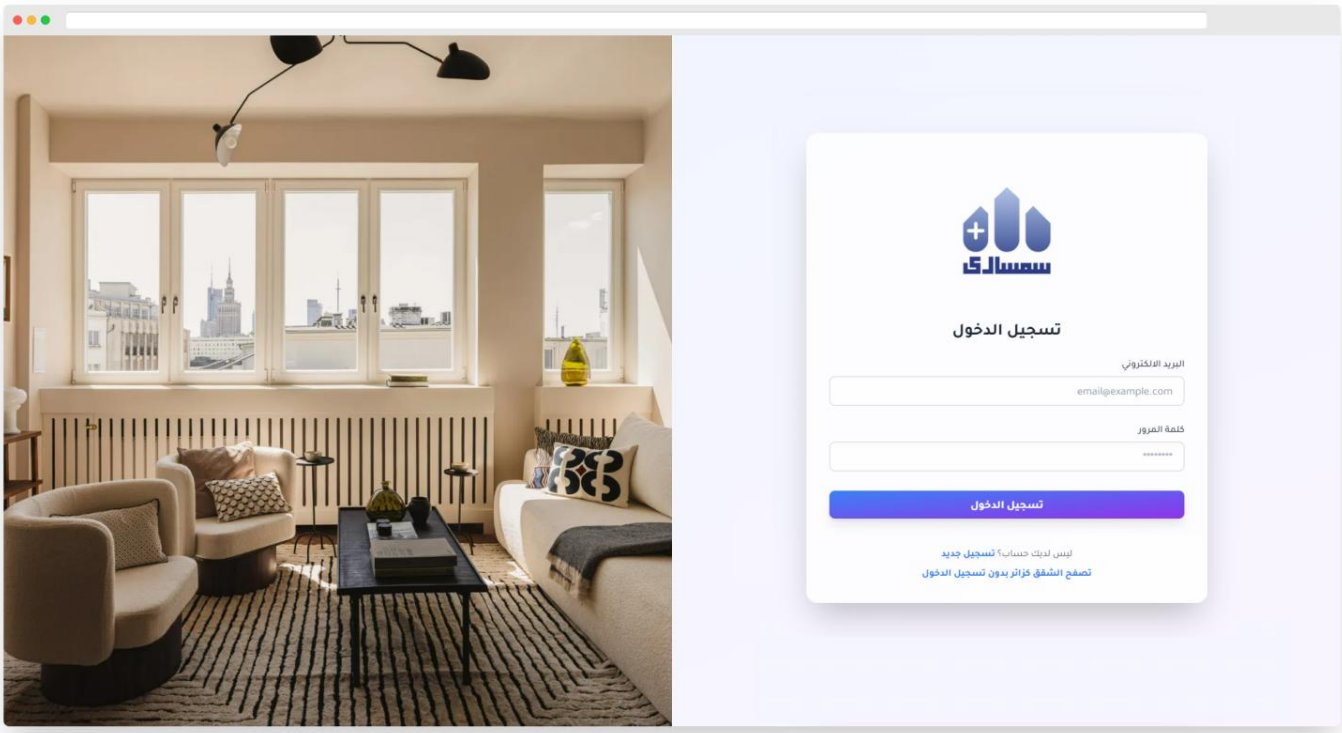


Figure 61: Design Mockup - Website UI Design (Users Login)

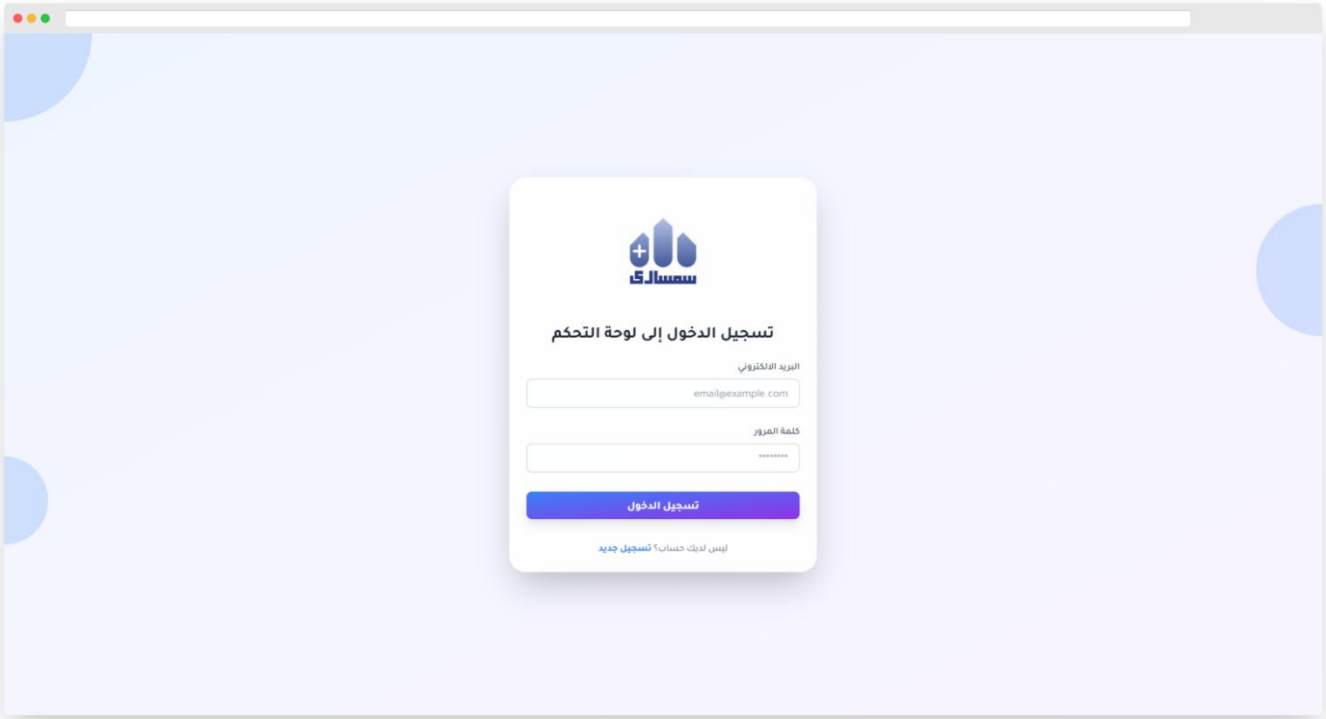


Figure 62: Design Mockup - Website UI Design (Admin Login)

4.10.2 Home pages and details

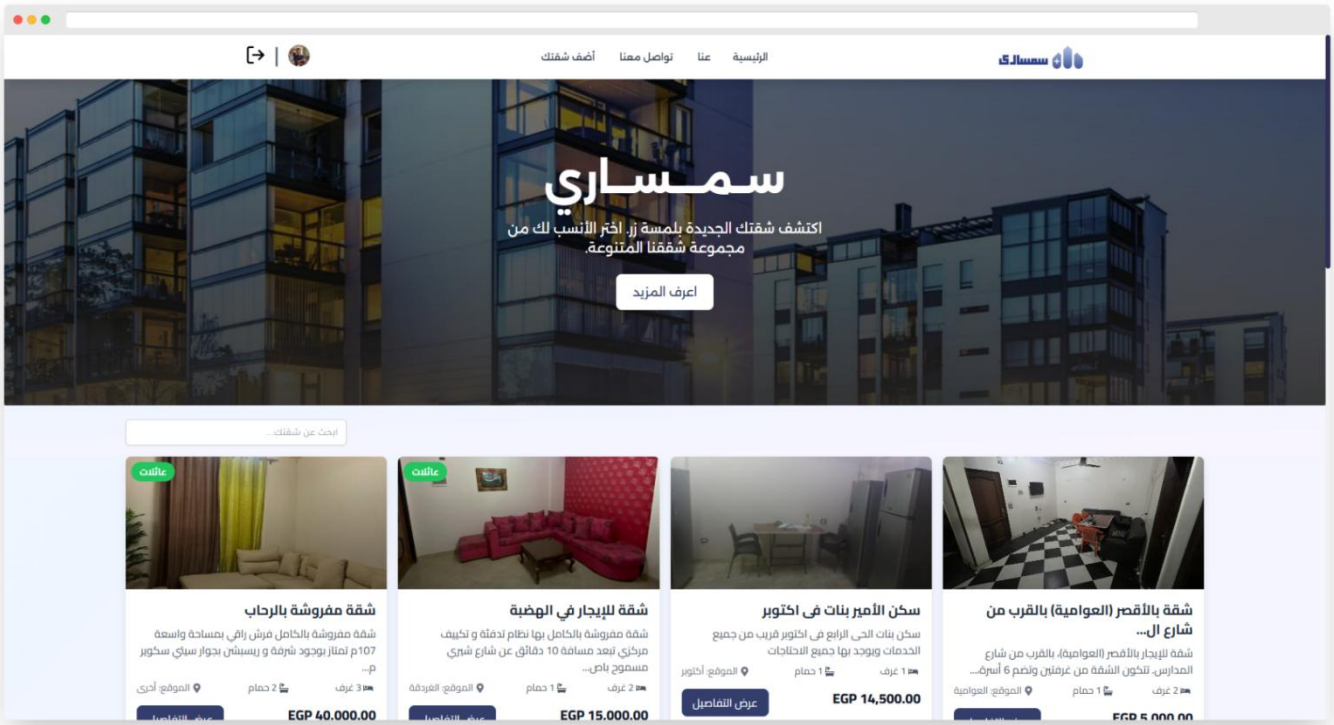


Figure 63: Design Mockup - Website UI Design (Home Page)

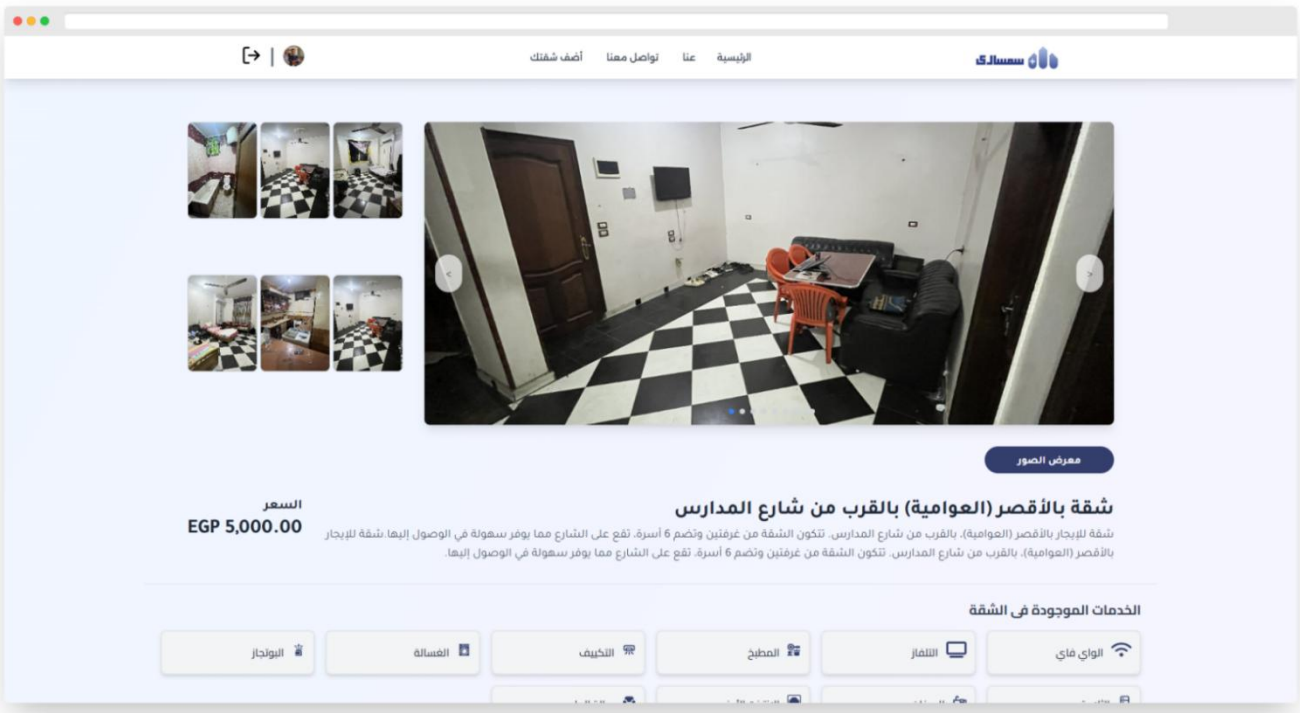


Figure 64: Design Mockup - Website UI Design (Apartment details)

4.10.3 Add Apartment

Figure 65: Design Mockup - Website UI Design (Add Apartment 1)

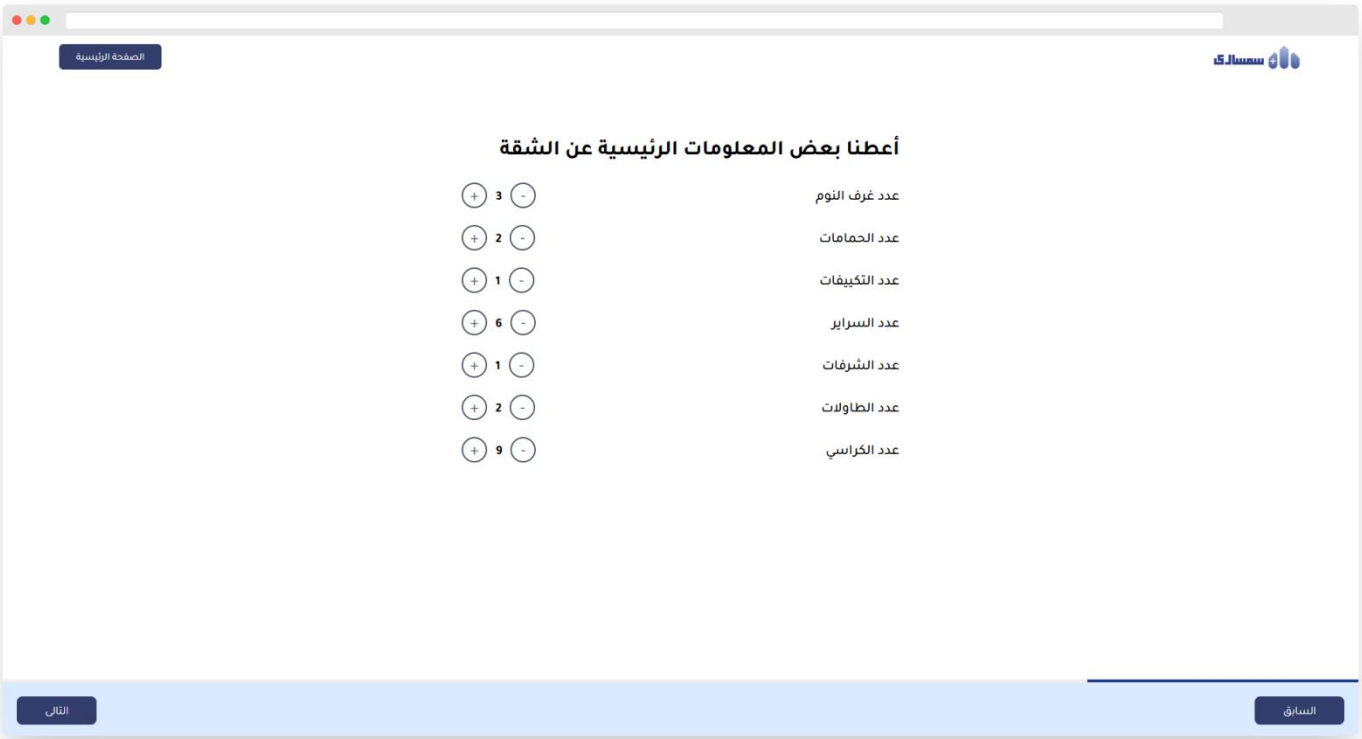


Figure 66: Design Mockup - Website UI Design (Add Apartment 2)

Figure 67: Design Mockup - Website UI Design (Add Apartment 3)

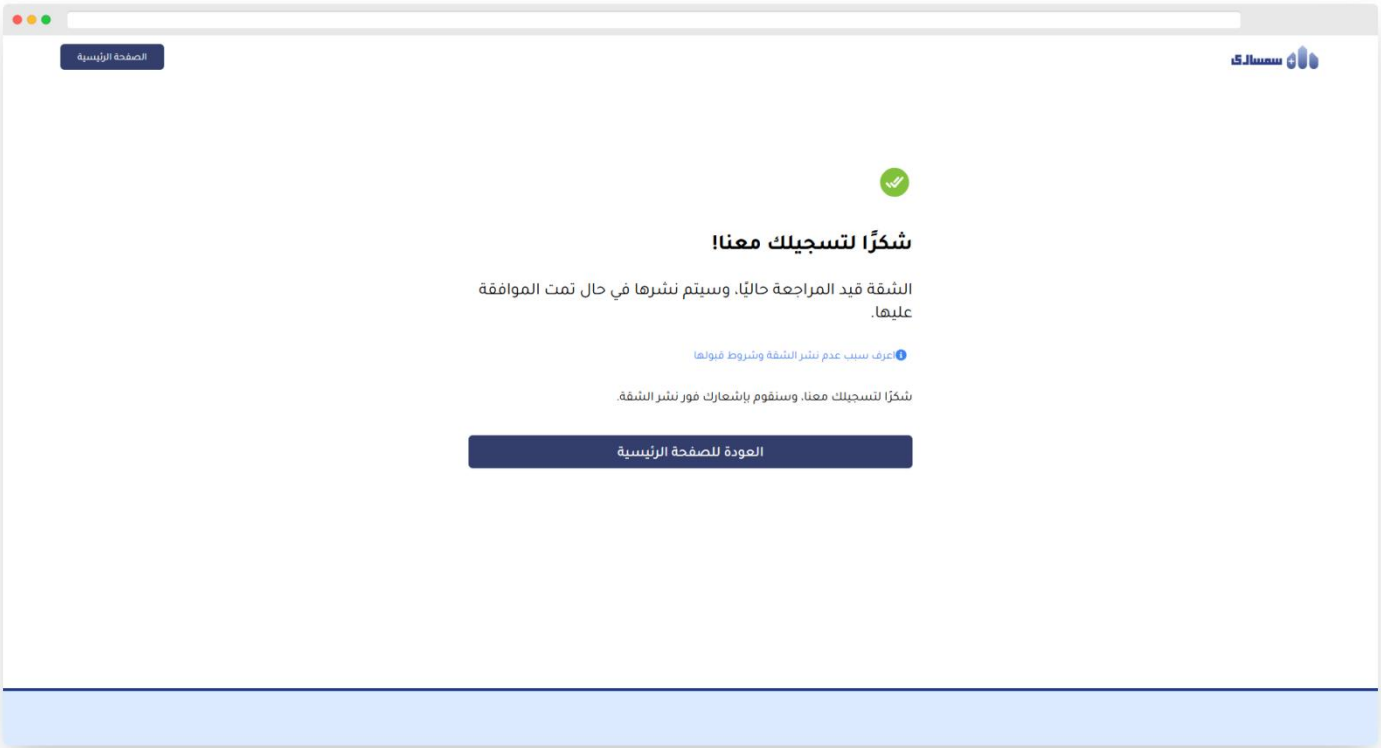


Figure 68: Design Mockup - Website UI Design (Add Apartment 4)

4.10.4 Profile Page

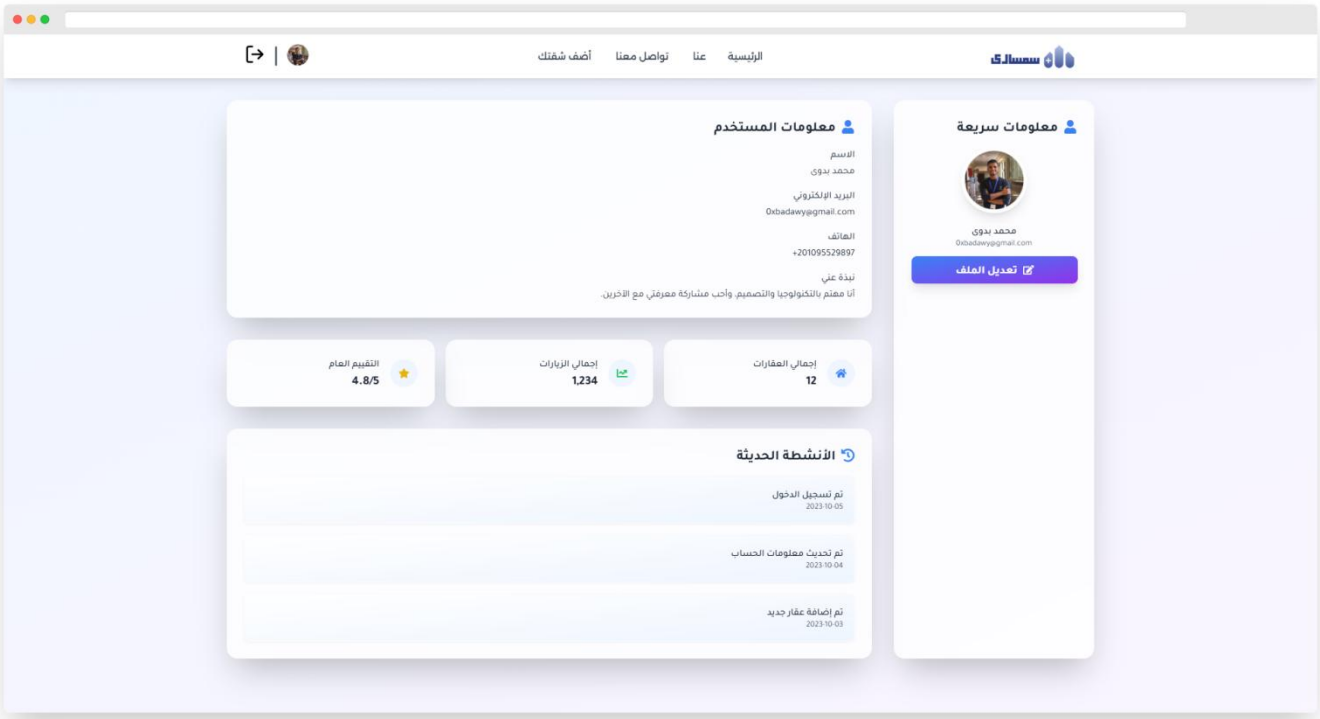


Figure 69: Design Mockup - Website UI Design (Profile Page)

4.10.5 Contact Us page

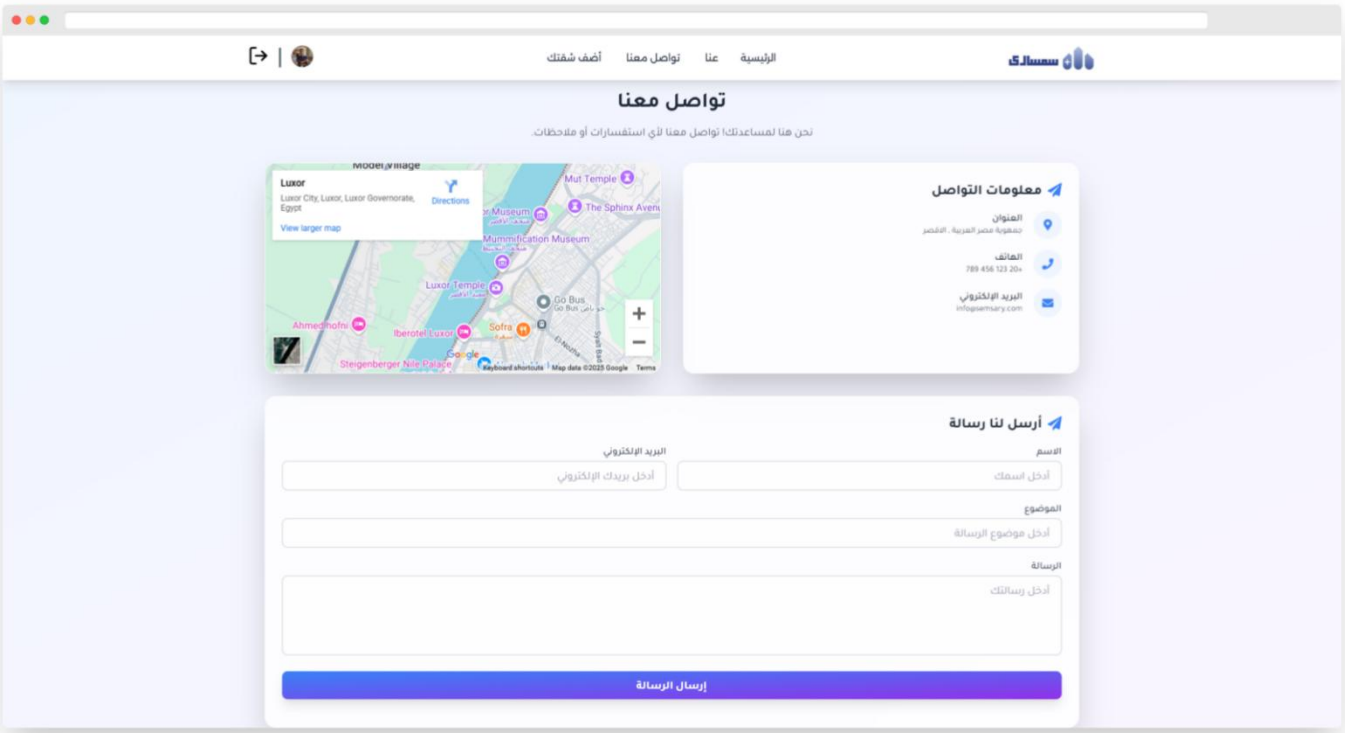


Figure 70: Design Mockup - Website UI Design (Contact Us page)

4.10.6 User reviews and notifications

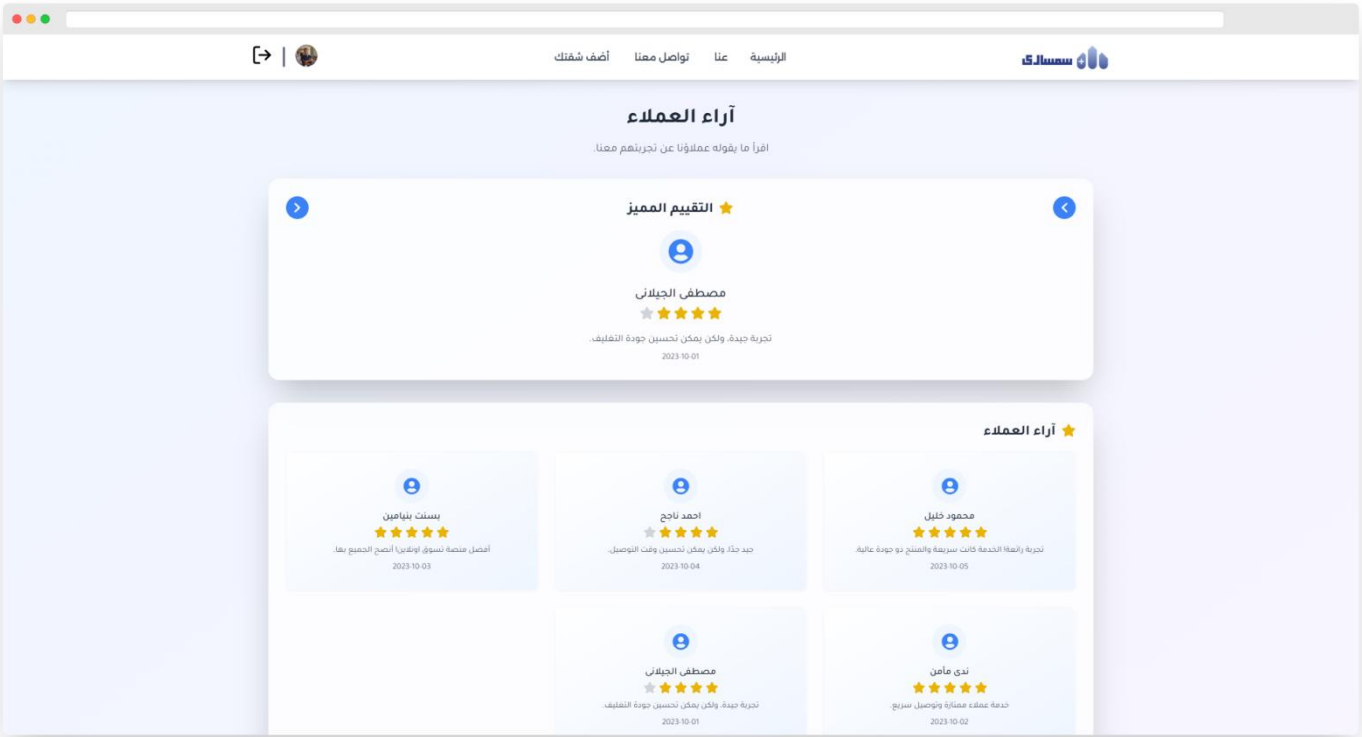


Figure 71: Design Mockup - Website UI Design (User reviews)

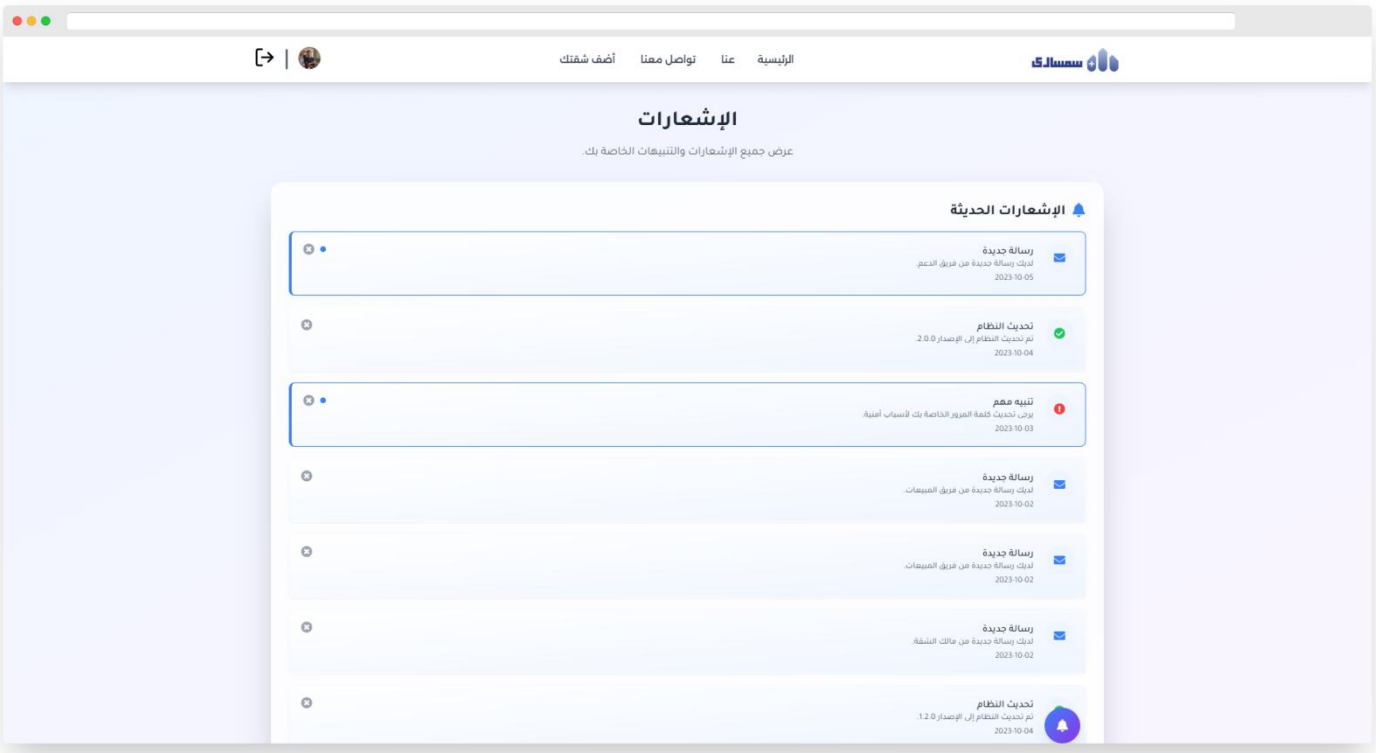


Figure 72: Design Mockup - Website UI Design (Notifications Page)

4.10.7 Payments and conversations

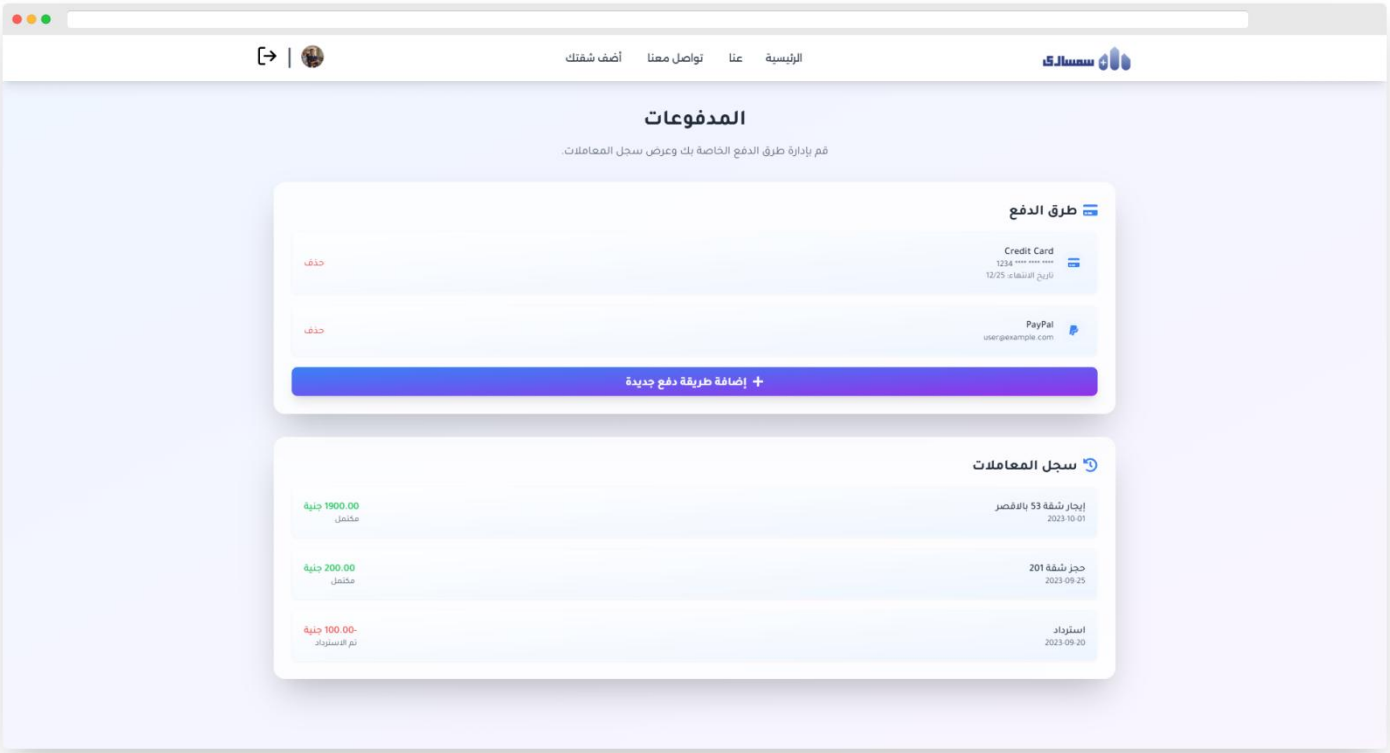


Figure 73: Design Mockup - Website UI Design (Payments)

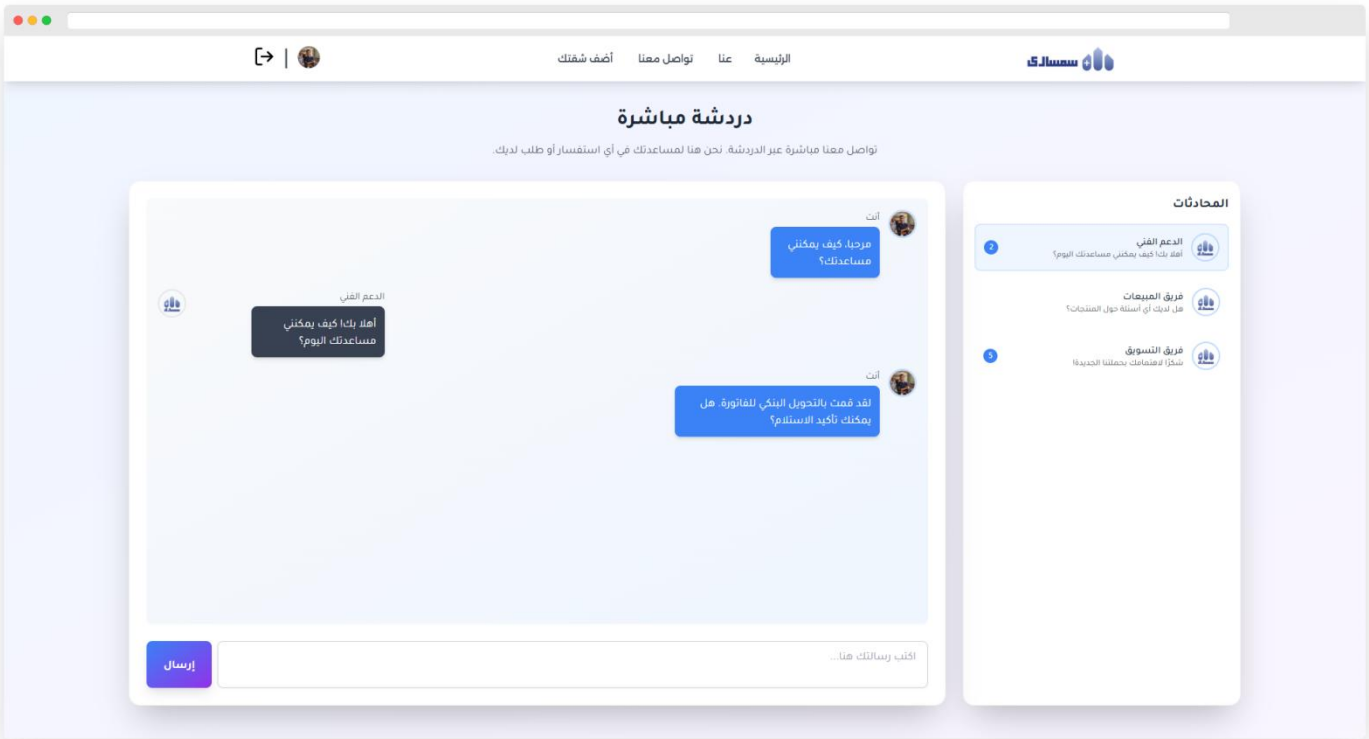


Figure 74: Design Mockup - Website UI Design (Conversations)

Chapter 5

Implementation & Testing

5.1 Programming Languages And Frameworks

5.1.1 Front-End

- Hypertext Markup Language (HTML): used to structure the web pages and their contents.
- Cascading Style Sheets (CSS): used to style and layout web pages.
- Tailwind CSS: utility-first CSS framework that streamlines web development by providing a set of pre-designed utility classes.
- JavaScript: used in adding interactive behavior to web pages. JavaScript allows users to interact with web pages, writing configuration of the tailwind CSS.
- React.js: a framework that employs Web pack to automatically compile React, JSX, and ES6 code while handling CSS file prefixes, and we used it as the main web framework of our project.
- Redux: To manage data flow and application state.

5.1.2 Back-End

- C#: was chosen as the programming language for backend development due to its versatility and strong integration with .NET. Key features.
- NET Framework: ASP.NET Core, a modern and high-performance web framework.
- SQL Server: served as the relational database for managing structured data in the application.
- Firebase Storage: we use it in storing files in a Google Cloud Storage bucket, making them accessible through both Firebase and Google Cloud

5.1.3 Others

- Python: served as the primary programming language for implementing the recommendation system.
- TensorFlow: an open-source machine learning framework, was used for building and training the recommendation model.
- PayMob: is a leading payment gateway in Egypt, designed to facilitate secure online transactions for businesses and consumers. It provides an API for processing payments, enabling businesses to accept payments via credit/debit cards, mobile wallets, and other local payment methods.
- Git: a distributed version control system that tracks versions of files, we use GitHub for version control for our project.

5.2 Algorithm

5.2.1 Signup Process

a) The user selects "SignUp" option on the platform.

b) The user enters their full name, email or phone number, password, and uploads an image which proofs his identity.

c) The user receives an OTP or SMS code via their email address or phone number.

d) If the registration is confirmed, the user is redirected to the main page or a page where he can enter additional information about his interests, but he can skip it (optional).

e) The user can login in the platform using the “Login” button.

5.2.2 Apply For A Rent Process

a) The tenant logs into the platform using the “Login” button.

b) The tenant search for a house and selects the "Apply for Rent" option on the advertisement.

c) The system checks if the tenant has sufficient funds for the guarantee amount.

d) If funds are sufficient, the platform forwards the rental request to the landlord and updates the status. The warranty amount is withdrawn from the tenant’s balance.

e) If funds are insufficient, the platform prompts the tenant to deposit the required amount.

f) Once the request is successfully submitted, the landlord is notified of the rental request.

5.2.3 Make House Inspection Process

a) The landlord submits a specification request through the platform.

b) The platform verifies the validity of the landlord’s property request.

c) If the request is valid, the customer service team schedules and performs a property inspection.

d) The customer service defines property specifications such as number of rooms, amenities, and uploads relevant photos.

e) If more documentation is needed, customer service contacts the landlord for additional proof.

f) Once all details are verified, the landlord can publish the advertisement on the platform.

5.2.4 Publish Advertisement Process

a) The landlord logs into the platform using the “Login” button.

b) The landlord navigates to their list of house advertisements.

c) The landlord selects the "Publish House" option for the desired house.

d) The landlord chooses a suitable advertisement plan from the available options.

e) The cost of the selected advertisement plan is automatically withdrawn from the landlord’s balance.

F) If the landlord does not have sufficient balance, the system notifies them and prompts for a deposit.

G) Once payment is successful, the house advertisement is published and displayed on the platform.

5.2.5 Communication Process

a) The tenant logs into the platform using the “Login” button.

b) The tenant selects the "Communicate" option.

c) The tenant writes and sends a message to the landlord through the platform.

d) The platform notifies the tenant when the landlord replies to the message.

e) If communication fails, the tenant is notified and prompted to try again.

f) Once successful, both tenant and landlord are connected for further communication.

5.3 Testing Scenarios

5.3.1 Front-End Testing

ID	DESCRIPTION	PRE-REQUIS	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
FTC-01	Verify that the user can access the signup page via the LOGIN Bottom	The website is up and running.	1-Open the platform. 2- Click on the "LOGIN" button 3- User Enter information.	None	The user should be redirected to the signup page	The user is redirected to the signup page	pass
FTC-02	Verify that the user can submit their full name, email, phone number, password, and upload an identity image.	Access to the signup page	1-Fill in the full name, email, phone number, and password. 2-Upload an identity image. 3-Click on the "Submit" button	- Full Name: John Doe Email: temp@gmail.com -Phone Number: 123-456-7890 -password: ****** - identity: image.png	The request should be sent to the administrator for review, and a confirmation message should be displayed	The request sent to the administrator for review, and a confirmation message displayed	pass
FTC-03	Verify that the tenant can search for houses	Tenant is logged in	1. Go to search bar 2. Apply filters (e.g. location, price) 3. Click search	- Location: Cairo - Price range: 2000–8000 EGP	Relevant listings should be displayed	Listings returned as per filters	Pass
FTC-04	Verify tenant can apply for a rental	Tenant is logged in	1. Select house 2. Click 'Apply for a rent' 3. Confirm rental.	- House ID: APT101 - Wallet: 5000 EGP	Rental request submitted to landlord	Rental request sent successfully	Pass

FTC-05	Verify tenant can cancel rental	Tenant has a pending rental	1. Go to advertisement 2. Click 'Cancel' 3. Confirm cancellation within 4 days	- Rental ID: BKG1001 - Days since rental: 2	Rental canceled and refund initiated	Cancellation successful and funds refunded	Pass
FTC-06	Verify tenant can confirm arrival	Tenant has approved rental	1. Go to advertisement 2. Click 'Confirm Arrival'	- Rental ID: BKG1001	Arrival confirmed and warranty money activated	Confirmation recorded	Pass
FTC-07	Verify tenant can submit complaints	Tenant is logged in	1. Go to ‘main page' 2. Click 'make complain inspection' 3. Describe issue 4. Submit	- rental ID: BKG1002 - Complaint: 'Specs do not match'	Complaint sent to customer service	Complaint recorded and under review	Pass
FTC-08	Verify tenant can rate the house	Tenant has complete d stay	1. Go to rentals history 2. Click 'Rate house' 3. Add rating and comments	- Stars: 4 - Comment: 'Nice and clean'	Rating saved and shown under house listing	Rating submitted successfully	Pass
FTC-09	Verify landlord can publish house	Landlord is logged in	1. Go to 'My Reviews' 2. Select house 3. Choose a plan 4. Click 'Publish'	- House ID: APT200 - Balance: 3000 EGP	House published and visible to tenants	House published successfully	Pass
FTC-10	Verify landlord can accept/rejec t rental requests	Landlord has active listing	1. Open rental request 2. Click 'Accept' or 'Reject'	- Request ID: REQ1001	Status updated and tenant notified	Request accepted and tenant notified	Pass
FTC-11	Verify landlord can confirm tenant arrival	Rental is active	1. Go to accepted rentals 2. Click 'Confirm Arrival'	- Request ID: REQ1001	Arrival confirmed and tenant can recover warranty money	Arrival confirmation successful	Pass

FTC-12	Verify customer service can inspect and add house info	Customer service logged in	1. Open new inspection request 2. Visit house 3. Fill specs 4. Submit	- Owner ID: LND123 - Address: 22 El-Nasr St.	House details submitted and sent to landlord	House info successfully added to platform	Pass
FTC-13	Verify admin can add new customer service agent	Admin logged in	1. Go to admin dashboard 2. Add agent details 3. Click 'Submit' 4. Send credentials via email	- Name: Ahmed - Email: ahmed@semsary.com -Password: ******	Agent created and email sent with login credentials	Agent added and received credentials	Pass

Table 32: Testing Scenarios - Front-End Testing

5.3.1 Back-End Testing

ID	DESCRIPTION	PRE-REQUISITES	TEST STEPS	TEST DATA	EXPECTED RESULT	ACTUAL RESULT	STATUS
BTC-01	Tenant account registration with email verification	Tenant not registered	Submit valid email and password for registration	email: test@domain.com	Account is created and verification email sent	Account created, email sent	Pass
BTC-02	Tenant login with valid credentials	Registered tenant	Enter email and password	email/password	Login successful and main page displayed	Login successful	Pass
BTC-03	Password reset flow for tenant	Tenant account exists	Request password reset via email	email: tenant@domain.com	Reset link sent to email	Reset link received	Pass
BTC-04	Search with filters: location + price	Database has listings	Apply filters on homepage	location: Giza, price: <5000	Filtered results displayed	Results match filters	Pass
BTC-05	Rental with insufficient wallet balance	Wallet balance = 0	Click 'Apply for a rent' on house	User balance = 0	Rental fails with error	Rental rejected	Pass

BTC-06	Wallet deposit via card	Tenant logged in	Initiate deposit process	card: valid	Balance updated	Balance increased	Pass
BTC-07	Tenant and landlord chat after rental	Rental confirmed	Send message via chat	Hello	Message delivered	Chat message visible	Pass
BTC-08	Cancel rent request within allowed time	Rental exists <4 days	Click cancel on rental	Rental ID: 123	Money returned to tenant	Balance refunded	Pass
BTC-09	Complaint about mismatched house spec	Tenant renting and arrived	Submit complaint	Tenant ID, House ID	Complaint submitted to support	CS review visible	Pass
BTC-10	Tenant confirms arrival	Rental accepted	Click 'Confirm Arrival'	Rental ID	Warranty money returned	Money returned	Pass
BTC-11	Submit rating and review	Tenant completed stay	Submit rating form	Rate: 4	Review posted	Review visible	Pass
BTC-12	Landlord registration and verification	No existing account	Submit registration form	email, password, ID image	Account created & verification sent	Verified landlord	Pass
BTC-13	Landlord login	Account created	Enter credentials	email/pass	Login successful	Landlord dashboard shown	Pass
BTC-14	Landlord adds house listing	Verified account	Submit new listing	Listing form	Listing appears in dashboard	Published after approval	Pass
BTC-15	Landlord publishes house	House reviewed, balance enough	Click publish	House ID, balance	House is live	Listing active	Pass
BTC-16	Landlord updates ad	Existing ad	Submit updated info	New price, status	Changes saved	Updated details visible	Pass
BTC-17	Landlord deletes house	House exists	Click delete	House ID	Ad removed	No longer listed	Pass

BTC-18	Landlord accepts rental request	Request pending	Click accept	Tenant request ID	Request confirmed	Rental confirmed	Pass
BTC-19	Administrator login	Administrator account exists	Enter credentials	admin@sys.com / pass	Administrator panel loaded	Dashboard shown	Pass
BTC-20	Administrator creates support agent	Administrator logged in	Submit agent form	Agent name, email, password	Agent added	Agent listed	Pass
BTC-21	Customer Service reviews complaint	Complaint submitted	Open complaint panel	Complaint ID	Landlord banned if true	Tenant refunded	Pass
BTC-22	Tenant refund after cancel	Cancel within 4 days	Click cancel	Rental ID	Refund issued	Balance credited	Pass
BTC-23	False complaint submitted	Invalid claim	Submit complaint	Complaint ID	Landlord not banned	Money stays with system	Pass
BTC-24	Unauthorized access to rental	JWT token expired	Send API request	token: expired	Access denied	401 Unauthorized	Pass
BTC-25	JWT authentication for landlord	Valid token	Access /landlord route	Authorization: Bearer token	Authorized access	Landlord panel shown	Pass
BTC-26	SQL Injection prevention	Login form active	Inject SQL via field	' OR 1=1 --	Request rejected	Safe from injection	Pass
BTC-27	XSS attack detection	Input form enabled	Submit script	<script>alert(1)</script>	Script blocked	No popup shown	Pass
BTC-28	Backup restore test	Database backup exists	Run restore	Backup file	System state restored	Restored successfully	Pass
BTC-29	Transaction rollback on rental failure	Tenant starts renting	Simulate failure	Missing balance	No partial update	Rental not saved	Pass

BTC-30	Rental API GET test	Auth token valid	GET /api/rental	token	Returns rental list	List received	Pass
BTC-31	House filter API	Listings in DB	Call filter endpoint	price < 3000	Filtered list returned	Matched entries shown	Pass
BTC-32	Twilio OTP SMS test	Valid phone number	Send OTP	Phone: 010000	OTP delivered	SMS received	Pass
BTC-33	PayMob transaction success	Card valid	Make payment	Card: valid	Payment processed	Status = success	Pass
BTC-34	Firebase image upload	Logged-in user	Upload image	Apartment image	URL returned	Image visible	Pass
BTC-35	Review flagged by user	Inappropriate content	Flag review	Review ID	Review hidden pending review	Flag acknowledged	Pass

Table 33: Testing Scenarios - Back-End Testing

5.4 Back-End Implementation

POST	https://api.semsary.site/api/Auth/Login	
	Request body	**Response body**

	{
	"email":"badawy@gmail.com ",
	"password":"Password 123",
	"deviceToken": "string"
	}
		{
		"token":
		"eyJhbGciOiJIUzI1XNhcnkiLCJhdWQiOiJTZW1zYXJ5In0.XVGuwCe9oKbMzt3KTKXBykA-nuUk",
		"askForNotificationPermission": true
		}

	Authenticates a user by accepting login credentials in the request body.	

POST	https://api.semsary.site/api/Auth/verifyEmail	
	Request body	**Response body**

	{
	"email": " "badawy@gmail.com",
	"otp": "038286"
	}
		{
		Email verified successfully.
		}

	Verifies a user's email address by accepting a verification code	

POST	https://api.semsary.site/api/Auth/Tenant/register	
	Request body	**Response body**

	{
	"password": "Password 123",
	"firstname": "my_first_name",
	"lastname": "my_last_name",
	"email": "badawy@gmail.com"
	}
		{
		Registration successful. Please check your email for verification.
		}

	Registers a new Tenant account by accepting user details	

POST	https://api.semsary.site/api/Auth/AllowNotifications	
	Request body	**Response body**

	{
	"deviceToken": "string"
	}
		Sending notifications is allowed successfully.

	Enables push or in-app notifications for a user by accepting relevant notification settings or device identifiers in the request body.	

POST	https://api.semsary.site/api/Auth/Landlord/register	
	Request body	**Response body**

	<pre>{
	  "password": "Password 123",
	  "firstname": "my_first_name",
	  "lastname": "my_last_name",
	  "email": "badawy@gmail.com"
	}</pre>
		<pre>{
		  Registration successful. Please check your email for verification.
		}</pre>

	Registers a new landlord account by accepting user details	

GET	https://api.semsary.site/api/Auth/GetUserInfo	
	Request body	**Response body**

		<pre>{
		  "firstname": "my_first_name",
		  "lastname": "my_last_name",
		  "username": "01JXQ2E2TRDWXEKASQHNEJDYH6",
		  "emails": [
		  "mostafaelgelani@gmail.com"
		  ],
		  "address": null,
		  "userType": 2
		}</pre>

	Retrieves the authenticated user's profile information.	

POST	https://api.semsary.site/api/Auth/resetpassword	
	Request body	**Response body**

	<pre>{
	  "email": "badawy@gmail.com",
	  "otp": "524898",
	  "password": "Password 123"
	}</pre>
		password updated successfully

	Initiates the password reset process for a user.	

GET	https://api.semsary.site/api/Auth/UpdatePasword?email= cados44532%40forcrack.com	
	Request body	**Response body**
		an otp was sent to your email
	Updates the password for the user associated with the specified email address.	

POST	https://api.semsary.site/api/Auth/Customerservice/Add	
	Request body	**Response body**

	{
	
	"**email**":"CustomerService@gmail.com",
	"**firstname**": "customer",
	"**lastname**": "service",
	"**password**": "Password 123"
	}
		customer service was created successfully

	Adds a new customer service request or message.	

GET	https://api.semsary.site/api/Auth/GetVerifictioncode?email=CustomerService%40gmail.com	
	Request body	**Response body**
		verification code was sent to your email
	Sends or retrieves a verification code for the specified email address.	

POST	https://api.semsary.site/api/Auth/Customerservice/Add	
	Request body	**Response body**

	{
	
	"**email**":"CustomerService@gmail.com",
	"**firstname**": "customer",
	"**lastname**": "service",
	"**password**": "Password 123"
	}
		customer service was created successfully

	Adds a new customer service request or message.	

POST	https://api.semsary.site/api/Auth/SubmitIdentity	
	Request body	**Response body**

	{
	"**UserImage**":"image",
	"**IdentityFront**": "image",
	"**IdentityBack**": "image"
	}
		identity was submitted successfully

	Submits user identity verification information.	

GET	https://api.semsary.site/api/Auth/GetAllIdentity	
	Request body	**Response body**

		[
		{
		"ownerUsername": "01JXQ2E2TRDWXEKASQHNEJDYH6",
		"id": "01JXQHFKDA3R3NNTDAXY6H3Z95",
		"ownerFirstName": "my_first_name",
		"ownerLastName": "my_last_name",
		"imageURLS": [
		"https://pub-7525d29.r2.dev/01JXQHEZ1M1TNDH9M2EBYSY4ST.jpg",
		"https://pub-87525d29.r2.dev/01JXQHF0DRDAVPQZJKYBTV7P4B.jpg",
		"https://pub-d29.r2.dev/01JXQHF1AZT7SXRWE9RPS6XEWB.jpg"
		],
		"submitedAt": "2025-06-14T15:38:40.4263303Z"
		}
		]

	Retrieves identity verification records for all users.	

POST	https://api.semsary.site/api/Auth/ReviewIdentity?id=01JXQHFKDA3R3NNTDAXY6H3Z95&status=1&comment=this%20is%20my%20comment	
	Request body	**Response body**

	{
	"id":"01JXQHFKDA3R3NNTDAXY6H3Z95",
	"status": "1",
	"comment": "this is my comment"
	}
		identity was reviewed successfully

	Reviews a user's identity submission by updating its verification status. It accepts the following query	

PUT	https://api.semsary.site/api/Auth/Edit/Profile	
	Request body	**Response body**

	{
	"firstname": "string",
	"lastname": "string",
	"address": {
	"id": 0,
	"_gover": 0,
	"_city": "string",
	"street": "string"
	},
	"profileImageUrl": "string",
	"height": 0,
	"gender": 0,
	"age": 0,
	"isSmoker": true,
	"needPublicService": true,
	"needPublicTransportation": true,
	"needNearUniversity": true,
	"needNearVitalPlaces": true
	}
		Profile updated successfully.

	Updates the authenticated user's profile information.	

GET	https://api.semsary.site/api/Chat/User/Info/01JXQ2E2TRDWXEKASQHNEJDYH6	
	Request body	**Response body**

		{
		"firstName": "Moahmed",
		"lastName": "Badawy",
		"profilePic": null
		}

	Retrieves chat-related information for a specific user identified by their unique `username`.	

GET	https://api.semsary.site/api/Auth/Auth/Me	
	Request body	**Response body**

		{
		"basicUserInfo": {
		"firstname": "customer",
		"lastname": "service",
		"profileImageUrl": null,
		"emails": ["cados44532@forcrack.com"],
		"address": null,
		"userType": 3
		},
		"otherTenantData": null,
		"otherLanlordData": null
		}

	Returns the details of the currently authenticated user.	

POST	https://api.semsary.site/api/LandLord/create/house	
	Request body	**Response body**

	{
	"address": {
	"id": 0,
	"_gover": 0,
	"_city": "string",
	"street": "string"
	}
	}
		{
		"message": "House created successfully",
		"houseId": "01JXQN1VVKM1C28HKVZ02MTJ65"
		}

	Allows a landlord to create a new house listing.	

POST	https://api.semsary.site/api/LandLord/inspection/request/01JXQMXEV2SAQAZQM84101GJNY	
	Request body	**Response body**

	{
	"HouseId":"01JXQMXEV2SAQAZQM84101GJNY"
	}
		{
		"message": "Inspection requested successfully"
		}

	Submits a request for inspection of a specific house, identified by its `houseId`.	

GET	https://api.semsary.site/api/LandLord/inspection/get/01JXQMXEV2SAQAZQM84101GJNY	
	Request body	**Response body**

	{
	"HouseId":"HJLKFD456JKGFD759"
	}	

	Retrieves the inspection details for a specific house identified by `houseId`.	

GET	https://api.semsary.site/api/LandLord/Houses/GetAll	
	Request body	**Response body**

		{
		"notInspectedHouses": [
		{
		"houseId": "01JXQN1VVKM1C28HKVZ02MTJ65",
		"governorate": 0,
		"city": "Luxor",
		"street": "string"
		}
		],
		"inspectedHouses": []
		}

	Retrieves a list of all houses created by the currently authenticated landlord.	

PUT	https://api.semsary.site/api/LandLord/inspection/approve/01JXQN1VVKM1C28HKVZ02MTJ65	
	Request body	**Response body**

	{
	"HouseId":""
	}
		Profile updated successfully.

	Approves the inspection request for a specific property, identified by `inspectionId`.	

GET	https://api.semsary.site/api/LandLord/get/house/01JXQN1VVKM1C28HKVZ02MTJ65	
	Request body	**Response body**

	{
	"HouseId":"01JXQN1VVKM1C28HKVZ02MTJ65"
	}
		{
		"houseId": "01JXQN1VVKM1C28HKVZ02MTJ65",
		"landlordUsername":
		"01JXQ2E2TRDWXEKASQHNEJDYH6",
		"governorate": 0,
		"city": "string",
		"street": "string",
		"rates": [],
		"comments": null,
		"houseInspections": [],
		"advertisements": [],
		"rentals": null
		}

	Retrieves detailed information about a specific house listed by a landlord, using the house's unique `houseId`.	

PUT	https://api.semsary.site/api/LandLord/rentalrequest/0?status=1	
	Request body	**Response body**

	{
	"RentalId":""
	"status":""
	}	

	Updates the status of a specific rental request.	

POST	https://api.semsary.site/api/LandLord/CreateAdvertisement	
	Request body	**Response body**

	{
	"houseId": "01JXQMXEV2SAQAZQM84101GJNY",
	"rentalType": 0,
	"monthlyCost": 0,
	"dailyCost": 0
	}	

	Creates a new advertisement for a house listed by a landlord.	

POST	https://api.semsary.site/api/Tenant/Make/Complaint/0	
	Request body	**Response body**

	{
	"RentalId" :"",
	"complaintDetails": "string"
	}	

	Allows a tenant to submit a complaint related to a specific rental request.	

POST	https://api.semsary.site/api/Tenant/Set/Rate/01JXQN1VVKM1C28HKVZ02MTJ65	
	Request body	**Response body**

	{
	"houseId": "",
	"starsNumber":""
	}	

	Allows a tenant to submit a rating or review for a specific house, identified by `houseId`.	

PUT	https://api.semsary.site/api/LandLord/arrivalrequest/0?rentalStatus=1	
	Request body	**Response body**

	{
	"RentalId":\""
	"RentalStatus":""
	}	

	Updates the **arrival status** of a tenant related to a rental request.	

POST	https://api.semsary.site/api/Tenant/Add/Comment/01JXQMXEV2SAQAZQM84101GJNY	
	Request body	**Response body**

	{
	"houseId": "",
	"commentDetails":""
	}	

	Allows a tenant to add a public comment or review on a specific house, identified by `houseId`.	

POST	https://api.semsary.site/api/Tenant/Make/Rental/Request	
	Request body	**Response body**

	{
	"startDate": "2025-06-14T19:09:33.168Z",
	"endDate": "2025-06-14T19:09:33.168Z",
	"rentalType": 0,
	"houseId": "string",
	"startArrivalDate": "2025-06-14T19:09:33.168Z",
	"endArrivalDate": "2025-06-14T19:09:33.168Z",
	"rentalUnitIds": [
	"string"
	]
	}	

	Allows a tenant to submit a rental request for a specific property.	

POST	https://api.semsary.site/api/Tenant/Cancel/rental/Request/3	
	Request body	**Response body**

	{
	"RentalId" :""
	}	

	Allows a tenant to cancel a previously submitted rental request, identified by `rentalId`.	

PUT	https://api.semsary.site/api/CustomerService/HouseInspection/create/01JXQN1VVKM1C28HKVZ02MTJ65	
	Request body	**Response body**

	{
	"floorNumber": 0,
	"numberOfAirConditionnar": 0,
	"numberOfPathRooms": 0,
	"numberOfBedRooms": 0,
	"numberOfBeds": 0,
	"numberOfBalacons": 0,
	"numberOfTables": 0,
	"numberOfChairs": 0,
	"houseFeature": 0,
	"houseImages": [
	"string"
	]
	}	

	Creates a new house inspection record for the specified house, identified by `houseId`.	

POST	https://api.semsary.site/api/CustomerService/BlockLandlord/123	
	Request body	**Response body**

	{
	"reason": "string"
	}	

	Blocks a specific landlord, identified by `landlordId`, from performing further actions on the platform.	

Chapter 6

Evaluation Techniques & Results

6.1 Evaluation Techniques

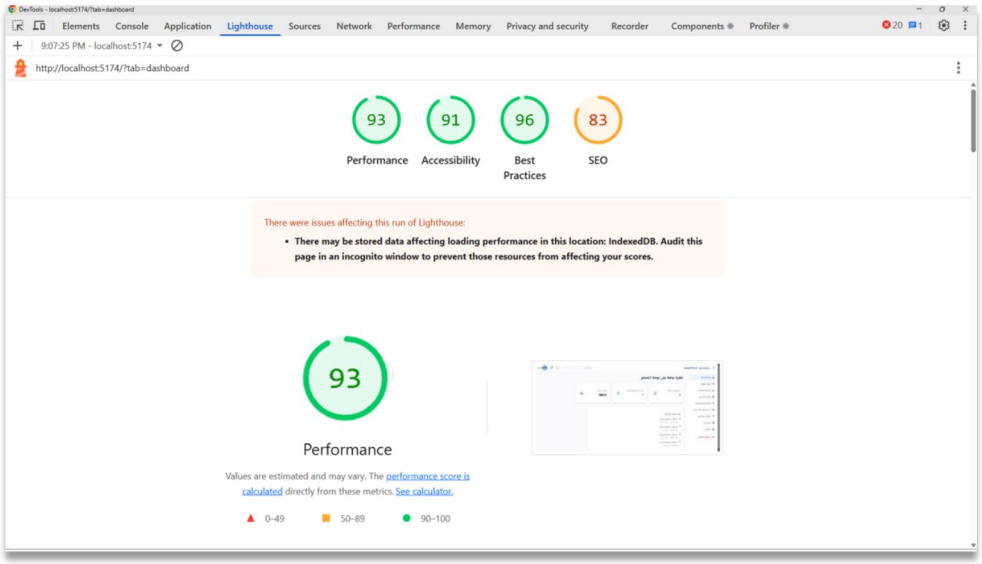
This section presents the evaluation methods and results used for assessing the performance, accessibility, and best practices of the Semsary web platform. The objective is to ensure the platform maintains high standards of speed, user experience, scalability, and reliability.

- **Front-End Evaluation:**
  We used **Google Lighthouse**, a powerful automated tool provided by Google to audit the quality of web pages. Lighthouse evaluates key performance indicators such as load speed, accessibility, SEO, and adherence to best practices. It generates detailed reports that guide improvements in front-end code and user experience.
- **Back-End Evaluation:**
  For server-side evaluation, we utilized tools such as **Apache JMeter** to assess performance under load, **OWASP** for security vulnerability scanning, and **Firebase Monitoring** for real-time performance insights. Additionally, **SQL Server Profiler** was used to monitor database interactions and ensure optimal query execution.

6.2 Front-End Results

Below are the front-end evaluation results of the Semsary platform using Google Lighthouse. The platform scored highly across all core categories, reflecting its responsiveness, accessibility, and user-centric design.

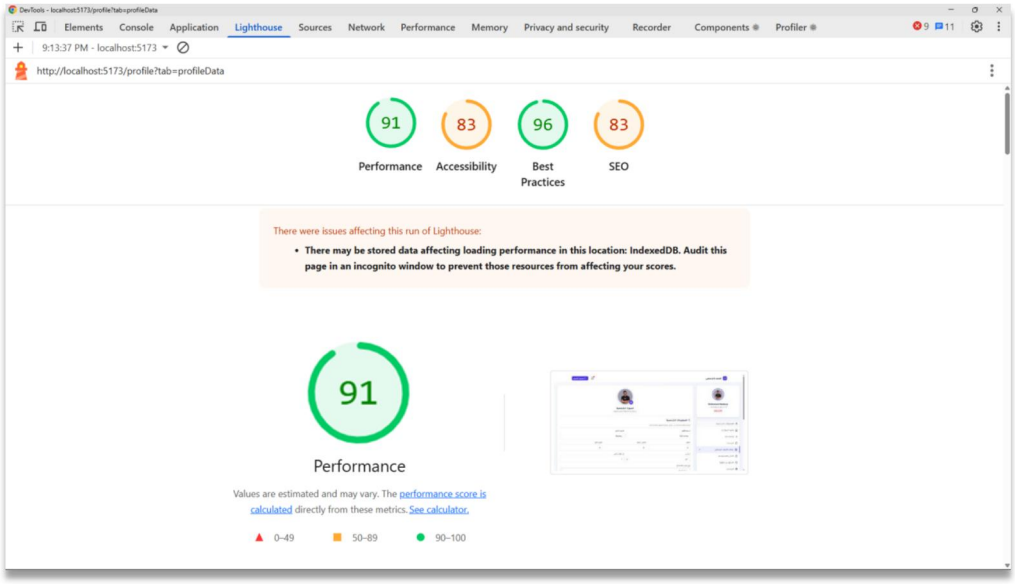
6.2.1 Results of Dashboard



Dashboard Google Lighthouse Testing				
Metric	Performance	Accessibility	Best Practices	SEO
Score	**93**	**91**	**96**	**83**

Figure 75: Dashboard Google Lighthouse Testing

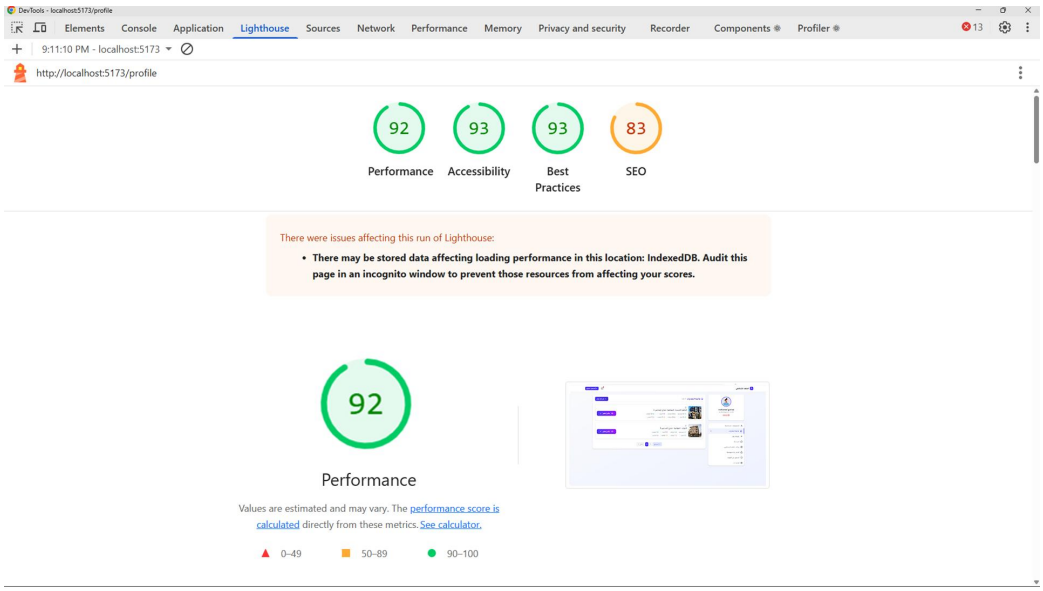
6.2.2 Results of Profile



Profile page Google Lighthouse Testing				
Metric	Performance	Accessibility	Best Practices	SEO
Score	**91**	**83**	**96**	**83**

Figure 76: Profile page Google Lighthouse Testing

6.2.3 Results of User Apartments



User Apartments Google Lighthouse Testing				
Metric	Performance	Accessibility	Best Practices	SEO
Score	**92**	**93**	**93**	**83**

Figure 77: User Apartments Profile Lighthouse Testing

6.2.1 Performance

The **Performance** score is a critical indicator that measures how quickly and efficiently the Semsary platform loads and becomes usable for users. Evaluated using **Google Lighthouse**, the performance score aggregates several key web vitals that directly impact user experience and satisfaction.

The Semsary platform achieved an impressive **performance score of 96**, indicating excellent front-end responsiveness and speed. Below are the primary metrics considered in this evaluation:

- **First Contentful Paint (FCP):**
  Measured at 1,650 ms
  This metric indicates the time at which the browser first renders any visible content. A fast FCP reassures users that the page is loading correctly.
- **Time to Interactive (TTI):**
  Measured at 3,250 ms
  TTI measures how long it takes for the page to become fully interactive. A lower TTI ensures users can interact with the page quickly without lag.
- **Speed Index:**
  Measured at 1,720 ms
  This metric reflects how quickly the page's content is visually displayed. A lower value results in a faster perceived load time.
- **First CPU Idle:**
  Measured at 3,200 ms
  Indicates the first point when the page's main thread is sufficiently idle to handle input. It ensures responsiveness during initial interaction.
- **Estimated Input Latency:**
  Measured at 12 ms
  A measure of how quickly the site responds to user input. Lower latency results in a smoother and more responsive interface.

Although **Total Blocking Time (TBT)** and **Largest Contentful Paint (LCP)** were not explicitly reported in this test, they are inherently considered in the overall score, and their influence is reflected in the high performance achieved.

These results confirm that the Semsary platform delivers an optimized and fast-loading experience, which is essential for maintaining user engagement and satisfaction, particularly in a real estate platform where users frequently navigate between listings.

6.2.2 Progressive Web App (Pwa)

The PWA (Progressive Web App) score in Google Lighthouse is a metric that measures how well a website meets the criteria of being a progressive web app.

A progressive web app should feel and behave like a native mobile app, providing features such as push notifications, home screen installation, and full-screen mode.

The PWA score is based on several factors:

- If the site is Fast and reliable

- If the site is installable

- If the site is PWA Optimized.

6.2.3 Accessibility

The accessibility audits cover a range of areas, such as proper use of headings, alternative text for images, keyboard navigation, color contrast, and more. This can help you identify and address accessibility issues to ensure your website is accessible to all users.

6.2.4 Best Practices

The Best Practices score checks the quality of the platform code, and whether it meets industry standards. This involves ensuring the page is fast, and secure and creates a good user experience. This score also takes into account any deprecated technologies (such as APIs).

6.3 Back-End Results

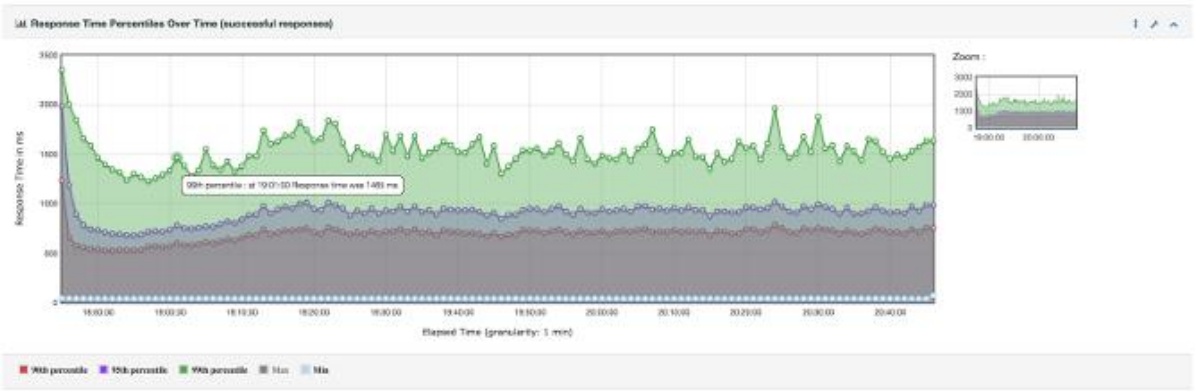


Figure 78: Performance Monitoring - Apache JMeter Report

6.3.1 Performance Monitoring

Description: Evaluates the responsiveness and stability of the platform under various loads. This includes load testing to simulate high traffic and response time measurements to assess how quickly the backend handles requests.

6.3.2 security testing

1. A01:2021 – Broken Access Control

Issue in Semsary: Unauthorized access to admin or landlord functions (e.g., publishing houses, managing complaints) if role-based authorization is not strictly enforced.

Impact: Attackers could perform admin-level operations such as deleting listings, changing prices, or viewing private data.

Mitigation: Implement strict role-based access control (RBAC) and verify permissions server-side for every sensitive action.

2. A02:2021 – Cryptographic Failures

Issue in Semsary: Insufficient encryption for sensitive data such as passwords, user identity documents, or payment data. Lack of HTTPS or TLS enforcement across all endpoints.

Impact: User data could be intercepted or tampered with during transmission. Could lead to identity theft, fraud, and platform distrust.

Mitigation: Enforce HTTPS/TLS site-wide. Store passwords using secure hashing algorithms (bcrypt). Encrypt sensitive data at rest and in transit.

3. A03:2021 – Security Misconfiguration

Issue in Semsary: Using default .NET or SQL Server configurations, or not restricting access to Firebase storage URLs.

Impact: Attackers could access internal API documentation, exposed endpoints, or stored apartment images/documents directly.

Mitigation: Lock down storage buckets with IAM policies. Disable unnecessary services and debug/error messages in production. Use security headers like CSP, X-Frame-Options.

4. A04:2021 – Identification and Authentication Failures

Issue in Semsary: Weak login system without rate-limiting, OTP validation bypass, or insecure JWT implementation.

Impact: Account takeover through brute-force or replay attacks. Impersonation of landlords or tenants.

Mitigation: Implement multi-factor authentication (MFA). Use secure, short-lived JWTs with strong signing algorithms. Rate-limit login and registration attempts.

5. A05:2021 – Software and Data Integrity Failures

Issue in Semsary: Use of invalidated or outdated open-source libraries (e.g., React, Tailwind, Firebase SDKs) or third-party APIs (like PayMob, Twilio).

Impact: May introduce known vulnerabilities, leading to arbitrary code execution or data leakage.

Mitigation: Use dependency scanning tools (e.g., Dependabot). Validate third-party APIs and restrict their scope with API keys and rate limits.

6. A06:2021 – Server-Side Request Forgery (SSRF)

Issue in Semsary: If image upload, feedback, or verification systems fetch remote URLs (e.g., for image validation or data parsing), SSRF may be possible.

Impact: Could allow attackers to make internal server requests and access protected resources.

Mitigation: Block internal IP ranges in requests. Validate and sanitize all URLs before requesting them server-side.

7. A07:2021 – Remote Code Execution (RCE)

Issue in Semsary: In Semsary, users (landlords or tenants) can upload images of rental houses. However, if the platform does not strictly validate the file type, MIME type, and size of uploaded content, an attacker can upload a malicious file disguised as an image.

Impact:

Remote Code Execution (RCE):
If the server stores and serves the uploaded files without isolating them (e.g., on Firebase or CDN), and then executes them in a vulnerable environment (e.g., shared hosting or misconfigured CDN), the attacker can execute arbitrary commands.

Defacement or Takeover:
Attackers may upload files that contain malicious scripts to be executed in the browser (XSS), or even backdoors on misconfigured servers.

Denial of Service (DoS):
Uploading extremely large files can exhaust disk space or crash the image handling service.

Bypass Business Logic:
Uploading non-image content may interfere with image previews or processing logic, leading to app crashes or unpredictable behavior.

Mitigation Recommendations:

Server-side validation:

Check MIME type and extension (only allow image.png, image.jpeg, image.webp).

Reject or sanitize files with dual extensions (file.php.jpg, file.exe.png).

Reject files larger than a reasonable size (2 MB).

Strip EXIF data from images before saving.

Isolation and sandboxing:

Store files in object storage (Firebase Storage or S3), not on the server file system.

Use Content-Disposition: attachment when serving uploads to prevent browser execution.

Scanning:

Use libraries like Image Sharp (.NET) or Pillow (Python) to verify image integrity and sanitize them.

Run uploaded files through a virus scanner (ClamAV).

Logging & Alerting:

Log file upload activities, including type, size, and user ID.

Alert on suspicious uploads or repeated failures.

8. A08:2021 – Rate Limit

Issue: Semsary lacks rate limiting on the comment/review system, allowing users to post unlimited comments in a short time without restrictions on frequency or volume.

Impact:

Spam & Review Manipulation: Attackers can flood listings with fake positive or negative reviews.

System Resource Abuse: Excessive comment submissions can increase server load and database bloat.

Reputation Damage: Malicious users can unfairly degrade landlords' credibility or inflate ratings.

Reduced Platform Trust: Tenants may distrust reviews due to spam or manipulation.

Mitigation:

Implement backend rate limits (1 comment per listing per 10 minutes).

Enforce user-level and IP-based thresholds (10 comments/hour).

Add frontend cool-downs and disable comment buttons after submission.

Use logging and alerting to detect abuse patterns or bots.

Optionally, add CAPTCHA for suspicious behavior.

Figure 79: Performance Monitoring - SolarWinds Report

6.3.3 Database Performance

Description: Evaluates the responsiveness and stability of the database under various loads. This includes load testing to simulate high traffic and response time measurements to assess how quickly the backend handles requests.

Chapter 7

Conclusions & Future Work

7.1 Conclusions

The Management Intelligent System for Managing Individuals' Residence in Cities **(Semsary)** represents a groundbreaking solution to the challenges faced by tenants, landlords, and urban communities in the rental market. By leveraging advanced technologies such as .NET for backend development, React for a responsive and intuitive frontend, and Python for machine learning integrations, Semsary has successfully created a platform that streamlines the rental process, enhances transparency, and fosters trust among users. This project is not just a technological achievement but also a step toward building more sustainable and equitable urban communities.

One of the key strengths of Semsary lies in its ability to centralize rental listings and provide users with a comprehensive, data-driven approach to finding and managing properties. The platform's advanced search and filtering capabilities allow tenants to easily find accommodations that match their preferences, whether they are looking for proximity to schools, transportation, or specific amenities. For landlords, Semsary offers a powerful tool to market their properties, reach a wider audience, and ensure fair pricing based on real-time market trends. The integration of machine learning for price recommendations ensures that both tenants and landlords benefit from transparent and competitive pricing, reducing the risk of exploitation and fostering a fair rental ecosystem.

Another significant achievement of Semsary is its focus on security and reliability. The platform incorporates robust fraud prevention mechanisms, verified reviews, and secure payment options, ensuring that users can engage in transactions with confidence. The dynamic administrative layer further enhances the platform's reliability by enabling managers to handle advertising approvals, resolve disputes, and monitor tenant reports. This multi-layered approach to security and oversight ensures that Semsary remains a trustworthy platform for all stakeholders.

Semsary also addresses some of the most pressing issues in the rental market, such as the lack of transparency, broker exploitation, and the difficulty of finding short-term, flexible rentals. By providing a centralized platform that consolidates reliable listings, automates price assessments, and offers location-based insights, Semsary significantly reduces the time and effort required for tenants to find suitable accommodations. For landlords, the platform provides an efficient way to manage properties, respond to rental requests, and optimize pricing strategies based on market trends. This dual focus on tenant and landlord needs ensures that Semsary creates a balanced and sustainable rental ecosystem.

Moreover, Semsary's emphasis on user experience is evident in its intuitive interface, streamlined workflows, and personalized features. The platform's ability to adapt to user preferences, such as filtering properties by proximity to essential services or offering tailored recommendations, ensures that users can navigate the platform with ease. The inclusion of verified reviews and feedback mechanisms further enhances the user experience by providing tenants with reliable information to

make informed decisions. For landlords, the platform's tools for managing listings, tracking rental requests, and receiving pricing recommendations simplify the process of renting out properties, making it more accessible and efficient.

The project also aligns with broader societal goals, such as promoting sustainable urban development and community well-being. By making housing access more equitable and transparent, Semsary contributes to the creation of smarter, more sustainable cities. The platform's focus on reducing friction in the rental process, ensuring fair pricing, and fostering trust among users supports the United Nations' Sustainable Development Goals (SDGs), particularly those related to sustainable cities and communities and reduced inequalities . In this way, Semsary is not just a technological solution but also a tool for social and economic progress.

In conclusion, Semsary represents a significant step forward in the field of urban rental management. Its innovative features, user-centric design, and commitment to transparency and security make it a valuable resource for tenants, landlords, and urban communities. The successful implementation of this project demonstrates the potential of technology to address real-world challenges and improve the quality of life for individuals and communities. As Semsary continues to evolve, it has the potential to set new standards for rental platforms, not only in Egypt but also in other regions facing similar challenges. By fostering a more accessible, trustworthy, and sustainable rental market, Semsary is poised to make a lasting impact on urban living and contribute to the development of smarter, more inclusive cities.

7.2 Future work

While **Semsary** has achieved its primary objectives, there are several areas for future development and enhancement to further improve the platform's functionality, scalability, and user experience. These future directions aim to ensure that Semsary remains at the forefront of rental management solutions and continues to adapt to the evolving needs of the market.

Expansion to Other Regions and Markets: Currently, Semsary is tailored for the Egyptian market. Future work could involve expanding the platform to other regions or countries, adapting to local rental laws, market trends, and user preferences. This expansion would require localization efforts, including language support, currency conversion, and compliance with regional regulations.

Enhanced Machine Learning Models: The current machine learning model provides pricing recommendations based on market trends and property features. Future iterations could incorporate more advanced algorithms, such as **deep learning**, to improve the accuracy of price predictions and offer personalized recommendations based on user behavior and preferences. Additionally, the model could be expanded to predict rental demand, helping landlords optimize their pricing strategies.

AI-Powered Customer Support: Integrating AI-driven chatbots or virtual assistants could improve customer service by providing instant responses to user queries, resolving common issues, and guiding users through the platform. These AI tools could also be used to automate routine tasks, such as answering frequently asked questions or processing rental requests, freeing up human customer service agents to handle more complex issues.

Advanced Analytics for Landlords: Providing landlords with more detailed analytics and insights, such as rental demand forecasts, tenant demographics, and market trends, could help them make more informed decisions about their properties. These insights could be presented in an easy-to-understand dashboard, allowing landlords to track the performance of their listings and adjust their strategies accordingly.

Enhanced User Experience through Personalization: Future versions of Semsary could leverage user data to offer personalized recommendations and tailored experiences. For example, the platform could use machine learning to suggest properties based on a tenant's past searches, preferences, and behavior. Similarly, landlords could receive personalized tips on how to improve their listings based on market trends and tenant feedback.

Integration with Financial Services: Semsary could explore partnerships with financial institutions to offer additional services, such as rental insurance, financing options for tenants, or investment opportunities for landlords. These services would provide added value to users and create new revenue streams for the platform.

Enhanced Security Measures: As cyber threats continue to evolve, Semsary must stay ahead by implementing advanced security measures. This could include multi-factor authentication, biometric verification, and regular security audits to protect user data and ensure the platform remains secure.

Partnerships with Local Governments and NGOs: Collaborating with local governments and non-governmental organizations (NGOs) could help Semsary address broader housing challenges, such as affordable housing and homelessness. By working with these organizations, Semsary could contribute to initiatives that promote equitable access to housing and support vulnerable populations.

References

1. **React.js Documentation.** *[Online]*. URL: https://reactjs.org/docs/getting-started.html (Available at: January 21, 2025)
2. **Tailwind CSS Documentation.** *[Online]*. URL: https://tailwindcss.com/docs (Available at: October 15, 2024)
3. **ASP.NET Core Documentation.** *[Online]*. URL: https://docs.microsoft.com/en-us/aspnet/core/ (Available at: December 1, 2024)
4. **SQL Server Documentation.** *[Online]*. URL: https://docs.microsoft.com/en-us/sql/sql-server/ (Available at: January 21, 2025)
5. **Python Documentation.** *[Online]*. URL: https://docs.python.org/3/ (Available at: October 15, 2024)
6. **TensorFlow Documentation.** *[Online]*. URL: https://www.tensorflow.org/overview (Available at: December 1, 2024)
7. **Firebase Documentation.** *[Online]*. URL: https://firebase.google.com/docs (Available at: January 21, 2025)
8. **SendGrid Documentation.** *[Online]*. URL: https://docs.sendgrid.com/ (Available at: October 15, 2024)
9. **PayMob Documentation.** *[Online]*. URL: https://docs.paymob.com/ (Available at: December 1, 2024)
10. **Twilio Documentation.** *[Online]*. URL: https://www.twilio.com/docs (Available at: January 21, 2025)
11. **Google Analytics Documentation.** *[Online]*. URL: https://developers.google.com/analytics (Available at: October 15, 2024)
12. **Postman Documentation.** *[Online]*. URL: https://learning.postman.com/docs/ (Available at: December 1, 2024)
13. **Agile Development Methodology.** *[Online]*. URL: https://www.agilealliance.org/agile101/ (Available at: January 21, 2025)
14. **Machine Learning Best Practices.** *[Online].* URL: https://developers.google.com/machine-learning/guides/rules-of-ml (Available at: October 15, 2024)
15. **Airbnb.** *[Online]*. URL: https://www.airbnb.com (Available at: December 1, 2024)
16. **Property Finder Egypt.** *[Online].* URL: https://www.propertyfinder.eg (Available at: January 21, 2025)
17. **Aqarmap.** *[Online].* URL: https://aqarmap.com.eg (Available at: October 15, 2024)
18. **Bayut.** *[Online].* URL: https://www.bayut.com (Available at: December 1, 2024)
19. **Aqary Masr**. *[Online].* URL: https://aqaryamasr.com (Available at: January 21, 2025)

سمسار

Graduation Project

2024/2025